OKS Grammar, Query Language & C++ API

A Deep-Dive Reference for Understanding and Using OKS

Built from full verbatim source extracts of the DUNE-DAQ **oks**/**dal** repositories and the original CERN ATLAS TDAQ GitLab sources, focused specifically on: the query grammar implementation, the C++ API surface, the DAL config-version pattern, and the GUI/CLI tools used to actually work with OKS. Large boilerplate (full production schemas, raw HTML help trees, historical Wayback snapshots, Python build scripts) has been trimmed or summarised in favour of the material that directly explains grammar/query/API usage.

Prepared for: Minor Project — CERN-focused Intelligent Agent for Physics Data Queries (Asmit, B.E. Computer Engineering)

Table of Contents

1. Scope of This Document — What Was Kept, What Was Trimmed

This reference is built from five newly-harvested source files (~166,000 words of verbatim code/HTML/markdown, generated by automated extraction renderers against local git clones and Wayback Machine snapshots). Rather than reproducing every byte, this document keeps everything directly relevant to understanding **OKS's grammar, its query language, its C++ API, and how to actually use it**, and trims or summarises material that doesn't serve that purpose:

- **Kept in full or near-full**: the query grammar tokens and parser source, the complete public C++ API declarations for every core class (OksQuery, OksKernel, OksClass, OksObject, OksAttribute, OksRelationship, OksIndex, OksFile), all CLI tool usage banners, the Python bindings surface, the 16 worked C++ examples, the canonical oks_tutorial.cpp, the OKS Data Editor Query Window manual, the DAL config-version pattern and tutorial, and the **complete** CERN oks RELEASE_NOTES.md history (tdaq-01 through tdaq-13).
- **Trimmed or summarised**: the full 83 KB production `core.schema.xml` (already excerpted in the earlier report), the 51 KB original ATLAS Twiki text dump, most of the raw OKS Data Editor HTML help pages (kept only Query Window, the one that documents the query grammar), the raw Python bindings source bodies (kept the exposed-surface summary instead), the swrod/oks2coral schema and archiver source (already covered structurally in the earlier report), and the ~350 KB of individual Wayback Machine HTML snapshot dumps (kept only the compiled 'Observations' summary).
- **Excluded entirely, per your instruction**: the Python build/renderer scripts (`build_02.py`, `build_03.py`, `build_05.py`, `check_05.py`, `download_pcatd12.py`) — these are your own scraping tooling, not OKS content, and are irrelevant to understanding the grammar/query/API.

2. The Query Grammar — Reserved Tokens (verbatim)

```
const char * OksQuery::OR = "or";
const char * OksQuery::AND = "and";
const char * OksQuery::NOT = "not";
const char * OksQuery::SOME = "some";
const char * OksQuery::THIS_CLASS = "this";
const char * OksQuery::ALL_SUBCLASSES = "all";
const char * OksQuery::OID = "object-id";
const char * OksQuery::EQ = "=";
const char * OksQuery::NE = "!=";
const char * OksQuery::RE = "~=";
const char * OksQuery::LE = "<=";
const char * OksQuery::GE = ">=";
const char * OksQuery::LS = "<";
const char * OksQuery::GT = ">";
const char * OksQuery::PATH_TO = "path-to";
const char * OksQuery::DIRECT = "direct";
const char * OksQuery::NESTED = "nested";
```

The seven comparators (`=`, `!=`, `~=`, `<=`, `>=`, `<`, `>`) bind to function pointers in the same file (lines 56-64, verbatim):

```
bool OksQuery::equal_cmp(const OksData *d1, const OksData *d2) {return (*d1 == *d2);}
bool OksQuery::not_equal_cmp(const OksData *d1, const OksData *d2) {return (*d1 != *d2);}
bool OksQuery::less_or_equal_cmp(const OksData *d1, const OksData *d2) {return (*d1 <= *d2);}
bool OksQuery::greater_or_equal_cmp(const OksData *d1, const OksData *d2) {return (*d1 >= *d2);}
bool OksQuery::less_cmp(const OksData *d1, const OksData *d2) {return (*d1 < *d2);}
bool OksQuery::greater_cmp(const OksData *d1, const OksData *d2) {return (*d1 > *d2);}
bool OksQuery::reg_exp_cmp(const OksData *d, const OksData * re) {
  return boost::regex_match(d->str(), *reinterpret_cast<const boost::regex *>(re));
}
```

Regex comparator uses **boost::regex** (implemented via lazy compilation in `OksObject::SatisfiesQueryExpression`).

---

3. The Query String Parser (src/query.cpp)

Top-level grammar enforced by `OksQuery::OksQuery(const OksClass*, const std::string&)` (lines 90-183). Rules, verbatim:

```
// the first token must be 'all' or 'this'
  if(s.substr(0, p) == OksQuery::ALL_SUBCLASSES)
    p_sub_classes = true;
  else if(s.substr(0, p) == OksQuery::THIS_CLASS)
    p_sub_classes = false;
  else {
    Oks::error_msg(fname)
      << "Can't parse query expression \"" << str << "\"\n"
         "the first token must be \'"<< OksQuery::ALL_SUBCLASSES
      << "\' or \'"<< OksQuery::THIS_CLASS << "\'\n";
    return;
  }
```

The recursive-descent expression parser is `OksQuery::create_expression(const OksClass*, const std::string&)` (lines 186-431). Tokenizer: splitting on spaces, with quoted strings `"` / `'` / `` ` `` as delimiters, and nested parens balanced with a depth counter `(lines 214-274)`. The grammar in code form:

```
  const std::string first = slist.front();
  slist.pop_front();

  if(
   first == OksQuery::AND ||
   first == OksQuery::OR
  ) {
    if(slist.size() < 2) { /* error: 'and'/'or' must have two or more args */ }
    qe = ((first == OksQuery::AND)
            ? (OksQueryExpression *)new OksAndExpression()
            : (OksQueryExpression *)new OksOrExpression());
    while(!slist.empty()) { ... qe2 = create_expression(c, item2); ... }
  }
  else if(first == OksQuery::NOT) {
    if(slist.size() != 1) { /* error: 'not' must have exactly one command */ }
    qe = new OksNotExpression(); ...
  }
  else if(slist.size() != 2) { /* error */ }
  else {
    const std::string second = ...; const std::string third = ...;
    if(second == OksQuery::SOME || second == OksQuery::ALL_SUBCLASSES) {
      // relationship expression: "name some/any subquery"
      OksRelationship *r = c->find_relationship(first);
      ...
      qe = new OksRelationshipExpression(r, qe2, b);   // b = true for 'all'
    }
    else {
      // comparator expression: name value op
      OksAttribute *a = ((first != OksQuery::OID) ? c->find_attribute(first) : 0);
      ...
      OksQuery::Comparator f = (
        (third == OksQuery::EQ) ? OksQuery::equal_cmp :
        (third == OksQuery::NE) ? OksQuery::not_equal_cmp :
        (third == OksQuery::RE) ? OksQuery::reg_exp_cmp :
        (third == OksQuery::LE) ? OksQuery::less_or_equal_cmp :
        (third == OksQuery::GE) ? OksQuery::greater_or_equal_cmp :
        (third == OksQuery::LS) ? OksQuery::less_cmp :
        (third == OksQuery::GT) ? OksQuery::greater_cmp :
        0);
      if(a) {
        if(f == OksQuery::reg_exp_cmp) {
          d->type = OksData::string_type;
          d->data.STRING = new OksString(second);
        }
        else {
          d->type = OksData::unknown_type;
          d->SetValues(second.c_str(), a);
        }
      }
      else {
        d->Set(second);      // object-id comparison
      }
      ...
```

```
      qe = new OksComparator(a, d, f);
    }
  }
```

Relationship expressions pick up the target class from the relationship type: `create_expression(relc, third)` where `relc = c->get_kernel()->find_class(r->get_type())`.

```
 *  \par Example
 *
 *  The example of query is shown below:
 *    "(path-to "my-id@my-class" (direct "A" "B" (nested "N" (direct "X" "Y" "Z"))))"
 *
 *  The destination object is "my-id@my-class". The search can be started from any object of any class.
 *  In our example the start object has to have two relationships named "A" and "B".
 *  An object referenced via "A" and "B" should have relationship "N". In our example
 *  it is possible to lookup for path via nested objects linked via relationship "N".
 *  Finally all objects referenced via "N" should have relationships "X", "Y" and "Z".
 *  If the destination object is referenced by them, the path is found. The result of path
 *  query execution is list of objects between the start and the destination object.
```

Query Execution

`OksClass::execute_query(OksQuery*)` (src/query.cpp lines 435-544) — verbatim highlights:

- checks `sqe->CheckSyntax()`; on false prints `"Can't execute query \"...\""` and returns 0

- indexes: if class `p_indices` has an index on the comparator attribute, `OksIndex::find_all()` is used (indexed search)

- for `and`/`or` with 2 comparator-type expressions on the same attribute, two-sided index lookup

- otherwise linear scan over `p_objects->begin()..end()`, calling `o->SatisfiesQueryExpression(sqe)`

- if `qe->search_in_subclasses()`, loops over `p_all_sub_classes`, same scan

- throws `oks::QueryFailed` wrapping any `oks::exception`/`std::exception`

`OksObject::SatisfiesQueryExpression` (lines 634-780): for comparator type, if attribute is nil compares `OksData d(GetId())` (object-ID query), else compares `data[offset]`. For relationship type: many-cardinality loops over list; `checkAllObjects==true` means ALL-of, false means SOME-of; single-value returns whether referenced object satisfies. Unbound references (uid2 in memory) throw `std::runtime_error` with verbatim text. And: all must satisfy; Or: any must satisfy; Not: negate.

`operator<<` (serialization back to string, lines 783-911) prints `(all|this (<expr>))` — this is the canonical printed query form.

---

4. QueryPath — Path Query Parser (path-to / direct / nested)

```
oks::QueryPath::QueryPath(const std::string& str, const OksKernel& kernel) : p_start(0)
{
  std::string s(str);
  erase_empty_chars(s);

  if(s.empty()) {
    throw oks::bad_query_syntax( "Empty query" );
  }

  if(s[0] == '(') {
    std::string::size_type p = s.rfind(')');
    if(p == std::string::npos)
      throw oks::bad_query_syntax(std::string("Query expression \'") + str + "\' must contain closing bracket");
    s.erase(p);  s.erase(0, 1);
  }
  else
    throw oks::bad_query_syntax(std::string("Query expression \'") + str + "\' must be enclosed by brackets");

  erase_empty_chars(s);

  Oks::Tokenizer t(s, " \t\n");
  std::string token;
  t.next(token);

  if(token != OksQuery::PATH_TO)
    throw oks::bad_query_syntax(std::string("Expression \'") + s + "\' must start from " + OksQuery::DIRECT + " or " + OksQuery::NESTI

  s.erase(0, token.size());
  erase_empty_chars(s);

  if( s[0] == '\"' ) {
    // destination object is "class@id"
    std::string::size_type p = s.find('\"', 1);
    if(p == std::string::npos)
      throw oks::bad_query_syntax(std::string("No trailing delimiter of object name in query \'") + str + "\'");

    std::string::size_type p2 = s.find('@');
    if(p2 == std::string::npos || p2 > p)
      throw oks::bad_query_syntax(std::string("Bad format of object name ") + s.substr(0, p+1) + " in query \'" + str + "\'");

    std::string object_id = std::string(s, 1, p2 - 1);
    std::string class_name = std::string(s, p2 + 1, p - p2 - 1);

    if(OksClass * c = kernel.find_class(class_name)) {
      if((p_goal = c->get_object(object_id)) == 0)
        throw oks::bad_query_syntax(std::string("Cannot find object ") + s.substr(0, p+1) + " in query \'" + str + "\': no such objec
    }
    else
      throw oks::bad_query_syntax(std::string("Cannot find object ") + s.substr(0, p+1) + " in query \'" + str + "\': no such class")

    s.erase(0, p + 1);
  }
  else
    throw oks::bad_query_syntax(std::string("No name of object in \'") + str + "\'");

  try { p_start = new QueryPathExpression(s); }
  catch ( oks::bad_query_syntax& e ) {
    throw oks::bad_query_syntax(std::string("Failed to parse expression \'") + str + "\' because \'" + e.what() + "\'");
  }
}
```

`QueryPathExpression` parser (lines 1129-1231): each element is `(direct|nested "rel1" "rel2" ... (nested-expr))`; relationship names **must** be quoted; nested expression parsed recursively; `p_use_nested_lookup` set true for `nested` (a path may look through recursively nested objects). The path search itself is in `OksObject::satisfies()` (lines 954-1052) — explores each relationship, checks whether it points at the goal object, then recurses; cycles prevented by checking `path` membership. `OksObject::find_path(const oks::QueryPath&)` returns newly-allocated `OksObject::List *` (0 if none).

---

5. Storage Format — XML DTDs (src/file.cpp)

Header + both DTDs:

```
const char OksFile::xml_file_header[]  = "<?xml version=\"1.0\" encoding=\"ASCII\"?>";

const char OksFile::xml_schema_file_dtd[] =
  "<!DOCTYPE oks-schema [\n"
  "  <!ELEMENT oks-schema (info, (include)?, (comments)?, (class)+)>\n"
  "  <!ELEMENT info EMPTY>\n"
  "  <!ATTLIST info\n"
  "      name CDATA #IMPLIED\n"
  "      type CDATA #IMPLIED\n"
  "      num-of-items CDATA #REQUIRED\n"
  "      oks-format CDATA #FIXED \"schema\"\n"
  "      oks-version CDATA #REQUIRED\n"
  "      created-by CDATA #IMPLIED\n"
  "      created-on CDATA #IMPLIED\n"
  "      creation-time CDATA #IMPLIED\n"
  "      last-modified-by CDATA #IMPLIED\n"
  "      last-modified-on CDATA #IMPLIED\n"
  "      last-modification-time CDATA #IMPLIED\n"
  "  >\n"
  "  <!ELEMENT include (file)+>\n"
  "  <!ELEMENT file EMPTY>\n"
  "  <!ATTLIST file path CDATA #REQUIRED>\n"
  ...
  "  <!ELEMENT class (superclass | attribute | relationship | method)*>\n"
  "  <!ATTLIST class name CDATA #REQUIRED description CDATA \"\" is-abstract (yes|no) \"no\">\n"
  "  <!ELEMENT superclass EMPTY>\n  <!ATTLIST superclass name CDATA #REQUIRED>\n"
  "  <!ELEMENT attribute EMPTY>\n"
  "  <!ATTLIST attribute\n"
  "      name CDATA #REQUIRED description CDATA \"\"\n"
  "      type (bool|s8|u8|s16|u16|s32|u32|s64|u64|float|double|date|time|string|uid|enum|class) #REQUIRED\n"
  "      range CDATA \"\" format (dec|hex|oct) \"dec\"\n"
  "      is-multi-value (yes|no) \"no\" init-value CDATA \"\"\n"
  "      is-not-null (yes|no) \"no\" ordered (yes|no) \"no\"\n"
  "  >\n"
  "  <!ELEMENT relationship EMPTY>\n"
  "  <!ATTLIST relationship\n"
  "      name CDATA #REQUIRED description CDATA \"\" class-type CDATA #REQUIRED\n"
  "      low-cc (zero|one) #REQUIRED high-cc (one|many) #REQUIRED\n"
  "      is-composite (yes|no) #REQUIRED is-exclusive (yes|no) #REQUIRED is-dependent (yes|no) #REQUIRED\n"
  "      ordered (yes|no) \"no\"\n"
  "  >\n"
  ...
  "]>";

const char OksFile::xml_data_file_dtd[] =
  "<!DOCTYPE oks-data [\n"
  "  <!ELEMENT oks-data (info, (include)?, (comments)?, (obj)+)>\n"
  ...
  "  <!ATTLIST info name CDATA #IMPLIED type CDATA #IMPLIED num-of-items CDATA #REQUIRED\n"
  "      oks-format CDATA #FIXED \"data\" oks-version CDATA #REQUIRED\n"
  "      created-by CDATA #IMPLIED created-on CDATA #IMPLIED creation-time CDATA #IMPLIED\n"
  "      last-modified-by CDATA #IMPLIED last-modified-on CDATA #IMPLIED last-modification-time CDATA #IMPLIED\n"
  "  >\n"
  "  <!ELEMENT include (file)*>\n" "  <!ELEMENT file EMPTY>\n" "  <!ATTLIST file path CDATA #REQUIRED>\n"
  "  <!ELEMENT obj (attr | rel)*>\n"
  "  <!ATTLIST obj class CDATA #REQUIRED id CDATA #REQUIRED>\n"
  "  <!ELEMENT attr (data)*>\n"
  "  <!ATTLIST attr name CDATA #REQUIRED\n"
  "      type (bool|s8|u8|s16|u16|s32|u32|s64|u64|float|double|date|time|string|uid|enum|class|-) \"-\"\n"
  "      val CDATA \"\"\n"
  "  >\n"
  "  <!ELEMENT data EMPTY>\n  <!ATTLIST data val CDATA #REQUIRED>\n"
  "  <!ELEMENT rel (ref)*>\n"
  "  <!ATTLIST rel name CDATA #REQUIRED class CDATA \"\" id CDATA \"\"\n"
  "  <!ELEMENT ref EMPTY>\n  <!ATTLIST ref class CDATA #REQUIRED id CDATA #REQUIRED>\n"
  "]>";
```

(Whitespace inside `<!ATTLIST ...>` collapsed for compactness; content is verbatim.)

Data file formats (src/kernel.cpp): the `oks-format` info attribute is either `"data"` (old format), `"extended"`, or `"compact"` (new format saved by default, see `save_data()` doc line 1338-1341: "By default the format of data file is

compact."). `\"compact\"` = values inside tags (`<data val="...">` with nested/aliased representation); `\"extended\"` = explicit `<attr name type val ...>` tags. The `OksAliasTable` doc in kernel.hpp (lines 430-449) explains the alias technique used to compress class names in data files:

```
The technique of aliases is used to reduce size of OKS data files:
if a class name appears first time somewhere in OKS data file it is marked in front by '@' symbol and the alias to it is used
later, e.g.:
■- first object stored in data file is "Detector@First"
■  and it will be stored as "@Detector@First"
■- second object stored in data file is "Detector@Second"
■  and it will be stored as "0@Second"
■- in this case "0" is alias for "Detector"
```

`OksFile::get_oks_format()` returns `p_oks_format` ("data"/"schema"/"extended"/"compact").

---

6. The Full C++ API Surface (include/oks/*.hpp)

Public declarations from every core OKS header, verbatim from `include/oks/*.hpp`.

6.1 query.hpp — Full Public API

```
class OksQuery {
  public:
    OksQuery(bool b, OksQueryExpression *q = 0) : p_sub_classes (b), p_expression (q), p_status (0) {};
    OksQuery(const OksClass *, const std::string &);
    virtual ~OksQuery();
    friend std::ostream& operator<<(std::ostream&, const OksQuery&);
    bool search_in_subclasses() const {return p_sub_classes;}
    void search_in_subclasses(bool b) {p_sub_classes = b;}
    OksQueryExpression * get() const {return p_expression;}
    void set(OksQueryExpression* q) {p_expression = q;}
    bool good() const {return (p_status == 0);}
    enum QueryType { unknown_type, comparator_type, relationship_type, not_type, and_type, or_type };
    static const char * OR, AND, NOT, SOME, THIS_CLASS, ALL_SUBCLASSES, OID,
                        EQ, NE, RE, LE, GE, LS, GT, PATH_TO, DIRECT, NESTED;
    static bool equal_cmp(const OksData*, const OksData*);
    static bool not_equal_cmp(const OksData*, const OksData*);
    static bool less_or_equal_cmp(const OksData*, const OksData*);
    static bool greater_or_equal_cmp(const OksData*, const OksData*);
    static bool less_cmp(const OksData*, const OksData*);
    static bool greater_cmp(const OksData*, const OksData*);
    static bool reg_exp_cmp(const OksData*, const OksData * regexp);
    typedef bool (*Comparator)(const OksData *, const OksData *);
  private:
    bool p_sub_classes; OksQueryExpression * p_expression; int p_status;
    static OksQueryExpression *create_expression(const OksClass *, const std::string &);
};

class OksQueryExpression {
  friend std::ostream& operator<<(std::ostream&, const OksQueryExpression&);
  public:
    virtual ~OksQueryExpression() {;}
    OksQuery::QueryType type() const {return p_type;}
    bool CheckSyntax() const;
    bool operator==(const class OksQueryExpression& e) const {return (this == &e);}
  protected:
    OksQueryExpression(OksQuery::QueryType qet = OksQuery::unknown_type) : p_type (qet) {};
  private:
    const OksQuery::QueryType p_type;
};

class OksComparator : public OksQueryExpression {
  friend class OksObject; friend class OksQueryExpression;
  public:
    OksComparator(const OksAttribute *a, OksData *v, OksQuery::Comparator f) :
        OksQueryExpression (OksQuery::comparator_type), attribute (a), value (v),
        m_comp_f (f), m_reg_exp (0) {};
    virtual ~OksComparator() { delete value; if(m_reg_exp) delete m_reg_exp; }
    const OksAttribute * GetAttribute() const {return attribute;}
    void SetAttribute(const OksAttribute* a) {attribute = a;}
    OksData * GetValue() {return value;}
    void SetValue(OksData *v);
    void clean_reg_exp();
    OksQuery::Comparator GetFunction() const {return m_comp_f;}
    void SetFunction(OksQuery::Comparator f) {m_comp_f = f;}
  private:
    const OksAttribute *  attribute;
    OksData *             value;
    OksQuery::Comparator  m_comp_f;
    boost::regex *        m_reg_exp;
};

class OksRelationshipExpression : public OksQueryExpression {
  public:
    OksRelationshipExpression(const OksRelationship *r, OksQueryExpression *q, bool b = false) : ... ;
    virtual ~OksRelationshipExpression() {delete p_expression;}
    const OksRelationship * GetRelationship() const;
    void SetRelationship(const OksRelationship*);
    OksQueryExpression * get() const;
    void set(OksQueryExpression*);
    bool IsCheckAllObjects() const;
```

```
    void SetIsCheckAllObjects(const bool b);
};

class OksNotExpression : public OksQueryExpression {
  public:
    OksNotExpression(OksQueryExpression *q = 0) : ... ;
    virtual ~OksNotExpression() {delete p_expression;}
    OksQueryExpression * get() const; void set(OksQueryExpression*);
};

class OksListBaseQueryExpression {  // abstract list of expressions
  public:
    virtual ~OksListBaseQueryExpression() {while(!p_expressions.empty()) {...}}
    const std::list<OksQueryExpression *> & expressions() const {return p_expressions;}
    void add(OksQueryExpression *q) {p_expressions.push_back(q);}
};

class OksAndExpression : public OksQueryExpression, public OksListBaseQueryExpression {
  public: OksAndExpression() : OksQueryExpression(OksQuery::and_type) {}; virtual ~OksAndExpression() {;}
};
class OksOrExpression : public OksQueryExpression, public OksListBaseQueryExpression {
  public: OksOrExpression() : OksQueryExpression(OksQuery::or_type) {}; virtual ~OksOrExpression() {;}
};

class bad_query_syntax : public std::exception { ... };   // thrown by QueryPath parsing

class QueryPathExpression {
  public:
    bool get_use_nested_lookup() const;
    const std::list<std::string>& get_rel_names() const;
    const QueryPathExpression * get_next() const;
  protected:
    QueryPathExpression(bool v);           // direct
    QueryPathExpression(const std::string&);  // parse "(direct|nested ...)" expr
};

class QueryPath {
  public:
    QueryPath(const OksObject * o, QueryPathExpression * qpe) : p_goal(o), p_start(qpe) { }
    QueryPath(const std::string& query, const OksKernel&);
    ~QueryPath() {delete p_start;}
    const QueryPathExpression * get_start_expression() const { return p_start; }
    const OksObject * get_goal_object() const { return p_goal; }
};
```

6.2 OksKernel (kernel.hpp) — Key Public Methods

```
OksKernel(bool silence_mode = false, bool verbose_mode = false, bool profiling_mode = false,
          bool allow_repository = true, const char * version = nullptr, std::string branch_name = "");
OksKernel(const OksKernel& src, bool copy_repository = false);
~OksKernel();
static const char * GetVersion();                       // CVS tag + date of build
static std::string& get_host_name(); static std::string& get_domain_name(); static std::string& get_user_name();
bool get_verbose_mode/get_silence_mode/get_profiling_mode() const;   void set_* (bool)
bool get_allow_duplicated_classes_mode() const; bool get_allow_duplicated_objects_mode() const; void set_* (bool)
bool get_test_duplicated_objects_via_inheritance_mode() const; void set_... (bool)
static bool get_skip_string_range(); static void set_skip_string_range(const bool b);
std::shared_mutex& get_mutex() {return p_kernel_mutex;}
OksFile * find_schema_file(const std::string&) const; OksFile * find_data_file(const std::string&) const;
std::list<OksClass *> * create_list_of_schema_classes(OksFile *) const;
std::list<OksObject *> * create_list_of_data_objects(OksFile *) const;
OksFile * create_file_info(const std::string& short_file_name, const std::string& file_name);
static bool check_read_only(OksFile * f);
std::string get_file_path(const std::string& path, const OksFile * parent_file = 0, bool strict_paths = true) const;
static const std::string& get_repository_root();          // TDAQ_DB_REPOSITORY
const std::string& get_repository_version();
bool is_user_repository_created() const;
static const std::string& get_repository_mapping_dir();
const std::string& get_user_repository_root() const;
void set_user_repository_root(const std::string& path, const std::string& version = "");
static std::string get_tmp_file(const std::string& file_name);
void get_includes(const std::string& file_name, std::set<std::string>& includes, bool use_repository_name = false);
void k_create_dangling_includes();
OksFile * load_file(const std::string& name, bool bind = true);
OksFile * load_schema(const std::string& name, const OksFile * parent = 0);
OksFile * new_schema(const std::string& name);
void save_schema(OksFile * file_h, bool force = false, OksFile * true_file_h = 0);
void save_schema(OksFile * file_h, bool force, const OksClass::Map& classes);
```

```
void backup_schema(OksFile * pf, const char * suffix = ".bak");
void save_as_schema(const std::string& name, OksFile * file_h);
void save_all_schema(); void close_schema(OksFile * file_h); void close_all_schema();
void set_active_schema(OksFile * file_h); void k_set_active_schema(OksFile * file_h);
OksFile * get_active_schema() const {return p_active_schema;}
const OksFile::Map & schema_files() const; const OksFile::Map & data_files() const;
void create_lists_of_updated_schema_files(std::list<OksFile *> **, std::list<OksFile *> **) const;
void get_updated_repository_files(std::set<std::string>&, std::set<std::string>&, std::set<std::string>&);
OksFile * load_data(const std::string& name, bool bind = true);
void reload_data(std::set<OksFile *>& files, bool allow_schema_extension = true);
OksFile * new_data(const std::string& name, const std::string& logical_name = "", const std::string& type = "");
void save_data(OksFile * file_h, bool ignore_bad_objects = false, OksFile * true_file_h = nullptr, bool force_defaults = false);
void save_data(OksFile * file_h, const OksObject::FSet& objects);
void backup_data(OksFile * pf, const char * suffix = ".bak");
void save_as_data(const std::string& new_name, OksFile * file_h);
void save_all_data(bool force_defaults=false);
void close_data(OksFile * file_h, bool unbind_objects = true); void close_all_data();
void set_active_data(OksFile * file_h); void k_set_active_data(OksFile *);
OksFile * get_active_data() const;
void create_lists_of_updated_data_files(std::list<OksFile *> **, std::list<OksFile *> **) const;
void get_modified_files(std::set<OksFile *>& mfs, std::set<OksFile *>& rfs, const std::string& version);
const std::list<std::string>& get_repository_dirs() const;
void commit_repository(const std::string& comments, const std::string& credentials = "");
void tag_repository(const std::string& tag);
std::time_t get_repository_checkout_ts() const;
enum RepositoryUpdateType { DiscardChanges, MergeChanges, NoChanges };
void update_repository(const std::string& hash_val, RepositoryUpdateType update_type);  // "hash"
void update_repository(const std::string& param, const std::string& val, RepositoryUpdateType update_type); // "tag"|"date"|"hash"
std::list<std::string> get_repository_versions_diff(const std::string& sha1, const std::string& sha2);
std::list<std::string> get_repository_unmerged_files();  // == diff("","")
std::vector<OksRepositoryVersion> get_repository_versions(bool skip_irrelevant, const std::string& command_line);
std::vector<OksRepositoryVersion> get_repository_versions_by_hash(bool skip_irrelevant = true,
    const std::string& sha1 = "", const std::string& sha2 = "");
std::vector<OksRepositoryVersion> get_repository_versions_by_date(bool skip_irrelevant = true,
    const std::string& since = "", const std::string& until = "");
std::string read_repository_version();
static const char * get_cwd(); static void reset_cwd(); static void set_use_strict_repository_paths(bool);
std::string insert_repository_dir(const std::string& dir, bool push_back = true);
void remove_repository_dir(const std::string& dir);
const OksClass::Map & classes() const; size_t number_of_classes() const;
const OksObject::Set & objects() const; size_t number_of_objects() const;
OksClass * find_class(const std::string&) const; OksClass * find_class(const char *) const;
void get_all_classes(const std::vector<std::string>& names, ClassSet& classes_out) const;
void registrate_all_classes(bool skip_registered = false);
bool is_dangling(OksClass * class_ptr) const; bool is_dangling(OksObject * obj_ptr) const;
void subscribe_create_class(void (*f)(OksClass *));
void subscribe_change_class(void (*f)(OksClass *, OksClass::ChangeType, const void *));
void subscribe_delete_class(void (*f)(OksClass *));
void subscribe_create_object(OksObject::notify_obj, void * parameter);
void subscribe_change_object(OksObject::notify_obj, void *);
void subscribe_delete_object(OksObject::notify_obj, void *);
void bind_objects(); const std::string& get_bind_objects_status() const;
const std::string& get_bind_classes_status() const;
void unset_repository_created();
```

`OksRepositoryVersion` struct (kernel.hpp lines 514-530, verbatim):

```
struct OksRepositoryVersion
{
  std::string m_commit_hash;
  std::string m_user;
  std::time_t m_date;
  std::string m_comment;
  std::vector<std::string> m_files;

  void clear()
  {
    m_commit_hash.clear();
    m_user.clear();
    m_date = 0;
    m_comment.clear();
    m_files.clear();
  }
};
```

`OksKernel` doc comment (kernel.hpp lines 532-579) summarizes the whole API surface, verbatim intro:

```
It is responsible for loading OKS data and schema files.  Multiple concurrent kernels....
```

```
 *  To work with OKS schema the following base methods are available:
 *   - load_schema() - load OKS schema from file (i.e. read description of classes with attributes, relationships and relationships)
...
 *  When schema or data are modified, the process can give callback ...
 *  - subscribe_create_class() - notify, when new class is created
 ...
 *  Most of the OksKernel methods are thread-safe. Those which are not thread-safe, are starting from prefix "k_".
```

6.3 OksClass (class.hpp) — Public API

```
OksClass (const std::string& name, OksKernel * kernel, bool transient = false);
OksClass (const std::string& name, const std::string& description, bool is_abstract, OksKernel * kernel, bool transient = false);
OksClass (OksKernel * kernel, const std::string& name, const std::string& description, bool is_abstract); // fast internal
static void destroy(OksClass * c);
bool operator==(const OksClass&) const; bool operator!=(const OksClass&) const;
bool compare_without_methods(const OksClass & v) const noexcept;
OksKernel * get_kernel() const noexcept {return p_kernel;}
OksFile * get_file() const noexcept {return p_file;}
void set_file(OksFile * f, bool update_owner = true);
const std::string& get_name() const noexcept; const std::string& get_description() const noexcept;
void set_description(const std::string& description); bool get_is_abstract() const noexcept; void set_is_abstract(bool);
const FList * all_super_classes() const noexcept; const std::list<std::string *> * direct_super_classes() const noexcept;
OksClass * find_super_class(const std::string&) const noexcept; bool has_direct_super_class(const std::string&) const noexcept;
void add_super_class(const std::string& name); void k_add_super_class(const std::string&);
void remove_super_class(const std::string& name); void swap_super_classes(const std::string&, const std::string&);
const FList * all_sub_classes() const noexcept;
const std::list<OksAttribute *> * all_attributes() const noexcept;
const std::list<OksAttribute *> * direct_attributes() const noexcept;
OksAttribute * find_attribute(const std::string& name) const noexcept;
OksAttribute * find_direct_attribute(const std::string&) const noexcept;
void add(OksAttribute * a); void k_add(OksAttribute * a); void remove(const OksAttribute * a);
void swap(const OksAttribute * a1, const OksAttribute * a2);
size_t number_of_direct_attributes() const noexcept; size_t number_of_all_attributes() const noexcept;
OksClass * source_class(const OksAttribute * a) const noexcept;
const std::list<OksRelationship *> * all_relationships() const noexcept;
const std::list<OksRelationship *> * direct_relationships() const noexcept;
OksRelationship * find_relationship(const std::string& name) const noexcept;
OksRelationship * find_direct_relationship(const std::string&) const noexcept;
void add(OksRelationship * r); void k_add(OksRelationship * r); void remove(const OksRelationship * r, bool call_delete = true);
void swap(const OksRelationship * r1, const OksRelationship * r2);
size_t number_of_direct_relationships() const noexcept; size_t number_of_all_relationships() const noexcept;
OksClass * source_class(const OksRelationship * r) const noexcept;
// methods:
const std::list<OksMethod *> * all_methods() const noexcept; const std::list<OksMethod *> * direct_methods() const noexcept;
OksMethod * find_method(const std::string&) noexcept; void add(OksMethod *); void remove(const OksMethod *); void swap(...);
size_t number_of_objects() const noexcept; const OksObject::Map * objects() const noexcept;
std::list<OksObject *> * create_list_of_all_objects() const noexcept;
OksObject * get_object(const std::string& id) const noexcept; OksObject * get_object(const std::string* id) const;
OksObject::List * execute_query(OksQuery * query) const;                          // << QUERY API
OksDataInfo * data_info(const std::string&) const noexcept; OksDataInfo * get_data_info(const std::string&) const noexcept;
enum ChangeType { ChangeSuperClassesList, ChangeSubClassesList, ChangeDescription, ChangeIsAbstract,
    ChangeAttributesList, ChangeAttributeType, ChangeAttributeRange, ChangeAttributeFormat,
    ChangeAttributeMultiValueCardinality, ChangeAttributeInitValue, ChangeAttributeDescription, ChangeAttributeIsNoNull,
    ChangeRelationshipOther, ChangeRelationshipClassType, ChangeRelationshipDescription, ChangeRelationshipLowCC,
    ChangeRelationshipHighCC, ChangeRelationshipComposite, ChangeRelationshipExclusive, ChangeRelationshipDependent,
    ChangeMethodsList, ChangeMethodDescription, ChangeMethodImplementation };
typedef void (*NotifyFN)(OksClass *); typedef void (*ChangeNotifyFN)(OksClass *, ChangeType, const void *);
```

6.4 OksObject (object.hpp) — Full Public API

```
OksObject (const OksClass * oks_class, const char * object_id = 0, bool skip_init = false);
OksObject (const OksObject & parent_object, const char * object_id = 0);
static void destroy(OksObject * obj, bool fast = false);
bool operator==(const OksObject&) const; static bool are_equal_fast(const OksObject*, const OksObject*);
bool operator!=(const OksObject&) const = delete;
const OksClass * GetClass() const; const std::string& GetId() const; void set_id(const std::string & id);
OksFile * get_file() const; void set_file(OksFile * file, bool update_owner = true);
OksData * GetAttributeValue(const std::string& name) const;
OksData * GetAttributeValue(const OksDataInfo *i) const noexcept;
OksData * GetRelationshipValue(const std::string&) const;
OksData * GetRelationshipValue(const OksDataInfo *i) const noexcept;
void SetAttributeValue(const std::string& name, OksData * data);
void SetAttributeValue(const OksDataInfo * data_info, OksData * data);
void SetRelationshipValue(const std::string& name, OksData * data, bool skip_non_null_check = false);
void SetRelationshipValue(const OksDataInfo * data_info, OksData * data, bool skip_non_null_check = false);
void SetRelationshipValue(const std::string& name, OksObject * object);
void SetRelationshipValue(const OksDataInfo * data_info, OksObject * object);
```

```
void AddRelationshipValue(const std::string& name, OksObject * object);
void AddRelationshipValue(const OksDataInfo * data_info, OksObject * object);
void RemoveRelationshipValue(const std::string& name, OksObject * object);
void RemoveRelationshipValue(const OksDataInfo * data_info, OksObject * object);
void SetRelationshipValue(const std::string& rel_name, const std::string& class_name, const std::string& object_id);
void AddRelationshipValue(const std::string& rel_name, const std::string& class_name, const std::string& object_id);
void RemoveRelationshipValue(const std::string& rel_name, const std::string& class_name, const std::string& object_id);
const std::list<OksRCR *> * reverse_composite_rels() const;   // RCR = reverse composite relationship
FList * get_all_rels(const std::string& name = "*") const;
void SetTransientData(void *d) const; void * GetTransientData() const;
void set_int32_id(int32_t object_id); int32_t get_int32_id() const;
bool SatisfiesQueryExpression(OksQueryExpression * query_exp) const;       // << QUERY
bool satisfies(const OksObject * goal, const QueryPathExpression& expression, OksObject::List& path) const;
OksObject::List * find_path(const QueryPath& query) const;              // << PATH-TO
bool is_consistent(const std::set<OksFile *>&, const char * msg) const;
std::string report_dangling_references() const;
void references(OksObject::FSet& refs, unsigned long recursion_depth, bool add_self = false, ClassSet * classes = 0) const;
bool is_duplicated() const;
```

`struct OksData` (object.hpp lines 454-748) — runtime typed value: `enum Type { unknown_type=0, s8..., u8..., s16..., u16..., s32..., u32..., s64..., u64..., float_type, double_type, bool_type, class_type, object_type, date_type, time_type, string_type, list_type, uid_type, uid2_type, enum_type }` — plus constructors per type, `Set*`/`ReadFrom*`/`str()`/`check_range()`/`set_init_value()`/comparison operators/`sort(bool ascending = true)`.

6.5 OksAttribute (attribute.hpp) — Public API

```
enum Format { Oct = 8, Dec = 10, Hex = 16 };
OksAttribute (const std::string& name, OksClass * p = nullptr);
OksAttribute (const std::string& name, const std::string& type, bool is_mv, const std::string& range,
              const std::string& init_v, const std::string& description, bool no_null,
              Format format = Dec, OksClass * p = nullptr);
const std::string& get_name() const noexcept;   void set_name(const std::string& name);
const std::string& get_type() const noexcept;   void set_type(const std::string& type, bool skip_init = false);
const std::string& get_range() const noexcept;  void set_range(const std::string& range);
static OksData::Type get_data_type(const std::string& type) noexcept;
static OksData::Type get_data_type(const char * type, size_t len) noexcept;
OksData::Type get_data_type() const noexcept;
Format get_format() const noexcept; void set_format(Format format);
bool is_integer() const noexcept;    bool is_number() const noexcept;
bool get_is_multi_values() const noexcept; void set_is_multi_values(bool);
const std::string& get_init_value() const noexcept; void set_init_value(const std::string&);
std::list<std::string> get_init_values() const;    // mv-init, "class_type" -> AttributeReadError
const std::string& get_description() const noexcept; void set_description(const std::string&);
bool get_is_no_null() const noexcept; void set_is_no_null(bool);
static bool find_token(const char * token, const char * range) noexcept;
int  get_enum_index(const char * s, size_t length) const noexcept;
int  get_enum_index(const std::string&) const noexcept;
const std::string * get_enum_value(const char * s, size_t length) const;
const std::string * get_enum_value(const std::string&) const;
uint16_t get_enum_value(const OksData& d) const noexcept;
const std::string * get_enum_string(uint16_t idx) const noexcept;
static const char * bool_type; static const char * s8_int_type; ... static const char * u64_int_type;
static const char * float_type; static const char * double_type; static const char * date_type;
static const char * time_type; static const char * string_type; static const char * uid_type;
static const char * enum_type; static const char * class_type;
static Format str2format(const char *) noexcept; static const char * format2str(Format) noexcept;
```

`set_range` doc (verbatim, notation UML): `"A,B,C..D,*..F,G..*"` means: `A`, `B`, `C..D` (closed interval), `*..F` (<=F), `G..*` (>=G); spaces not allowed; `enum` attrs must have non-empty range; `bool` has predefined range "true,false". Range validation implemented by `OksRange` class (attribute.hxx lines 36-90): conditions decomposed into `m_less, m_equal, m_interval, m_great, m_like(+) boost::regex` lists.

6.6 OksRelationship (relationship.hpp) — Public API + Doc Example

```
enum CardinalityConstraint { Zero, One, Many };
OksRelationship (const std::string& name, OksClass * p = nullptr);
OksRelationship (const std::string& name, const std::string& type, CardinalityConstraint low_cc,
                 CardinalityConstraint high_cc, bool composite, bool exclusive, bool dependent,
                 const std::string& description, OksClass * p = nullptr);
const std::string& get_name() const noexcept; void set_name(const std::string&);
OksClass * get_class_type() const noexcept;  const std::string& get_type() const noexcept; void set_type(const std::string&);
const std::string& get_description() const noexcept; void set_description(const std::string&);
```

```
CardinalityConstraint get_low_cardinality_constraint() const noexcept; void set_low_cardinality_constraint(CardinalityConstraint cc);
CardinalityConstraint get_high_cardinality_constraint() const noexcept; void set_high_cardinality_constraint(CardinalityConstraint);
bool get_is_composite() const noexcept;  void set_is_composite(bool);
bool get_is_exclusive() const noexcept;  void set_is_exclusive(bool);
bool get_is_dependent() const noexcept;  void set_is_dependent(bool);
static CardinalityConstraint str2card(const char *) noexcept; static const char * card2str(CardinalityConstraint) noexcept;
```

Conceptual doc paragraph (relationship.hpp lines 36-56, verbatim):

```
A relationship can be either a general (weak) or composite (strong) reference.
... A composite reference may be exclusive or shared (an exclusive object is component of only one object,
a shared object may be referenced by more than one object). ... a composite reference can be dependent or independent.
The existence of object, referenced though dependent composite reference, is dependent from existence of their composite parents.
The deleting of all composite parents results the deleting of their own composite child objects.
```

Code example (relationship.hpp lines 58-95) — `main()` showing construction & stream operator output; output:

```
Relationship name: "consists of"
 class type: "Element"
 low cardinality constraint is zero
 high cardinality constraint is many
 is composite reference
 is exclusive reference
 is dependent reference
 has description: "A structure consists of zero or many elements"
```

6.7 OksIndex (index.hpp)

```
class OksObjectSortBy { public: OksObjectSortBy(size_t i = 0) : offset(i) {};
    bool operator() (const OksObject * o1, const OksObject * o2) const { return o1->data[offset] < o2->data[offset]; } };

class OksIndex : public std::multiset<OksObject *, OksObjectSortBy> {
  public:
    struct SortByAttribute { ... };   typedef std::map<const OksAttribute *, OksIndex *, SortBy> Map;
    OksIndex (OksClass *, OksAttribute *);
    ~OksIndex ();
    OksObject *         FindFirst(OksData *d) const;
    OksObject::List *   FindEqual(OksData *d) const;         // equal_cmp
    OksObject::List *   FindLessEqual(OksData *d) const;     // less_or_equal_cmp
    OksObject::List *   FindGreatEqual(OksData *d) const;    // greater_or_equal_cmp
    OksObject::List *   FindLess(OksData *d) const;          // less_cmp
    OksObject::List *   FindGreat(OksData *d) const;         // greater_cmp
    OksObject::List *   FindLessAndGreat(...), FindLessAndGreatEqual(...), FindLessEqualAndGreat(...),
                        FindLessEqualAndGreatEqual(...), FindEqualAndEqual(...), FindEqualAndLess(...),
                        FindEqualAndLessEqual(...), FindEqualAndGreat(...), FindEqualAndGreatEqual(...);
    OksObject::List *   FindLessOrGreat(...), FindLessOrGreatEqual(...), FindLessEqualOrGreat(...),
                        FindLessEqualOrGreatEqual(...), FindEqualOrEqual(...), FindEqualOrLess(...),
                        FindEqualOrLessEqual(...), FindEqualOrGreat(...), FindEqualOrGreatEqual(...);
  private:
    OksClass * c; OksAttribute * a; size_t offset;
    void find_all(OksData *, OksQuery::Comparator) const;
    OksObject::List * find_all(bool, OksData *, OksQuery::Comparator, OksData *, OksQuery::Comparator) const;
    void find_interval(OksData *, OksQuery::Comparator, ConstPosition&, ConstPosition&) const;
    static size_t get_offset(OksClass *, OksAttribute *);
};
```

(Function-pointer complication is lifted from the CERN original; in this DUNE fork the comparator is a `std::function`. The complete member list is folded onto mnemonicized lines above.)

6.8 OksFile (file.hpp) — Public API

```
enum Mode { ReadOnly, ReadWrite };
enum FileStatus { FileNotModified, FileModified, FileWasNotSaved, FileRemoved };
void lock(); static void set_nolock_mode(bool nl);  // Nasty hack to allow code generation on the fly (off by default)
void unlock();
void add_include_file(const std::string& name);
void remove_include_file(const std::string& name);
void rename_include_file(const std::string& from, const std::string& to);
void add_comment(const std::string& text, const std::string& author);
void modify_comment(const std::string& creation_time, const std::string& text, const std::string& author);
void remove_comment(const std::string& creation_time);
const std::map<std::string, oks::Comment *>& get_comments() const;
void set_logical_name(const std::string& name); void set_type(const std::string& type);
const std::string& get_short_file_name() const; const std::string& get_full_file_name() const;
bool is_repository_file() const;
```

```
const std::string& get_repository_name() const;
const std::string& get_well_formed_name() const;
const std::string& get_lock_file() const; const std::string& get_logical_name() const;
const std::string& get_oks_format() const; long get_number_of_items() const; long get_size() const;
const std::string& get_created_by() const; const boost::posix_time::ptime get_creation_time() const;
const std::string& get_created_on() const; const std::string& get_last_modified_by() const;
const boost::posix_time::ptime get_last_modification_time() const; const std::string& get_last_modified_on() const;
bool is_locked() const; bool is_updated() const; bool is_read_only() const;
const std::list<std::string>& get_include_files() const;
void get_all_include_files(const OksKernel * kernel, std::set<OksFile *>& out);
FileStatus get_status_of_file() const; void update_status_of_file(bool update_local = true, bool update_repository = true);
bool get_lock_string(std::string& info) const;
static bool compare(const char * file1_name, const char * file2_name); bool compare(const char * name) const;
const OksFile * get_parent() const; OksFile * check_parent(const OksFile * parent_h);
```

Comment in file.hpp on `is_repository_file()` (verbatim): "If the `TDAQ_DB_REPOSITORY` env var is set and file is on it: GlobalRepository; if `TDAQ_DB_USER_REPOSITORY`: UserRepository; else NoneRepository." — detailing the G-version/repository concepts.

---

7. CLI Tools

oks_dump

```
Usage: oks_dump
    [--files-only | --files-stat-only | --schema-files-only | --schema-files-stat-only | --data-files-only | --data-files-stat-only]
    [--class name-of-class [--query query [--print-references recursion-depth [class-name*] [--]] [--print-referenced_by [name] [--]]
    [--path object-from object-to query]
    [--allow-duplicated-objects-via-inheritance]
    [--version]
    [--help]
    [--input-from-files] database-file [database-file...]

Options:
    -f | --files-only                               print list of oks files names
    -F | --files-stat-only                          print list of oks files with statistic details (size, number of items)
    -s | --schema-files-only                        print list of schema oks files names
    -S | --schema-files-stat-only                   print list of oks schema files with statistic details
    -d | --data-files-only                          print list of data oks files names
    -D | --data-files-stat-only                     print list of oks data files with statistic details
    -c | --class class_name                         dump given class (all objects or matching some query)
    -q | --query query                              print objects matching query (can only be used with class)
    -r | --print-references N C1 C2 ... CX          print objects referenced by found objects (can only be used with query), where:
                                                        * the parameter N defines recursion depth for referenced objects (> 0)
                                                        * the optional set of names {C1 .. CX} defines [sub-]classes for above obje
    -b | --print-referenced_by [name]               print objects referencing found objects (can only be used with query), where:
                                                        * the optional parameter name defines name of relationship
    -p | --path object-from path-expression         print path from object 'object-from' to object of path-expression
    -i | --input-from-files                         read oks files to be loaded from file(s) instead of command line
                                                    (to avoid problems with long command line, when there is huge number of files)
    -a | --allow-duplicated-objects-via-inheritance do not stop if there are duplicated object via inheritance hierarchy
    -v | --version                                  print version
    -h | --help                                     print this text

Description:
    Dumps contents of the OKS database.

Return Status:
    0 - no problems found
    1 - bad command line parameter
    2 - bad oks file(s)
    3 - bad query passed via -q or -p options
    4 - cannot find class passed via -c option
    5 - loaded objects have dangling references
    6 - caught an exception
```

(Source has short forms -f/-F/-s/-S/-d/-D/-c/-q/-r/-b/-p/-i/-a/-v/-h; enum at lines 29-37: `__Success__=0, __BadCommandLine__, __BadOksFile__, __BadQuery__, __NoSuchClass__, __FoundDanglingReferences__, __ExceptionCaught__`.)

Query usage example from source (line 269): `OksQuery * q = new OksQuery(c, query); if(q->good()) { OksObject::List * objs = c->execute_query(q); ...}` — full flow shown above.

oks_clone_repository

```
usage: oks_clone_repository
    --branch=B | --version=V | --output-directory=D | --verbose | --help

By default the master branch is cloned. The branch is created. If TDAQ_DB_VERSION env var
is set, or versions via --version, the repository is checked-out for that hash/date/tag.
If TDAQ_DB_USER_REPOSITORY is set, the utility makes no effect.
```

(Literal banner from source, lines 30-35: `"Clone and checkout oks repo.\\n\\n...By default the master branch is checkout. The \\"branch\\" command line option can be used to specify particular one. A branch will be created, if does not exist yet. TDAQ_DB_VERSION process environment variable or \\"version\\" command line option can be used to specify particular version... If TDAQ_DB_USER_REPOSITORY process environment variable is set, the utility makes no effect."`)

Options (verbatim):

```
  -b [ --branch ] arg            checkout or create given branch name
  -e [ --version ] arg           oks config version in type:value format, where type is "hash", "date" or "tag"
  -o [ --output-directory ] arg output directory; if not defined, create temporal
  -v [ --verbose ]               print verbose information
  -h [ --help ]                  print help message
```

Exit codes: `EXIT_SUCCESS` (0) on success; `EXIT_FAILURE` (1) on command line parsing error. It sets env `TDAQ_DB_USER_REPOSITORY_PATH` from `--output-directory` before constructing the kernel; prints the created user repository root to stdout when no output dir given.

oks_check_schema

```
Options: -f, --file (required)   Schema file

Return codes:
  0 Success, file is OK
  1 Bad command line
  2 Failed to find file
  3 Failed to load file -- invalid schema
  4 File contains relationship to non loaded class
  5 File contains object with relationship to non loaded class/object
  6 File refers to class in file not directly included
```

Behavior: loads file; works on `*.schema.xml` (checks every class in it: superclass existence implied by load success, direct relationship class type loaded (`rel->get_class_type() == nullptr` → `Exitcode::BADRELATIONSHIP`), and `file->get_include_files()` coverage → `MISSING_INCLUDE`), and on `*.data.xml` (checks each object's class's schema-file inclusion, and every relationship target object's file inclusion); finally `kernel.get_bind_objects_status()` non-empty → `UNRESOLVED`.

oks_validate_repository

```
usage: oks_validate_repository
  -a [ --add ] arg                 list of new OKS files and directories to be added to the repository
  -u [ --update ] arg              list of new OKS files and directories to be updated in the repository
  -r [ --remove ] arg              list of new OKS files and directories to be removed from the repository
  -C [ --permissive-circular-dependencies-between-includes ]
                                   downgrade severity of detected circular dependencies between includes from errors to warnings
  -U [ --user ] arg                user id
  -t [ --threads-number ] (=4)     number of threads used by validation pipeline
  -v [ --verbose ]                 print debug information
  -h [ --help ]                    print help message
```

Exit codes enum (lines 32-43): `__Success__=0, __BadCommandLine__=1, __UserAuthenticationFailure__=2, __NoRepository__=3, __ConsistencyError__=4, __IncludesCircularDependencyError__=5, __NoIncludedFile__=6, __ExceptionCaught__=7` (names in source: `__Success__`/`__BadCommandLine__`/`__UserAuthenticationFailure__`/`__NoRepository__`/`__ConsistencyError__`/`__IncludesCircularDependencyError__`/`__NoIncludedFile__`/`__ExceptionCaught__`). Behavior: requires `TDAQ_DB_REPOSITORY` env; runs `get_includes()` on every file (skipping `.git`, `admin`, `README.md`), checks every include exists, builds transitive include graph and circular dependency detection, then validates every create/update file (and files whose includes changed) via `OksPipeline` parallel jobs that `load_file()` in silence mode and inspect bind status.

oks_git_repository

```
Usage: oks_git_repository [-h|--help]
   Prints repository root from $TDAQ_DB_REPOSITORY to stdout
```

---

8. Python Bindings (pybindsrc/module.cpp)

Module name: `_daq_oks_py`, wrapped by `python/oks/__init__.py` (`from ._daq_oks_py import *`). Verbatim binding list:

```
PYBIND11_MODULE(_daq_oks_py, m)
{
  m.doc() = "C++ implementation of the application dal modules";

  py::class_<OksFile>(m, "OksFile");
  py::class_<OksClass, std::unique_ptr<OksClass, py::nodelete>>(m, "OksClass")
      .def("get_name",&OksClass::get_name, py::return_value_policy::reference_internal)
      .def("get_description",&OksClass::get_description, ...)
      .def("get_is_abstract",&OksClass::get_is_abstract)
      .def("all_super_classes",&OksClass::all_super_classes, ...)
      .def("direct_super_classes",&OksClass::direct_super_classes, ...)
      .def("all_sub_classes",&OksClass::all_sub_classes, ...)
  ;
  py::class_<OksObject, std::unique_ptr<OksObject, py::nodelete>>(m, "OksObject");

  py::class_<OksKernel>(m, "OksKernel")
      .def(py::init<bool, bool, bool, bool, const char *, std::string>(),
        "silence_mode"_a = false, "verbose_mode"_a = false, "profiling_mode"_a = false,
        "allow_repository"_a = true, "version"_a = nullptr, "branch_name"_a = "")
      .def("get_host_name",...).def("get_domain_name",...).def("get_user_name",...)
      .def("get_verbose_mode"/"set_verbose_mode")
      .def("get_silence_mode"/"set_silence_mode")
      .def("set_profiling_mode")
      .def("get_allow_duplicated_classes_mode"/"set_allow_duplicated_classes_mode")
      .def("get_allow_duplicated_objects_mode"/"set_allow_duplicated_objects_mode")
      .def("get_test_duplicated_objects_via_inheritance_mode"/"set_...")
      .def("find_schema_file"/"find_data_file")
      .def("create_list_of_schema_classes"/"create_list_of_data_objects")
      .def("create_file_info")
      .def("get_file_path","path"_a,"parent_file"_a=nullptr,"strict_path"_a=true)
      .def("get_repository_version")
      .def("is_user_repository_created")
      .def("get_user_repository_root"/"set_user_repository_root")
      .def("get_includes")
      .def("load_file","name"_a,"bind"_a=true)
      .def("load_schema","name"_a,"parent"_a=nullptr)
      .def("new_schema","name"_a)
      .def("backup_schema","pf"_a,"suffix"_a=".bak")
      .def("save_as_schema"/"save_all_schema"/"close_schema"/"close_all_schema"/"set_active_schema"/"get_active_schema")
      .def("schema_files")
      .def("load_data","name"_a,"bind"_a=true)
      .def("reload_data","files"_a,"allow_schema_extension"_a=true)
      .def("new_data","name"_a,"logical_name"_a="","type"_a="")
      .def("save_all_data","force_defaults"_a=false)
      .def("close_all_data")
      .def("set_active_data"/"get_active_data"/"data_files")
      .def("insert_repository_dir"/"remove_repository_dir")
      .def("classes"/"number_of_classes"/"objects"/"number_of_objects")
      .def("find_class",static_cast<OksClass*(OksKernel::*)(const std::string&) const>(&OksKernel::find_class), "class_name"_a)
      .def("registrate_all_classes")
      .def("bind_objects").def("get_bind_objects_status").def("get_bind_classes_status")
      .def("unset_repository_created")
  ;
}
```

Note: the OksData/OksQuery classes are NOT exposed; commented-out placeholders in the file (lines 46-66, 138-139, 168-197) show future intent (load_data/save_data with overloads, commit_repository/tag_repository/get_repository_versions, etc.)

---

9. Versioning — Repository Environment Variables and Git Helpers

Env vars referenced by source code (kernel.hpp / file.hpp / apps):

- `TDAQ_DB_REPOSITORY` — global (server) repository root; git-based

- `TDAQ_DB_USER_REPOSITORY` — per-user checkout dir (used by all oks-*.sh)

- `TDAQ_DB_PATH` — extra search path for data files (get_file_path docs)

- `OKS_DB_ROOT` — old-style OKS root

- `TDAQ_DB_VERSION` — version selector (used by oks_clone_repository)

- `OKS_KERNEL_VERBOSE`, `OKS_KERNEL_SILENCE`, `OKS_KERNEL_PROFILING`, `OKS_KERNEL_ALLOW_DUPLICATED_CLASSES`, `OKS_KERNEL_ALLOW_DUPLICATED_OBJECTS`, `OKS_KERNEL_TEST_DUPLICATED_OBJECTS_VIA_INHERITANCE`, `OKS_KERNEL_SKIP_STRING_RANGE` — mode toggles.

Script family (all `scripts/*.sh`), usage banners verbatim:

- `oks-checkout.sh`: `Usage: oks-checkout.sh [-v] [-u user-repository-dir] [-b branch] [-c commit-hash] [-t tag] [-d date] [-h]` — "checkout files and directories from OKS git repository into user repository..." (TDAQ_DB_REPOSITORY = git repo, TDAQ_DB_USER_REPOSITORY = user dir; else cwd).

- `oks-update.sh`: `Usage: oks-update.sh [-v] [-u user-repository-dir] [-c commit-hash] [-t tag] [-d date] [-f | -m] [-h]` — update to HEAD of master by default; `-f|--force|--discard` vs `-m|--merge`.

- `oks-commit.sh`: `Usage: oks-commit.sh [-v] [-u user-repository-dir] [-h] -m message | -f commit-message-file` — commits xml files via temp branch `temp_oks_commit_branch`, pull --rebase, push with lock-conflict retry. Exit codes 1-4 (error / cleanup / undo_merge / undo_exit).

- `oks-tag.sh`: `Usage: oks-tag.sh [-v] [-u user-repository-dir] [-h] -c sha -t tag` — "tag existing commit".

- `oks-import.sh`: `Usage: oks-import.sh [-v] [-t] [-n] -m message | -f commit-message-file what ...` — import files/dirs into git repo; `-n` dry-run.

- `oks-copy.sh`: `Usage: oks-copy.sh -s source-dir -d destination-dir -c commit-hash [-h]` — clone part of repo at given hash.

- `oks-diff.sh`: `Usage: oks-diff.sh [-v] [-u user-repository-dir] [--unmerged] [--sha sha1 sha2] [-h]` — diff between repository versions.

- `oks-log.sh`: `Usage: oks-log.sh [-v] [-u user-repository-dir] [-h] [-s date/time] [-t date/time] [-n num]` — show repository versions (git log).

- `oks-version.sh`: `Usage: oks-version.sh [-u user-repository-dir] [-h]` — prints `oks version <git rev-parse HEAD>` (verbatim line 53).

- `oks-status.sh`: `Usage: oks-status.sh [-u user-repository-dir] [-h]` — repo status.

- `oks-edit-branch.sh`: `Usage: oks-edit-branch.sh [-c] [-p directory] [-e editor] [-m "log message"] -b branch file+` — edit files on branch.

10. The 16 oks_utils Worked C++ Examples (verbatim)

Each example is a self-contained `main()` demonstrating one facet of the OKS API, from `oks_utils/examples/*.cpp` (CERN GitLab).

examples/comparator.cpp

```
#include <oks/attribute.h>
#include <oks/query.h>

int main()
{
  try
    {
      const OksAttribute a("Name", OksAttribute::string_type, false, "", "unknown", "describes address", true);

      OksComparator qc(&a, new OksData("Peter"), OksQuery::equal_cmp);

      std::cout << qc << std::endl;
    }
  catch (const std::exception& ex)
    {
      std::cerr << "Caught exception:\n" << ex.what() << std::endl;
    }

  return 0;
}
```

examples/query.cpp

```
#include <oks/kernel.h>
#include <oks/class.h>
#include <oks/object.h>
#include <oks/query.h>

int main()
{
  try
    {
      OksKernel k; /* create OKS kernel */

      k.set_silence_mode(true);
      k.new_schema("/tmp/car.schema"); /* create new schema */
      k.new_data("/tmp/car.data"); /* create new data */

      /* define class 'Car' */
      OksClass * p = new OksClass("Car", "Describes a car", false, &k);

      /* define attribute 'Max Speed' */
      OksAttribute * a = new OksAttribute("Max Speed", OksAttribute::u16_int_type, false, "", "160", "max speed of car (km/h)", true)

      p->add(a);

      /* Creates three instances of Car class */
      OksObject * bmw316i = new OksObject(p, "BMW 316i");
      OksObject * bmw318i = new OksObject(p, "BMW 318i");
      OksObject * bmw320i = new OksObject(p, "BMW 320i");

      /* set max speeds */
      OksData d((uint16_t) 196);
      bmw316i->SetAttributeValue("Max Speed", &d);
      d.Set((uint16_t) 201);
      bmw318i->SetAttributeValue("Max Speed", &d);
      d.Set((uint16_t) 214);
      bmw320i->SetAttributeValue("Max Speed", &d);

      OksQuery q(false, new OksComparator(a, new OksData((unsigned short) 200), OksQuery::greater_cmp));

      std::list<OksObject *> * result = p->execute_query(&q);

      std::cout << "Query \'" << q << '\'' << ' ' << "found " << result->size() << ' ' << "objects in class \"" << p->get_name() << "

      int i = 1;
      while (!result->empty())
        {
          OksObject * o = result->front();
```

```
          result->pop_front();
          std::cout << i++ << '.' << ' ' << *o;
        }

      delete result;

      OksObject::destroy(bmw320i);
      OksObject::destroy(bmw318i);
      OksObject::destroy(bmw316i);
    }
  catch (const std::exception & ex)
    {
      std::cerr << "Caught exception: " << ex.what() << std::endl;
    }

  return 0;
}
```

examples/and_expression.cpp

```
#include <oks/attribute.h>
#include <oks/query.h>

int main()
{
  OksAttribute a(
        "Weight",
        OksAttribute::float_type,
        false,
■"",
        "75",
        "person's weight",
        true
  );

  OksAndExpression and_q;

■/* Looking for 60 >= weight <= 90 */

  and_q.add(new OksComparator(&a, new OksData((float)60.0), OksQuery::greater_or_equal_cmp));
  and_q.add(new OksComparator(&a, new OksData((float)90.0), OksQuery::less_or_equal_cmp));

  std::cout << and_q << std::endl;

  return 0;
}
```

examples/or_expression.cpp

```
#include <oks/attribute.h>
#include <oks/query.h>

int main()
{
  OksAttribute a(
        "Heigth",
        OksAttribute::float_type,
        false,
■"",
        "1.77",
        "person's heigth",
        true
  );

  OksOrExpression or_q;

■/* Looking for tall (h >= 1.88) and short (h <= 1.65) */

  or_q.add(new OksComparator(&a, new OksData((float)1.65), OksQuery::greater_or_equal_cmp));
  or_q.add(new OksComparator(&a, new OksData((float)1.88), OksQuery::less_or_equal_cmp));

  std::cout << or_q << std::endl;

  return 0;
}
```

examples/not_expression.cpp

```
#include <oks/attribute.h>
#include <oks/query.h>

int main()
{
  OksAttribute a(
        "age",
        OksAttribute::u16_int_type,
        false,
■"",
        "33",
        "describes age",
        true
  );

  OksNotExpression ne(new OksComparator(&a, new OksData((unsigned short)25), OksQuery::less_cmp));

  std::cout << ne << std::endl;

  return 0;
}
```

examples/r_expression.cpp

```
#include <oks/attribute.h>
#include <oks/relationship.h>
#include <oks/query.h>

int main()
{
  OksRelationship r(
■"has car", "Car",
■OksRelationship::Zero, OksRelationship::Many,
■true, true, false, false,
■"A person has zero or more cars"
  );

  OksAttribute a(
■"Type",
■OksAttribute::string_type,
■false,
■"",
■"unknown",
■"describes car type",
■true
  );


  OksRelationshipExpression rqe(&r, new OksComparator(&a, new OksData("BMW"), OksQuery::equal_cmp), false);

  std::cout << rqe << std::endl;

  return 0;
}
```

examples/attribute.cpp

```
#include <oks/attribute.h>

int main()
{
  try
    {
      //  create simple attribute
      OksAttribute a(
■"Address", /* name is 'Address' */
■OksAttribute::string_type, /* the type is 'string' */
■false, /* attribute is 'single value' */
■"", /* any range */
■"unknown", /* initial value */
■"describes address", /* desription */
■true /* can not be empty */
      );

      std::cout << a;


      // check multi-value attribute's  get_init_values()
```

```
      OksAttribute b(
■"Numbers",
■OksAttribute::s32_int_type,
■true, /* attribute is 'multi-value' */
■"",
■"1,2, 4, 9, 16, 25, \"36\", \'49\',\'64\',81, 100",
■"just a test",
■true
      );

      std::cout << b;


      std::list<std::string> init_vals = b.get_init_values();

      std::cout << "initial values of attribute \"" << b.get_name() << "\" are:\n";

      for(auto & i : init_vals)
        {
          std::cout << " - \'" << i << '\'' << std::endl;
        }
    }
  catch (const std::exception& ex)
    {
      std::cerr << "Caught exception:\n" << ex.what() << std::endl;
    }

  return 0;
}
```

examples/relationship.cpp

```
#include <oks/relationship.h>

int main()
{
  try
    {
      OksRelationship r(
■"consists of", /* name */
■"Element", /* class type */
■OksRelationship::Zero, /* low cc in Zero */
■OksRelationship::Many, /* high cc is Many */
■true, /* is composite */
■true, /* is exclusive */
■true, /* is dependent */
■false, /* is not ordered */
■"A structure consists of zero or many elements" /* description */
      );

      std::cout << r;
    }
  catch (const std::exception& ex)
    {
      std::cerr << "Caught exception:\n" << ex.what() << std::endl;
    }

  return 0;
}
```

examples/class.cpp

```
#include <oks/class.h>
#include <oks/attribute.h>
#include <oks/relationship.h>

int main()
{
  try
    {
      OksClass * c = new OksClass(
            "Person", /* class name */
            "Describes a person", /* description */
            false, /* is not abstract */
            0 /* no kernel */
      );
■
      OksAttribute * a = new OksAttribute(
```

```
            "Name",
            OksAttribute::string_type,
            false,
            "",
            "Unknown",
            "Describes person name",
            true
      );

      OksRelationship * r = new OksRelationship(
            "Works at",
            "Department",
            OksRelationship::Zero, OksRelationship::Many,
            false, false, false, false,
            "Can have many work places"
      );

      c->add(a); /* add attribute to class */
      c->add(r); /* add relationship to class */

      std::cout << "Class description is:\n" << *c << std::endl;

      OksClass::destroy(c);
    }
  catch (const std::exception & ex)
    {
      std::cerr << "Caught exception: " << ex.what() << std::endl;
    }

  return 0;
}
```

examples/object.cpp

```
#include <oks/kernel.h>
#include <oks/class.h>
#include <oks/object.h>

int main()
{
  try
    {
      OksKernel k;                          // create OKS kernel

      k.set_silence_mode(true);
      k.new_schema("/tmp/test.schema.xml"); // create new schema
      k.new_data("/tmp/test.data.xml");     // create new data

      // define class 'Person'
      OksClass * p = new OksClass("Person", "Describes a person", false, &k);

      // define attribute 'Name'
      OksAttribute * n = new OksAttribute("Name", OksAttribute::string_type, false, "", "Unknown", "is used to set person's name", tr

      // define attribute 'Birthday'
      OksAttribute * a = new OksAttribute("Birthday", OksAttribute::date_type, false, "", "1997-04-19", "is used to set person's birt

      // add attributes to class
      p->add(n);
      p->add(a);

      // create object
      OksObject * o = new OksObject(p, "aPerson");

      // change 'Name'
      OksData d("Peter");
      o->SetAttributeValue("Name", &d);

      std::cout << *o;
    }
  catch (const std::exception & ex)
    {
      std::cerr << "Caught exception: " << ex.what() << std::endl;
    }

  return 0;
}
```

examples/kernel.cpp

```
#include <oks/kernel.h>
#include <oks/class.h>

int main(int argc, char **argv)
{
  OksKernel k;

  if(argc != 2) return 1;

  k.load_schema(argv[1]);

  std::cout << "Schema file contains:\n";
  for(OksClass::Map::const_iterator i = k.classes().begin(); i != k.classes().end(); ++i)
    std::cout << "\t\"" << i->first << "\" class\n";

  return 0;
}
```

examples/index.cpp

```
#include <oks/kernel.h>
#include <oks/class.h>
#include <oks/object.h>
#include <oks/query.h>

#include <stdlib.h>

#include <sstream>


int main()
{
  try
    {
      // create OKS kernel
      OksKernel k;

      // create new schema and data files
      k.new_schema("/tmp/index.schema");
      k.new_data("/tmp/index.data");

      // define class 'Randomizer'
      OksClass * p = new OksClass("Randomizer", "Describes a Randomizer", false, &k);

      // define attribute 'Value'
      OksAttribute * a = new OksAttribute("Value", OksAttribute::double_type, false, "", "0.5", "random value", false);

      p->add(a);


      // Create 100,000 instances of the class
      size_t i = 0;
      OksDataInfo * odi = p->data_info("Value");

      while (i++ < 100000)
        {
          std::ostringstream s;
          s << i;

          std::string buf = s.str();

          OksObject *o = new OksObject(p, buf.c_str());
          OksData d((double) (random() % (1 << 16)) / (double) (1 << 16));

          o->SetAttributeValue(odi, &d);
        }

      // Create index for attribute 'Value'
      OksIndex index(p, a);

      // Search values >= 0.9 using index
      OksData d(double(0.9));
      std::list<OksObject *> * result = index.FindGreatEqual(&d);

      // Prints number of found instances
      // The expected value should be about 10,000
```

```
      if (result)
        {
          std::cout << "Found " << result->size() << " instances >= 0.9\n";
          delete result;
        }
    }
  catch (const std::exception & ex)
    {
      std::cerr << "Caught exception: " << ex.what() << std::endl;
    }

  return 0;
}
```

examples/data.cpp

```
#include <oks/attribute.h>
#include <oks/object.h>

int main()
{
  OksData d(new OksData::List()); /* creates list */

  d.data.LIST->push_back(new OksData((uint32_t)123456789));
  d.data.LIST->push_back(new OksData((double)123.456789));
  d.data.LIST->push_back(new OksData(boost::posix_time::second_clock::universal_time()));
  d.data.LIST->push_back(new OksData("Class-X", "Obj-1"));

  std::cout.precision(9); /* default is 6 */
  std::cout << d << std::endl;

  return 0;
}
```

examples/method.cpp

```
#include <oks/method.h>

int main()
{
  OksMethod m(
    "print",       /* name */
    "it is a test" /* description */
  );

  m.add_implementation(
    "c++",
    "void print(const char * s)",
    "std::cout << s << std::endl;"
  );

  m.add_implementation(
    "c",
    "void print(const char * s)",
    "printf(\"%s\\n\", s);"
  );

  std::cout << m;

  return 0;
}
```

examples/alloc.cpp

```
#include <sys/time.h>
#include <sys/resource.h>

#include <chrono>
#include <list>
#include <iostream>

#include <boost/pool/pool_alloc.hpp>
#include <tbb/scalable_allocator.h>

  // Normal structure

class Test {
  int i;
```

```
  void * p;
};


  // Use Boost pool (with mutexes)

class Test3 : public Test {

public:
  void * operator new(size_t) {return boost::fast_pool_allocator<Test3>::allocate();}
  void operator delete(void *ptr) {boost::fast_pool_allocator<Test3>::deallocate(reinterpret_cast<Test3*>(ptr));}

};


  // Use Boost pool (without mutexes)

class Test4 : public Test {

public:
  void * operator new(size_t) {return boost::fast_pool_allocator<Test4, boost::default_user_allocator_new_delete, boost::details::pool
  void   operator delete(void *ptr) {boost::fast_pool_allocator<Test4, boost::default_user_allocator_new_delete, boost::details::pool
};

// Use Boost pool (with mutexes)

class Test5 : public Test {

public:
void * operator new(size_t) {return scalable_malloc(sizeof(Test5));}
void operator delete(void *ptr) {scalable_free(ptr);}

};



const size_t ArraySize  =  50000000;
const size_t ListSize   =  50000000;


int main()
{
  size_t count;

  Test  ** t_array  = new Test  * [ArraySize];
  Test3■** t3_array = new Test3 * [ArraySize];
  Test4■** t4_array = new Test4 * [ArraySize];
  Test5 ** t5_array = new Test5 * [ArraySize];

  for(count=0; count < ArraySize; count++) {
    t_array[count] = 0;
    t3_array[count] = 0;
    t4_array[count] = 0;
    t5_array[count] = 0;
  }

  std::cout << "Create and delete " << ArraySize << " objects (" << sizeof(Test) << " bytes per object)\n";

////////////////////////////////////////////////////////////////////////////

  auto tp = std::chrono::steady_clock::now();

  for(count=0; count < ArraySize; count++)
    t_array[count] = new Test();

  for(count=0; count < ArraySize; count++)
    delete t_array[count];

  std::cout << " * standard operators new and delete require " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::s


////////////////////////////////////////////////////////////////////////////

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ArraySize; count++)
    t4_array[count] = new Test4();

  for(count=0; count < ArraySize; count++)
```

```
    delete t4_array[count];

  std::cout << " * Boost operators new and delete require " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::stea


////////////////////////////////////////////////////////////////////////////

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ArraySize; count++)
    t3_array[count] = new Test3();

  for(count=0; count < ArraySize; count++)
    delete t3_array[count];

  std::cout << " * Boost operators new and delete require " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::stea

////////////////////////////////////////////////////////////////////////////

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ArraySize; count++)
    t5_array[count] = new Test5();

  for(count=0; count < ArraySize; count++)
    delete t5_array[count];

  std::cout << " * TBB operators new and delete require " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::steady

////////////////////////////////////////////////////////////////////////////


  delete [] t_array;
  delete [] t3_array;
  delete [] t4_array;
  delete [] t5_array;

////////////////////////////////////////////////////////////////////////////

  std::cout << "Create and free list with " << ListSize << " integers\n";

  std::list<size_t> l;

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ListSize; count++)
    l.push_back(count);

  while(!l.empty()) l.pop_front();

  std::cout << " * operation with list's standard allocator requires " << std::chrono::duration_cast<std::chrono::milliseconds>(std::c

////////////////////////////////////////////////////////////////////////////

  std::list<size_t, boost::fast_pool_allocator<size_t, boost::default_user_allocator_new_delete, boost::details::pool::null_mutex> > l

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ListSize; count++)
    l4.push_back(count);

  while(!l4.empty()) l4.pop_front();

  std::cout << " * operation with list's Boost allocator requires " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chro

////////////////////////////////////////////////////////////////////////////

  std::list<size_t, boost::fast_pool_allocator<size_t> > l3;

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ListSize; count++)
    l3.push_back(count);

  while(!l3.empty()) l3.pop_front();

  std::cout << " * operation with list's Boost allocator requires " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chro

////////////////////////////////////////////////////////////////////////////
```

```
  std::list<size_t, tbb::scalable_allocator<size_t> > l5;

  tp = std::chrono::steady_clock::now();

  for(count=0; count < ListSize; count++)
    l5.push_back(count);

  while(!l5.empty()) l5.pop_front();

  std::cout << " * operation with list's TBB allocator requires " << std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono

////////////////////////////////////////////////////////////////////////////

  return 0;
}
```

examples/profiler.cpp

```
#include <oks/kernel.h>
#include <oks/class.h>

int main(int argc, char **argv)
{
  OksKernel k;

  if(argc != 3) return 1;

  k.set_profiling_mode(true);
  k.load_schema(argv[1]);
  k.load_data(argv[2]);

  return 0;
}
```

11. The oks_tutorial.cpp Canonical Tutorial (verbatim)

The self-contained OKS tutorial program (Car / Manufacturer / Garage example classes), from `oks_utils/src/bin/oks_tutorial.cpp`.

```
/************************************************************************
*                                                                       *
* tutorial.cpp                                                          *
*        Explains basics of OKS C++ API including:                      *
*                - kernel initialization                                *
*                - schema design                                        *
*                - data creation                                        *
*                - data manipulation                                    *
*                - notification                                         *
*                                                                       *
* Author Igor Soloviev                                                  *
*                                                                       *
* Created: 14 Oct 1996                                                  *
*                                                                       *
* Modified:                                                             *
*        11 Feb 1998                                                    *
*                - add database quering                                 *
*                                                                       *
************************************************************************/

#include <boost/date_time/gregorian/gregorian.hpp>

#include <oks/kernel.h>
#include <oks/object.h>
#include <oks/class.h>
#include <oks/attribute.h>
#include <oks/relationship.h>
#include <oks/query.h>
#include <oks/exceptions.h>


  //
  // This function sets attribute values of class 'Person'
  //

void
setPersonValues(
  OksObject *o,                    // OKS object describing person
  const char *name,                // new person name
  boost::gregorian::date birthday, // new person birthday
  const char *familySituation      // new person family situation
)
{
  OksData d;                       // creates OKS data with unknown type

  d.Set(name);                     // sets OKS data to string 'name'
  o->SetAttributeValue("Name", &d);

  d.Set(birthday);                 // sets OKS data to date 'birthday'
  o->SetAttributeValue("Birthday", &d);

  d.Set(familySituation);          // sets OKS data to 'familySituation'
  d.type = OksData::enum_type;     // sets OKS data type to enumeration
  o->SetAttributeValue("Family Situation", &d);
}


  //
  // This function prints an instance of class 'Person'
  //

void
printPerson(
  const OksObject *o               // OKS object describing person
)
{
  OksData * name(o->GetAttributeValue("Name"));                // is used to store 'Name'
  OksData * birthday(o->GetAttributeValue("Birthday"));        // is used to store 'Birthday'
  OksData * family(o->GetAttributeValue("Family Situation"));  // is used to store 'Family Situation'

  std::cout << "Object " << o << " \n"
```

```
            " Name: " << *name << " \n"
            " Birthday: \'" << *birthday << "\" \n"
            " Family Situation: " << *family << std::endl;
}


  //
  // This function sets attribute values of class 'Employee'
  //

void
setEmployeeValues(
  OksObject *o,                   // OKS object that describes employee
  const char *name,               // new employee name
  boost::gregorian::date birthday, // new employee birthday
  const char *familySituation,    // new employee family situation
  uint32_t salary                 // new employee salary situation
)
{
    // we can use setPersonValues() because 'Employee' class
    // derived from 'Person' class

  setPersonValues(o, name, birthday, familySituation);

  OksData d(salary);              // creates OKS data with ulong 'salary'
  o->SetAttributeValue("Salary", &d);
}


  //
  // This function prints an instance of class 'Employee'
  //

void
printEmployee(
  const OksObject *o              // OKS object that describes employee
)
{
    // we can use printPerson() because 'Employee' class
    // is derived from 'Person' class

  printPerson(o);

  OksData *department(o->GetRelationshipValue("Works at")),  // is used to store 'Works at'
          *salary(o->GetAttributeValue("Salary"));           // is used to store 'salary'

  std::cout << " Salary: " << *salary << " \n"
               " Works at: \"" << department->data.OBJECT->GetId() << "\"\n";
}


  //
  // This function sets attribute values of class 'Department'
  //

void
setDepartmentValues(
  OksObject *o,                   // OKS object that describes department
  const char *name                // new department name
)
{
  OksData d(name);                // creates OKS data with string 'name'

  o->SetAttributeValue("Name", &d);
}


  //
  // This function prints an instance of class 'Department'
  //

void
printDepartment(
  const OksObject *o              // OKS object that describes department
)
{
  OksData *name(o->GetAttributeValue("Name")),       // is used to store 'Staff'
          *staff(o->GetRelationshipValue("Staff"));  // is used to store 'Name'
```

```
  std::cout << "Object " << o << "\n"
               " Name: " << *name << " \n"
               " Staff: \"" << *staff << "\"\n";
}


int
main(int argc, char **argv)
{
  const char * schema_file = "/tmp/tutorial.oks"; // default schema file
  const char * data_file   = "/tmp/tutorial.okd"; // default data file

  if(
    argc > 1 &&
    (
      !strcmp(argv[1], "--help") ||
      !strcmp(argv[1], "-help") ||
      !strcmp(argv[1], "--h") ||
      !strcmp(argv[1], "-h")
    )
  ) {
    std::cout << "Usage: " << argv[0] << " [new_schema new_data]\n";
    return 0;
  }

  if(argc == 3) {
    schema_file = argv[1];
    data_file = argv[2];
  }


    // Creates OKS kernel

  std::cout << "[OKS TUTORIAL]: Creating OKS kernel...\n";

  OksKernel kernel(false, false, false, false);

  std::cout << "[OKS TUTORIAL]: Done creating OKS kernel\n\n";


  try {


      // Creates new schema file and tests return status

    std::cout << "[OKS TUTORIAL]: Creating new schema file...\n";

    OksFile * schema_h = kernel.new_schema(schema_file);

    std::cout << "[OKS TUTORIAL]: Done creating new schema file...\n\n"
                 "[OKS TUTORIAL]: Define database class schema...\n\n"
                 "  **********        ************     1..1 **************\n"
                 "  * Person *<|------* Employee *--------<>* Department *\n"
                 "  **********        ************ 0..N     **************\n\n";


      // Creates class Person with three attributes "Name", "Birthday" and "Family Situation"

    OksClass * Person = new OksClass(
      "Person",
      "It is a class to describe a person",
      false,
      &kernel
    );

    Person->add(
      new OksAttribute(
        "Name",
        OksAttribute::string_type,
        false,
        "",
        "Unknown",
        "A string to describe person name",
        true
      )
    );

    OksAttribute * PersonBirthday = new OksAttribute(
```

```
  "Birthday",
  OksAttribute::date_type,
  false,
  "",
  "2009/01/01",
  "A date to describe person birthday",
  true
);

Person->add(PersonBirthday);

Person->add(
  new OksAttribute(
    "Family Situation",
    OksAttribute::enum_type,
    false,
    "Single,Married,Widow(er)",
    "Single",
    "A enumeration to describe a person family state",
    true
  )
);


  // Creates class Person with superclass Person, add "Salary" attribute and "Works at" relationship

OksClass * Employee = new OksClass(
  "Employee",
  "It is a class to describe an employee",
  false,
  &kernel
);

OksAttribute * EmployeeSalary = new OksAttribute(
  "Salary",
  OksAttribute::u32_int_type,
  false,
  "",
  "1000",
  "An integer to describe employee salary",
  false
);

OksRelationship * WorksAt = new OksRelationship(
  "Works at",
  "Department",
  OksRelationship::One,
  OksRelationship::One,
  false,
  false,
  false,
  false,
  "A employee works at one and only one department"
);

Employee->add_super_class("Person");
Employee->add(EmployeeSalary);
Employee->add(WorksAt);


  // Creates class Department with one attribute and one relationship

OksClass * Department = new OksClass(
  "Department",
  "It is a class to describe a department",
  false,
  &kernel
);

OksAttribute * DepartmentName = new OksAttribute(
  "Name",
  OksAttribute::string_type,
  false,
  "",
  "Unknown",
  "A string to describe department name",
  true
);
```

```
OksRelationship *DepartmentStaff = new OksRelationship(
  "Staff",
  "Employee",
  OksRelationship::Zero,
  OksRelationship::Many,
  true,
  true,
  true,
  false,
  "A department has zero or many employess"
);

Department->add(DepartmentName);
Department->add(DepartmentStaff);


  // Saves created schema file

std::cout << "[OKS TUTORIAL]: Saves created OKS schema file...\n";

kernel.save_schema(schema_h);


  // Creates new data file and tests return status

std::cout << "[OKS TUTORIAL]: Creating new data file...\n";

OksFile * data_h = kernel.new_data(data_file, "OKS TUTORIAL DATA FILE");

data_h->add_include_file(schema_file);


  // Creates instances of the classes

OksObject * person1     = new OksObject(Person, "peter");
OksObject * person2     = new OksObject(Person, "mick");
OksObject * person3     = new OksObject(Person, "baby");
OksObject * employee1   = new OksObject(Employee, "alexander");
OksObject * employee2   = new OksObject(Employee, "michel");
OksObject * employee3   = new OksObject(Employee, "maria");
OksObject * department1 = new OksObject(Department, "IT");
OksObject * department2 = new OksObject(Department, "EP");


  // Sets attribute values for instances of class 'Person'

setPersonValues(person1, "Peter", boost::gregorian::from_string("1960/02/01"), "Married");
setPersonValues(person2, "Mick", boost::gregorian::from_string("1956-09-01"), "Single");
setPersonValues(person3, "Julia", boost::gregorian::from_string("2000-May-25"), "Single");


  // Sets attribute values for instances of class 'Employee'

setEmployeeValues(employee1, "Alexander", boost::gregorian::from_string("1972/05/12"), "Single", 3540) ;
setEmployeeValues(employee2, "Michel", boost::gregorian::from_string("1963/01/28"), "Married", 4950) ;
setEmployeeValues(employee3, "Maria", boost::gregorian::from_string("1951/08/18"), "Widow(er)", 4020) ;


  // Sets attribute values for instance of class 'Department'

setDepartmentValues(department1, "IT Department");
setDepartmentValues(department2, "EP Department");


  // Sets relationships

department1->AddRelationshipValue("Staff", employee1);
department1->AddRelationshipValue("Staff", employee2);
department2->AddRelationshipValue("Staff", employee3);
employee1->SetRelationshipValue("Works at", department1);
employee2->SetRelationshipValue("Works at", department1);
employee3->SetRelationshipValue("Works at", department2);


  // Print out database contents

std::cout << "\n[OKS TUTORIAL]: Database contains the following data:\n";

printPerson(person1);
```

```
printPerson(person2);
printPerson(person3);
printEmployee(employee1);
printEmployee(employee2);
printEmployee(employee3);
printDepartment(department1);
printDepartment(department2);


  // Saves created data file

std::cout << "\n[OKS TUTORIAL]: Saves created OKS data file...\n";

kernel.save_data(data_h);


std::cout << "[OKS TUTORIAL]: Done with saving created OKS data file\n";


  // ************************************************************************
  //   ***** At this moment the database has been created and saved *****
  // ************************************************************************


std::cout << "\n[OKS TUTORIAL]: Start database querying tests\n\n";


  // This is a test for OksComparator and OksQuery

{
  std::cout << "[QUERY]: Start simple database querying...\n";

    // Looking for persons were born after 01 January 1960

  boost::gregorian::date aDate(boost::gregorian::from_string("1960/01/01")); // query date
  OksQuery query(true, new OksComparator(PersonBirthday, new OksData(aDate), OksQuery::greater_cmp));


    // executes query

  std::cout << "[QUERY]: Looking for persons were born after " << aDate << " ...\n\n";

  OksObject::List * queryResult = Person->execute_query(&query);


    // builds iterator over results if something was found

  if(queryResult) {
    std::cout << "[QUERY]: Query \'" << query
              << "\'\n  founds the following objects in class \'"
              << Person->get_name() << "\' and subclasses:\n";

    for(OksObject::List::iterator i = queryResult->begin(); i != queryResult->end(); ++i) {
      OksObject * o = *i;
      OksData * d(o->GetAttributeValue("Birthday"));
      std::cout << "   - " << o << " was born " << *d << std::endl;
    }


      // free list: we do not need it anymore

    delete queryResult;
  }

  std::cout << "[QUERY]: Done with simple database querying\n\n";
}

{
  std::cout << "[QUERY]: Start database querying with logical function...\n";


      // Looking for persons were born after 01 January 1960
      // and before 01 January 1970

  boost::gregorian::date lowDate(boost::gregorian::from_string("1960/01/01"));  // low date
  boost::gregorian::date highDate(boost::gregorian::from_string("1970/01/01")); // high date

  OksAndExpression * andExpression = new OksAndExpression();
```

```
  andExpression->add(new OksComparator(PersonBirthday, new OksData(lowDate), OksQuery::greater_cmp));
  andExpression->add(new OksComparator(PersonBirthday, new OksData(highDate), OksQuery::less_cmp));

  OksQuery query(true, andExpression);


    // executes query

  std::cout << "[QUERY]: Looking for persons were born between " << lowDate
            << " and " << highDate << " ...\n\n";

  OksObject::List * queryResult = Person->execute_query(&query);


    // builds iterator over results if something was found

 if(queryResult) {
    std::cout << "[QUERY]: Query \'" << query
              << "\'\n  founds the following objects in class \'"
              << Person->get_name() << "\' and subclasses:\n";

    for(OksObject::List::iterator i = queryResult->begin(); i != queryResult->end(); ++i) {
      OksObject * o = *i;
      OksData * d(o->GetAttributeValue("Birthday"));
      std::cout << "   - " << o << " was born " << *d << std::endl;
    }


      // free list: we do not need it anymore

    delete queryResult;
  }

  std::cout << "[QUERY]: Done database querying with logical function\n\n";

}

{

  std::cout << "[QUERY]: Start database querying with relationship expression...\n";


    // Looking for employee were born after 01 January 1971
    // and which works at IT Department

  boost::gregorian::date aDate(boost::gregorian::from_string("1971/01/01"));  // a date
  const char * departmentName = "IT Department";

  OksAndExpression * andExpression = new OksAndExpression();

  andExpression->add(new OksComparator(PersonBirthday, new OksData(aDate), OksQuery::greater_cmp));
  andExpression->add(new OksRelationshipExpression(WorksAt, new OksComparator(DepartmentName, new OksData(departmentName), OksQue

  OksQuery query(true, andExpression);


    // executes query

  std::cout << "[QUERY]: Looking for employee were born after " << aDate
            << " and works at department " << departmentName << " ...\n\n";

  OksObject::List * queryResult = Employee->execute_query(&query);


    // builds iterator over results if something was found

 if(queryResult) {
    std::cout << "[QUERY]: Query \'" << query
              << "\'\n  founds the following objects in class \'"
              << Employee->get_name() << "\' and subclasses:\n";

    for(OksObject::List::iterator i = queryResult->begin(); i != queryResult->end(); ++i) {
      OksObject * o = *i;
      OksData * d(o->GetAttributeValue("Birthday"));
      std::cout << "   - " << o << " was born " << *d << std::endl;
    }

      // free list: we do not need it anymore
```

```
        delete queryResult;
      }

      std::cout << "[QUERY]: Done database querying with relationship expression...\n";
    }

    std::cout << "\n[OKS TUTORIAL]: Done with database querying tests\n\n";

      // It is not necessary to free data or schema because all instances of OKS
      // classes are automatic C++ objects and they will deleted before last
      // close bracket `}`.

  }
  catch (const std::exception & ex) {
    std::cerr << "Caught exception:\n" << ex.what() << std::endl;
  }

    // returns success

  return 0;
}
```

12. The OKS Data Editor Query Window — GUI Query Builder (user manual)

Text content of `data/online-help/data-editor/QueryWindow.html` (CERN GitLab, oks_utils) — the authoritative user-facing explanation of how a query is built interactively, and how it maps onto the grammar in Sections 2–4.

Query Window

OKS Data Editor

Query Window

The OKS Data Editor provides database query builder. Created query can be used by user application via OKS and config APIs. An OKS query can be edited visually, saved in a file, loaded from file and executed. The query window consists of two main parts: the graphical query constructor and the result table:

Building query

To begin building a query, select desired class from the main window or from the class window, press right mouse button and select [Query] item from popup menu. OKS Data Editor will bring up query window with empty query form.

It is necessary to decide, if the query scope is limited by objects of the selected class, or also include objects of derived classes. This can be changed using toggle button [Search in subclasses]: if it is selected, the query will be performed over class and all subclasses, otherwise scope will be one class only.

Then press right mouse button in empty query form:

The attribute expression allows to run query selecting objects by attribute or UID value. The relationship expressions allow to apply attribute expression on referenced objects. The "Not", "And" and "Or" expressions allow to build query using arbitrary number of attribute and relationship expressions.

Attribute Expression

The attribute expression query form is simplest query form and it describes search by value of single attribute, for example "search all objects with initialisation timeout greater 20":

This form consists of the following items:

Selection radio-group "Attribute Value" vs. "Object ID". If "Attribute Value" is selected, the value of attribute is compared. Else, the object UID is compared.

Attribute option menu allows to select an attribute defined in class (disabled, if the "Object ID" is selected above).

Value text field allows to put any desired value of attribute that will be used during the search;

Comparator allows to define a function that will be used by query and predefined functions are:

"=" - equal,

"!=" - not equal,

"~=" - like (use regular expression),

"<=" - less or equal,

">=" - great or equal,

"<" - less,

">" - great.

The attribute name can be changed at any moment. If the attribute type of new selected attribute is different from previous one, the value can be converted to default of selected attribute type.

Relationship Expression

The relationship expression query form describes search through relationship attribute using nested query form, for example "search all segments which have infrastructure applications defining infrastructure environment variables with names ended by IS_SERVER":

This form consists of the following items:

Relationship option menu allows to select a relationship defined in class.

"Some Objects Match Query" vs. "All Objects Match Query" toggle button defines either an object referenced by relationship matches query from nested query form or at least one object referenced by relationship matches the query.

Nested Query Form allows to build nested query expression (e.g. attribute or even nested relationship expression). Press right mouse button to see popup menu and use the same rules to build query as for top level query form. Note, once the nested form is inserted, it is not possible to change the relationship name (the option menu becomes disabled).

Logical Expressions

Logical Query Expression Form can be used with any type of query form to build complex query expression. There are three types of logical query expressions:

"Not" query expression

"And" query expression

"Or" query expression

The "Not" query expression form can be concatenated with any another single query expression form. The "And" query expression form and "Or" query expression form can be used with any two or more query expression forms.

It is possible to build multi-level tree structure that consists of logical query expression forms. The leaves of that structure must be either attribute or relationship query expression form. An incomplete tree and popup menu is shown below:

Executing Query

To execute a complete OKS query it is necessary either to create it (see Building Query) or to load already existing one (see Loading Query). Press right mouse button on any free space of the query form and select [Execute Query] item from popup menu:

The result of query will appear in the query result table:

Saving Query

To save a complete query to file press right mouse button on any free space of Query Form and select Save Query item from popup menu. This will bring up dialog with prompt for query file name. Enter desired filename and press OK button, e.g.:

Note, incomplete query can not be saved.

The format of the query file is simple (it looks like a statement of LISP language) and can be edited manually by any text editor.

An OKS query does not strongly depend from class type. If two classes have an attribute with the same name, possibly a query can be applied to both classes.

Loading Query

To load a query, select query related class from main window or class window, press right mouse button and select [Load Query] item from popup menu. The OKS Data Editor will bring up Open OKS Query window. Choose query file name and the Query window with stored query will appear. Home - Next - Index

Modified 11-JUN-2009

Author Igor Soloviev

13. CERN oks Repository — Full Release-Notes History (tdaq-01 .. tdaq-13)

Complete, unabridged `doc/RELEASE_NOTES.md` of the CERN `atlas-tdaq-software/oks` repository — every dated entry documenting the evolution of the query grammar, the C++ API, the GUI, and the git-based versioning system, from the earliest tdaq tags through the current tdaq-13-01-00.

OKS

tdaq-13-01-00

Improve get repository versions

- add support for Git date formats like concrete date "2026-06-15", or a relative date such as "2 years 1 day 3 minutes ago"
- add support for Git branches

Improve creation of new files

- if TDAQ_DB_REPOSITORY is defined, new files are created in Git repository working area, otherwise in current working directory
- create sub-directories for new files if they do not exist
- add `OksKernel::import_file(to, from)` method to add external file into working area, where `to` should be a repository filename, e.g. `test/my.schema.xml`

tdaq-12-00-00

Add update data function

- new function keeps xml document layout and user comments unchanged only updating what has been changed
- contrary, the save-data function rewrites xml data file completely removing user comments and saves data using standard layout
- the update-data is integrated into OKS data editor in addition to previous save-data

tdaq-11-02-00

- use `tbb::scalable_allocator` allocator instead of `boost::fast_pool_allocator`
- add debug info (`.git/oks_proc_info`) when clone oks repository
- use `flock` on `/var/tmp` before entering `oks-commit.sh` pull/push section to avoid issue when a user runs multiple commits on the same node
- add `ordered` flag into oks relationship constructor to be used by rdb's `oks copy`

tdaq-10-00-00

- set `pull.rebase=true` when clone oks repository to avoid misleading merge commits by users

tdaq-09-03-00

Show hidden oks data xml attributes for text editors

Use new oks_refresh utility with -x option to enforce all attributes to be shown.

Deprecated OWL date/time format

The OWL date/time formats are deprecated and will be removed in next public release. ERS warning is reported by oks library, when a file containing data stored in deprecated format is loaded.

Such file can be refreshed using oks editors or new **oks_refresh** utility. For example:

```
$ git clone <url> .
$ TDAQ_DB_PATH=`pwd`:$TDAQ_DB_PATH oks_refresh -f ./<file>
$ git commit -m "refresh <file> to update date/time format" ./<file>
$ git push origin master
```

Ordering of multi-value attributes and relationships

Use new **ordered** attribute for multi-value attribute or relationship to sort its data on save. The default value is **no**.

Implement sort of multi-value attributes and relationships in OKS data editor.

tdaq-09-02-01

Postponed changes

The postponed changes are stored into a git branch and should not be applied immediately in the ongoing data taking session. If needed, they can be shared with other experts and tested using DAQ control and configuration tools. At an appropriate moment the branch will be merged with master branch and deleted.

When branch is created, the gitea pull request (also known as [merge request](https://docs.gitlab.com/ee/user/project/merge_requests/) in gitlab) can be created. All gitea pull requests can be accessed from a single point and applied using gitea web interface. In future, the pull requests will be integrated with DAQ Shifter Assistant.

A git branch can be created and updated using git, oks and gitea web interfaces.

Create and update branch

Create or checkout a branch into temporary private area:

```
$ cd `oks_clone_repository -b <branch-name>`
```

A branch can also be created using git command line and gitea web interfaces.

Modify oks files

Edit changes using config tools, a text editor or gitea web interface, for example:

```
$ export TDAQ_DB_USER_REPOSITORY=`pwd`
$ oks_data_editor x/y/z
$ vim x/y/z
```

Commit and push modifications

Use oks commit:

```
$ oks-commit.sh -m 'put here your commit message' -u `pwd`
```

or git command line interface:

```
$ git commit -m 'put here your commit message' x/y/z
$ git push origin <branch-name>
```

Edit branch script

The oks edit branch utility allows to create new or checkout existing git branch, modify and commit changes into it. It performs steps similar to described in three above sections.

Run:

```
oks-edit-branch.sh -h
```

to get more information about command line parameters and required process environment settings.

Create gitea pull request (recommended)

Create a new pull request for your branch using gitea Web interface (you need to be authorised first) using URL:

```
<gitea-release-url>/compare/master...<branch-name>
```

Merge postponed changes

When the changes have to be merged with the master branch, use git command line interface:

```
$ git checkout master
$ git merge <branch-name>
$ git push origin master
$ git push origin --delete <branch-name>
$ git branch -D <branch-name>
```

or, if the gitea pull request was created, use gitea web interface. Browse all pull requests:

```
<gitea-release-url>/pulls
```

Select your pull request, merge changes and delete branch.

How to run partition or use config-based tools

In case, if the postponed changes need to be validated running DAQ partition, a shared file system has to be used to checkout a branch. For example create it in NFS area:

```
$ export SHARED=<some-path> # e.g. "/tbed/scratch/`whoami`/$CMTRELEASE" on TestBed or "$HOME/oks/$CMTRELEASE" on Point-1
$ mkdir -p $SHARED
$ cd $SHARED
$ oks_clone_repository -b <branch-name> -o .
$ chgrp -R zp .
$ chmod -R g+w .
$ export TDAQ_DB_USER_REPOSITORY=`pwd`
```

Then edit and commit changes to branch and run setup:

```
$ setup_daq <path-to-partition-file> <partition-name>
```

Undo commit

If there is a need to undo already committed and pushed changes, the [git revert](https://git-scm.com/docs/git-revert) command has to be used.

To undo first commit, one has to:

1. clone git repository

2. find hash of the wrong commit

3. run the revert git command with this hash

4. push changes to git

For example:

- clone repository and run [git log](https://git-scm.com/docs/git-log) to see details of recent commits:

```
$ cd `oks_clone_repository`
$ git log -5
```

- find wrong commit in above log, run git revert command and push changes:

```
$ git revert --no-edit ab12... # use real commit hash
$ git push origin master
```

At this point the wrong commit is undone. The repository is fixed and its contains information about both, the wrong commit and its reverting.

tdaq-09-01-00

Jira: [ADTCC-214](https://its.cern.ch/jira/browse/ADTCC-214), [ADTCC-226](https://its.cern.ch/jira/browse/ADTCC-226) and [ADTCC-227](https://its.cern.ch/jira/browse/ADTCC-227)

[TWiki page](https://twiki.cern.ch/twiki/bin/view/Atlas/DaqHltOks#4_OKS_Git_Repository).

- Replace cvs repository by git repository.***

The oks repository is stored on a git server. To access oks configuration the git repository is cloned into a temporal area. When changes are committed, the git server validates them verifying consistency of all repository files and checking user permissions as defined by the Access Manager policy. If commit is successful, the git server updates latest version of oks files on the filesystem (git repository mapping). Contrary to cvs implementation, the repository mapping is not required to access oks data using git repository. It is only implemented for convenience of users (fast browse of files) or filesystem-based access.

This is the main difference between oks git and cvs repository implementations:

- the oks git implementation uses git database to store and access oks files (like DBMS) and ignores the files on the filesystem;
- the oks cvs implementation used filesystem to store and access oks files.

The ignorance of files on the filesystem by oks git is done on purpose to preserve used configurations, and to implement configuration archiving on top of git database.

If user needs to access configuration from files stored on filesystem, it is necessary to disable oks git as explained in next section.

User git repository permissions

The permissions to update files in oks git repository are defined by the [Access Manager](https://twiki.cern.ch/twiki/bin/view/Atlas/DaqHltAccessManager) policy rules and enabled user roles.

Environment Variables

The TDAQ_DB_REPOSITORY is the only setting that enables or disables use of the oks git repository. If it is set, the oks git repository is used. If not set, the filesystem repositories are used.

There are 3 shell functions available after release setup on TestBed and Point-1 to enable, disable and get status of the oks git repository:

- oks-git-on - to enable use of the oks git repository
- oks-git-off - to disable use of the oks git repository
- oks-git-status - to give status of the oks git repository use

The variable TDAQ_DB_REPOSITORY contains one or several git URLs (it does not point to a filesystem anymore). If there are several URLs, the used one depends on the OKS_GIT_PROTOCOL variable. If it is not set, the first URL is used. To get URL use oks_git_repository utility, e.g. to clone repository:

```
$ git clone `oks_git_repository`
```

The TDAQ_DB_USER_REPOSITORY can point to user git area. If it is set, oks-based tools will not clone git repository, but will use that area instead. It is responsibility of user to create that area and remove when not needed.

The TDAQ_DB_VERSION can be used to access particular revision of oks git repository. It is used internally by setup_daq to preserve concrete revision to be used by data taking session. In case of configuration reload this variable

is set into a new value and distributed to all processes accessing oks configuration. The variable can be used in two formats:

- hash:$value - select revision by explicit hash
- date:$value - select latest revision before given date or timestamp

If the variable is not defined, the latest revision will be checked out.

The variable OKS_REPOSITORY_MAPPING_DIR points to the repository mapping.

As before, the TDAQ_DB_PATH can be used for a filesystem-based access and can contain several colon-separated filesystem repositories. With filesystem based access, the oks data from git repository can be included using the repository mapping variable, for example:

```
$ unset TDAQ_DB_REPOSITORY
$ export TDAQ_DB_PATH=$OKS_REPOSITORY_MAPPING_DIR:/det/a/b/c:/home/x/y/z
```

Filepaths

To access an oks file from git repository one has to use its repository filename, for example:

```
$ oks_data_editor daq/segments/setup.data.xml
```

One should not use any filesystem relative or absolute paths referencing oks git repository files. There are two exceptions.

Filepaths relative to repository mapping

If the filepaths are relative to the OKS_REPOSITORY_MAPPING_DIR. That is done to help users with shell path-completion (hitting _TAB_ key), for example the following commands will correctly checkout the files from git repository and run appropriate tools:

```
$ export OKS_REPOSITORY_MAPPING_DIR=/tbed/git/oks/tdaq-99-00-00
$ oks_data_editor /tbed/git/oks/tdaq-99-00-00/daq/partitions/part_hlt.data.xml
$ setup_daq /tbed/git/oks/tdaq-99-00-00/daq/partitions/part_hlt.data.xml part_hlt
```

However, the preferable way is to use the repository filenames, e.g.:

```
$ oks_data_editor daq/partitions/part_hlt.data.xml
$ setup_daq daq/partitions/part_hlt.data.xml part_hlt
```

Filepaths relative to user repository

If the files are checked out by user, one should set TDAQ_DB_USER_REPOSITORY to access them.

The following example will result an error, since user provides path relative to current working directory:

```
$ cd `oks_clone_repository`
$ oks_dump -f oks_dump -f daq/segments/setup.data.xml
```

To make it working it is necessary to set the above variable:

```
$ cd `oks_clone_repository`
$ export TDAQ_DB_USER_REPOSITORY=`pwd`
$ oks_dump -f oks_dump -f daq/segments/setup.data.xml
```

OKS tools

The tools from previous implementation can be used with oks git repository.

oks_dump

As in the previous implementation, the oks_dump can be used for fast check of oks configuration. Now it shows details of git operations, if the git repository is used, e.g.:

```
$ oks_dump -f daq/segments/setup-initial.data.xml
2020-Aug-11 11:01:24.524 [OKS checkout] => oks-checkout.sh -u /tmp/oks.ou9Qxy
git clone -q -n ssh://gitea@pc-tdq-git.cern.ch/oks/tdaq-09-01-00.git .
git checkout -q
```

```
checkout oks version eb175611bc85f3061541cc20c62be8a00b4b26e1
2020-Aug-11 11:01:26.377 [OKS checkout] => done in 46.548 ms
Reading data file "/tmp/oks.ou9Qxy/daq/segments/setup-initial.data.xml" in normal format (51591 bytes)...
...
```

oks-import.sh

As before, this tool allows to import or to update user files from previous repositories or release installation areas. Change current working directory to a repository root and import required files and directories.

For example, to import some files from TDAQ release:

```
bash$ cd ${TDAQ_INST_PATH}/share/data
bash$ oks-import.sh -m "import release files" daq/schema daq/sw/*.data.xml daq/segments/common-environment.data.xml
```

To import some sub-detector folders from previous release:

```
bash$ cd /atlas/oks/tdaq-09-00-00
bash$ oks-import.sh -m "commit message" tile/hw tile/sw
```

OKS data editor

New functionality:

- commit updated files into git repository
- show notification about new commits in the git repository and merge local changes with master branch
- browse archived versions in the git repository and switch to an archived version

There is no need to create a temporal user repository, it is done automatically.

Direct use of GIT interface

The git interface is exposed to user. It is possible to clone git repository, copy, delete files, use any editors to update them and commit changes back to the git repository, e.g.:

```
$ cd `mktemp -d`
$ git clone `oks_git_repository` .
$ modify, copy, remove any files
$ git commit -m 'describe changes' x y z ...
$ git push origin
```

Above, first two lines normally can be replaced by:

```
$ cd `oks_clone_repository`
```

There are web interfaces to see historical changes and perform minimal modifications using web text editors on [Point-1](http://pc-atlas-www.cern.ch/gitea) and [TestBed](http://pc-tbed-git.cern.ch/gitea).

The CERN gitlab [atlas-tdaq-oks](https://gitlab.cern.ch/atlas-tdaq-oks) project is a read-only mirror of Point-1 and TestBed repositories. It can be accessed world-wide by authenticated ATLAS users.

All changes have to be stored into _master_ branch. The other branches are not yet supported (for the moment they will be rejected by git server). In future they are planned to be used for merge requests, when the committed changes have to be ignored during few next runs.

Undo commits

To undo changes use git revert. Avoid use commands causing losing commit history such as git reset.

Clone git repository. Then get the hash code of last commit, revert that commit and push changes:

```
$ git log -1
$ git revert <hash_code_from_git_log>
$ git push
```

Archiving

The oks2coral archiving in Oracle is disabled. If files are stored in git repository, they are automatically archived into git database.

The run number database stores a reference to the used config version and repository filename.

```
$ rn_ls -c "oracle://atonr_adg/rn_r" -w ATLAS_RUN_NUMBER -s '2020-07-31T12:00:00' -t '2020-08-02T12:00:00' -a '%xml'
=============================================================================================================================================================
|    Name |    Num | Start At (UTC)           |    Duration |   User |                 Host | Partition |                                                    |
=============================================================================================================================================================
| point-1 | 380689 | 2020-Jul-31 16:33:59.818 | 0:00:12.248 | isolov | pc-tdq-onl-05.cern.ch | all_hosts | hash:6800fe3b63a18859ef688
| point-1 | 380688 | 2020-Jul-31 14:52:26.896 |             | tdaqsw | pc-tdq-onl-01.cern.ch |   initial | hash:2c39688d965cbac84832e
=============================================================================================================================================================
```

In turn, the git repository is tagged by run number and partition name.

To clone git repository use *oks_clone_repository* utility. The user can specify output directory and particular version by commit hash, date or tag. For example checkout by tag configuration used for run 380689 and partition *all_hosts*:

```
$ oks_clone_repository --version tag:r380689@all_hosts
```

or by hash (one can provide first few characters of the hash, as long as that partial hash is at least four characters long and unambiguous), e.g.:

```
oks_clone_repository --version hash:6800fe3b
```

tdaq-08-03-01

OKS data file format

Jira: [ADTCC-185](https://its.cern.ch/jira/browse/ADTCC-185)

Store values inside tags

Store attribute and relationship data inside values of tags

Format for single value attributes

```
<attr name="xxx" type="yyy" val="zzz"/>
```

Format for multi value attributes

```
<attr name="xxx" type="yyy">
 <data val="zzz1"/>
 <data val="zzz2"/>
</attr>
```

Format for 0..1 and 1..1 relationships

```
<rel name="xxx" class="yyy" id="zzz"/>
```

Format for 0..N and 1..N relationships

```
<rel name="xxx">
  <ref class="yyy1" id="zzz1">
  <ref class="yyy2" id="zzz2">
</rel>
```

Skip empty data

Do not store attributes with values equal to empty initial and empty relationships

Compatibility and data conversion

The changes are backward compatible. New OKS library is able to read old "extended" and "compact" data formats.

For conversion open data file stored in old format using OKS Data Editor or DBE and save it.

Do not mixture old and new formats in the same file, when update file in a text editor.

tdaq-06-00-00

Change a meaning of the OKS attribute range for string data type. Before the tokens of range were used for lexical comparison of the string. Now regular expression is used instead.

To enforce a value of string attribute be non empty one can use ".+" regular expression for range.

If attribute range for string type is defined, initial value of that attribute may be required to validate schema, for example non-empty string may not have default value set to empty string.

If an OKS schema had range defined for a string attribute for previous release used for configuration databases it has to be converted to enumeration type. No other changes are needed for a programming code using config and generated DAL packages.

For IS schema the range is not needed, since IS does not validate values of IS objects, so irrespectively to any range defined for IS attribute, the IS will allow to put any value. It is up to user decide to remove range in this case, or to convert attribute type from string to enumeration. However in the latter case the programming code using such schema may need be changed.

tdaq-04-00-00

• Fix bug with wrong objects list returned by *referenced_by()* method from config package. The patch fixes calculation of RCRs by _OksObject::SetRelationshipValue(const OksDataInfo * odi, OksData * d)_ method (see bug [82615](https://savannah.cern.ch/bugs/?82615)).

• Fix problem when OKS file was created incorrectly by third-party tools: if creation date tag in the info section is missing (see bug [70563](https://savannah.cern.ch/bugs/?70563)), after saving by OKS on reload it resulted "not-a-date-time" error.

• Fix several internal problems connected with execution of OKS code in several threads and several OKS kernels discovered during RDB writer server exploitation:

• bug [76158](https://savannah.cern.ch/bugs/?76158): the server may go into error state when several clients are updating OKS server repository;

• bug [78326](https://savannah.cern.ch/bugs/?78326): the server may crash, when several clients actively update database caused by fast boost allocator with null mutex;

• bug [82762](https://savannah.cern.ch/bugs/?82762): the server may crash under certain conditions during database reload.

tdaq-02-01-00

To simplify future release patching procedure the oks package was split on two packages: oks and oks_utils. The oks package only contains library. All utilities including editors, relational oks and oks server were moved to oks_utils package.

tdaq-02-00-03

• use Boost date and time format instead of OWL package classes (patched in tdaq-02-00-03)

• performance improvements of XML files parsing (partly implemented as tdaq-02-00-03 patch)

• substitute round brackets variable name by value before error reporting (patch [3619](https://savannah.cern.ch/patch/index.php?3619))

• checking date and time attribute initial values (patch [3656](https://savannah.cern.ch/patch/index.php?3656))

• provide a possibility for fast objects destruction required for RDB server (patch [3798](https://savannah.cern.ch/patch/?3798) and [62853](https://savannah.cern.ch/bugs/index.php?62853))

• throw exception, if a schema file is modified on data files reload (patch [3798](https://savannah.cern.ch/patch/?3798))

• add mode, when duplicated classes are not allowed

tdaq-02-00-01

OKS Server

• the oks-commit.sh supports directories in addition to files

• add oks-import.sh utility to simplify import of new directories and files

• the repository locks remain from abnormally terminated oks commits can be removed on Point-1 by DAQ experts using /oks/admin/unlock-repository.sh script via sudo

• the "Replace" dialog of OKS Data editor proposes user to check-out repository file containing modified objects

• file-related relative pathnames and absolute pathnames includes are not allowed (in particular to avoid inclusion of files stored outside current repository and to simplify consistency check by oks-commit.sh)

Read more details on the [TWiki page](https://twiki.cern.ch/twiki/bin/view/Atlas/DaqHltOks#3_OKS_Server).

OKS Performance Improvements

• the OKS library uses pool of threads to load OKS data files, i.e. the data files can be read in parallel

• the number of threads by default is equal to number of the computer's CPU cores

• it can be modified via OKS_KERNEL_THREADS_POOL_SIZE environment variable

• the OKS library does not stop reading of files on first error, but continues loading of files in parallel threads until their ends or errors

• thus the final error report may contain several errors coming from different files

• this error report may change between different runs of OKS utilities even if the files were not updated

• for optimal performance it is recommended:

1. to reduce number of schema files (at some point after a schema file parsing OKS requires single active thread to update the database schema)

2. to avoid huge data files (processing of single data file is not parallelized since XML files are non indexed)

tdaq-02-00-00

C++ API Changes

Use new integer types from <stdint.h> to support 64-bits platform as shown in the following table:

| OKS Type | Old C++ Type | New C++ Type |

|----------|--------------:|--------------:|

| s8 (8-bits signed integer) | unsigned char | uint8_t |

| u8 (8-bits unsigned integer) | signed char | int8_t |

| s16 (16-bits signed integer) | unsigned short | uint16_t |

| u16 (16-bits unsigned integer) | signed short | int16_t |

| s32 (32-bits signed integer) | unsigned long | uint32_t |

| u32 (32-bits unsigned integer) | signed long | int32_t |

OKS Server

On Point-1 access to database repository will be controlled by the OKS Server. Read more about it on the [TWiki page](https://twiki.cern.ch/twiki/bin/view/Atlas/DaqHltOks#3_OKS_Server).

OKS Data Editor Changes

- Add search by file name in the main window
- Add search by class and object ID in the Data File dialog
- Add group file operations from the File menu:
- Save all updated files (<Ctrl-S> shortcut)
- OKS server related operations:
- Release User Repository (<Ctrl-D> shortcut)
- Update User Repository (<Ctrl-U> shortcut)
- Commit User Repository (<Ctrl-C> shortcut)
- Add "Referenced By" function from Object dialog (available on right mouse click from dialog's icon)

tdaq-01-09-00

OKS Library

- reload any consistent data file (also including changes in the included files)
- speed up OKS XML loading (about 25% faster comparing with release 1.8.4)
- when read XML schema file, throw exception if base class is not loaded
- oks query supports regular expressions (add attribute comparator '~=')

OKS Archiving Library

- use temporal tables to create "try" incremental data version (to reduce unnecessary overhead on Oracle stream replication as requested by ATLAS Oracle DBA)

OKS GUI Library

- add support for mouse wheel (can be used in most dialogs of OKS schema and data editors)

OKS Data Editor

- improvements in the Find/Replace dialog:
- optionally find by Class and Attribute/Relationship names
- present result as table
- select visible classes by name and objects by UID (request [34890](http://savannah.cern.ch/bugs/?34890))
- see search panel at bottom of main window and object dialogs
- the search panel supports simple string search (auto-select when modify the selection pattern) and regular expressions (press button appearing when this option selected to apply regular expression)
- improve performance when build class dialog containing big number of objects (can be seen in 1.8.4 when number of objects is greater than 10K)

tdaq-01-08-04

GUI Changes

OKS Data Editor Force Save

The users have possibility to save partly-inconsistent data using "Force Save" command. This is required to save partial work avoiding it's possible lost because of exterior problems. For more info see request [28879](https://savannah.cern.ch/bugs/index.php?28879)

OKS Editors Recovery Mode

The users have possibility to periodically save changes made with OKS Schema and Data editors. Such changes automatically go to ${FILE}.saved for any unsaved modifications. If user skips changes on exit or an editor stops the work unexpectedly, those ".saved" files remain and can be used for manual recovery. This option can be switched On/Off using an editor Options menu. From the same menu it is possible to set the period of such saving varying from 5" up to 1 hour. For more info see request [28879](https://savannah.cern.ch/bugs/index.php?28879)

Comments

The users have possibility to add comments to files. The comments can be browsed and edited from File dialog of the OKS Schema and Data editors. When file is saved, the editors can to ask user to provide comments, if "Ask comment on file save" option is activated in the "Options" menu (default option if user never saved GUI options). To add a comment user has to provide non-empty text. If no comment should be added on file save, press "Cancel" button in the Comment dialog.

XML Format Changes

The value of the "num-of-items" attribute of the oks schema info record is ignored. This was a reason of several wrong schema files after modifications made by users with text editors, when they forgot to set the correct value of this attribute.

API Changes

Note: the changes are completely transparent to users of config and DAL layers. Update your code only if you are using OKS directly!

OksObject Class Changes

For consistent error reporting and simplification of code the following methods have been changed.

| Old Method Spec | New Method Spec |

|---|---|

| OksReturnStatus GetAttributeValue(const std::string&, OksData **) const | OksData * GetAttributeValue(const std::string&) const throw (oks::exception) |

| void GetAttributeValue(const OksDataInfo *, OksData **) const | OksData * GetAttributeValue(const OksDataInfo *) const throw () |

| OksReturnStatus GetRelationshipValue(const std::string&, OksData **) const | OksData * GetRelationshipValue(const std::string&) const throw (oks::exception) |

| void GetRelationshipValue(const OksDataInfo *, OksData **) | OksData * GetRelationshipValue(const OksDataInfo *) const throw () |

| OksReturnStatus SetAttributeValue(const std::string&, OksData *) | void SetAttributeValue(const std::string&, OksData *) throw (oks::exception) |

| OksReturnStatus SetAttributeValue(const OksDataInfo *, OksData *) | void SetAttributeValue(const OksDataInfo *, OksData *) throw (oks::exception) |

| OksReturnStatus SetRelationshipValue(const std::string&, OksData *) | void SetRelationshipValue(const std::string&, OksData *) throw (oks::exception) |

| OksReturnStatus SetRelationshipValue(const OksDataInfo *, OksData *) | void SetRelationshipValue(const OksDataInfo *, OksData *) throw(oks::exception)

| OksReturnStatus SetRelationshipValue(const std::string&, OksObject *) | void SetRelationshipValue(const std::string&, OksObject *) throw (oks::exception) |

| OksReturnStatus SetRelationshipValue(const OksDataInfo *, OksObject *) | void SetRelationshipValue(const OksDataInfo *, OksObject *) throw (oks::exception) |

| OksReturnStatus SetRelationshipValue(const std::string&, const std::string&, const std::string&) | void SetRelationshipValue(const std::string&, const std::string&, const std::string&) throw (oks::exception) |

| OksReturnStatus AddRelationshipValue(const std::string&, OksObject *) | void AddRelationshipValue(const std::string&, OksObject *) throw (oks::exception) |

| OksReturnStatus AddRelationshipValue(const OksDataInfo *, OksObject *) | void AddRelationshipValue(const OksDataInfo *, OksObject *) throw (oks::exception) |

| OksReturnStatus AddRelationshipValue(const std::string&, const std::string&, const std::string&) | void AddRelationshipValue(const std::string&, const std::string&, const std::string&) throw (oks::exception) |

| OksReturnStatus RemoveRelationshipValue(const char *, OksObject *) | void RemoveRelationshipValue(const std::string&, OksObject *) throw (oks::exception) |

| OksReturnStatus RemoveRelationshipValue(const OksDataInfo *, OksObject *) | void RemoveRelationshipValue(const OksDataInfo *, OksObject *) throw (oks::exception) |

| OksReturnStatus RemoveRelationshipValue(const std::string&, const std::string&, const std::string&) | void RemoveRelationshipValue(const std::string&, const std::string&, const std::string&) throw (oks::exception) |

Also by needs of config Python bindings one new method have been added:

```
std::list<OksObject *> * get_all_rels(const std::string& name = "*") const
```

The method returns list of objects which have a reference on given one. If the relationship name is set to "*", then the method takes into account all relationships of all objects. The method performs full scan of all OKS objects and it is not recommended at large scale to build complete graph of relations between all database object; if only composite parents are needed, them the reverse_composite_rels() method has to be used.

By needs of tidb package there are two new methods to read OksObject from and to it put into standard streams:

```
static OksObject * get(std::istream&, OksKernel *) throw (oks::exception)
void put(std::ostream&) const throw (oks::exception)
```

OksClass Class Changes

By needs of tidb package there are two new methods to read OksClass from and to it put into standard streams:

```
static OksClass * get(std::istream&, OksKernel *) throw (oks::exception)
void put(std::ostream&) const throw (oks::exception)
```

OksFile Class Changes

To improve error reporting the following methods throw exception instead of returning bad error code:

| Old Method Spec | New Method Spec |

|-----------------|-----------------|

| Oks::ReturnStatus lock(bool = false) | void lock(bool force = false) throw (oks::exception) |

| Oks::ReturnStatus unlock() | void unlock() throw (oks::exception) |

| Oks::ReturnStatus set_logical_name(const std::string &) | void set_logical_name(const std::string& name) throw (oks::exception) |

| Oks::ReturnStatus set_type(const std::string &) | void set_type(const std::string& type) throw (oks::exception) |

OksKernel Class Changes

| Old Method Spec | New Method Spec |

|----------|----------|

| OksReturnStatus set_active_schema(OksFile *) | void set_active_schema(OksFile *) throw (oks::exception) |

| OksReturnStatus set_active_data(OksFile *) | void set_active_data(OksFile *) throw (oks::exception) |

| bool GetAllowDuplicatedObjectsMode() const | bool get_allow_duplicated_objects_mode() const |

| void SetAllowDuplicatedObjectsMode(const bool) | void set_allow_duplicated_objects_mode(const bool) |

| bool GetVerboseMode() cons | bool get_verbose_mode() const |

| void SetVerboseMode(const bool) | void set_verbose_mode(const bool) |

| bool GetSilenceMode() const | bool get_silence_mode() const |

| void SetSilenceMode(const bool) | void set_silence_mode(const bool) |

| bool GetProfilingMode() const | ool get_profiling_mode() const |

| void SetProfilingMode(const bool) | void set_profiling_mode(const bool) |

The OKS kernel provides new methods to check status and to change various kernel modes:

- the status of mode checking maximum length of string attributes of some OKS objects (see also 1.8.3 OKS release notes)

```
  static bool get_skip_max_length_check_mode()
  static void set_skip_max_length_check_mode(const bool)
  ```
  it can also be set using OKS_SKIP_MAX_LENGTH_CHECK environment variable.
* the status of the mode testing inherited duplicated objects:
  ```
  bool get_test_duplicated_objects_via_inheritance_mode() const
  void set_test_duplicated_objects_via_inheritance_mode(const bool)
  ```

 There are new methods to backup schema and data files (the operation is silent and ignores any consistency rules):
```

void backup_data(OksFile * pf, const char * suffix = ".bak") throw (oks::exception)

void backup_schema(OksFile * pf, const char * suffix = ".bak") throw (oks::exception)

```
 There are new methods to create OksClass and OksObject objects from standard streams:
```

OksObject * create_object(std::istream& input) throw (oks::exception)

OksClass * create_class(std::istream& input) throw (oks::exception)

```
 ## tdaq-01-08-03

 ### General Changes

 #### Max Length for OKS Names

 By needs of OKS archiving, limit maximum string length for attributes of some OKS types, which are:

 | OKS Type        | Attribute   | Maximum Length |
 |-----------------|-------------|----------------|
 | OksObject       | Object ID   | 64             |
```
```

```
| OksClass        | Name        | 64            |
| ^^              | Description | 2000          |
| OksAttribute    | Name        | 128           |
| ^^              | Description | 2000          |
| ^^              | Range       | 1024          |
| OksRelationship | Name        | 128           |
| ^^              | Description | 2000          |
| OksMethod       | Name        | 128           |
| ^^              | Description | 2000          |


#### New Oks Data Types

Add new OKS Data types:

* **s64_int_type** - signed 64-bits integer ("s64"); for implementation uses typed on the **int64_t** type
* **u64_int_type** - unsigned 64-bits integer ("u64"); for implementation uses typed on the **uint64_t** type
* **class_type** - reference on class; is implemented as string with range of allowed values equal to names of classes defined by the

Above types are fully supported by oks xml files, GUI editors and archiving.

If there are already existing and used OKS relational archives, check oks/src/rlib/create_db.[oracle|mysql|sqlite].sql bootstrap files

#### Bug Fixes

Use maximum compiler-supported precision when store or print out values of OKS **float** and **double** numeric types. Before the prec

### OKS GUI Changes

#### New Features

The Data and Schema editors store options of graphical windows in the ~/.oks-data-editor-rc.xml and  ~/.oks-schema-editor-rc.xml files

#### Bug Fixes and Known Problems of OKS Data Editor

Fix bug in a graphical view, when several objects were over-drawn on the same place.

OKS Data Editor cannot display too many graphical objects in a graphical view. The maximum allowed number of objects is limited by cap

## tdaq-01-08-00

### OKS Archiving

Add newly appeared classes (e.g. after integration with new detector) into schema already existing in archive (before it was required
* smaller number of versions
* reduce archive tool's downtime (human intervention is only needed when the database schema is modified, but not when it is extended

Add newly appeared objects into base data version (before such objects were created in the incremental version, that takes more space
* assume that any configuration object is referenced somehow (implicitly) by the partition object
* agreed with TDAQ groups and detectors; required schema changes in areas where string values were used for references instead of rela

As result, the utility to create new base or schema version in archive becomes much more robust since it does not require to put all c

### OKS Data Editor: Graphical Window

* fix several bugs when work with icons of small size
* add possibility to arrange objects of a relationship by one object per single line, that makes OKS graphical window look like more

### OKS Library

* most kernel functions throw exceptions instead of printing error messages; this allows better integration with oksconfig plug-in
* improve performance of OKS methods reading and saving database by caching names of tested files and directories


## tdaq-01-07-00

### API Changes

OKS library was starting to use exceptions to report problems. The methods dealing with input / output operations (i.e. _load...()_, _

The old code testing return status:
```

OksKernel kernel;

OksFile * fh1 = kernel.**load_file**("_test.in.xml_"); // (1) can return zero in case of error!

if(fh1 == 0) { std::cerr << "_ERROR: Can not load file \"test.in.xml\"\n_"; exit(1); }

OksFile * fh2 = kernel.**new_data**("_test.out.xml_"); // (2) can return zero in case of error!

if(fh2 == 0) { std::cerr << "_ERROR: Can not create file \"test.out.xml\"\n_"; exit(1); }

... // (3) some code modifying oks data

if(kernel.save_schema(fh2) != OksSuccess) { // (4) need to check OksStatus

std::cerr << "_ERROR: Can not save file \"test.out.xml\"\n_"; exit(1);

}

```
 has to be replaced with the following one:
```

OksKernel kernel;

try {

OksFile * fh1 = kernel.**load_file**("_test.in.xml_"); // (5) always returns non-zero!

OksFile * fh2 = kernel.**new_data**("_test.out.xml_"); // (6) always returns non-zero!

... // (7) some code modifying oks data

kernel.save_schema(fh2); // (8) is void

}

catch (oks::exception & ex) { std::cerr << "Caught OKS exception: " << ex << std::endl; exit(1); }

```
 Using exceptions there is no more need to test return values:

 * a returned pointer is always non-zero (compare lines 1, 2 with 5, 6)
 * _save...()_ methods become _void_ instead of returning _OksStatus_ (compare line 4 with 8)

 ### Improved Reporting of XML Problems

 Another advantage of exception usage is consistent error reporting, that is especially important in case of multiple include files. In
```

lxplus055:db6$ oks_dump -f /tmp/daq/partitions/be_test.data.xml

Reading data file "/tmp/daq/partitions/be_test.data.xml" in extended format (3331 bytes)...

- reading data file "/tmp/daq/segments/segments.data.xml" in extended format (10503 bytes)...
- loading 53 classes from file "/tmp/dal/schema/core.schema.xml"...
- reading data file "/tmp/DAQRelease/sw/repository.data.xml" in extended format (88245 bytes)...
- reading data file "/tmp/DAQRelease/sw/external.data.xml" in extended format (15272 bytes)...
- reading data file "/tmp/DAQRelease/sw/tags.data.xml" in extended format (1928 bytes)...

ERROR [OksXmlInputStream::read_tag_start()]:

(line 67, char 1)

Unexpected end of file

ERROR [OksXmlInputStream::read_tag_start()]:

(line 67, char 1)

Unexpected end of file

ERROR [OksObject::read_header()]:

(line 67, char 1)

Failed read start-of-object 'obj' tag

```
 When above problem took place using oksconfig plug-in, it was not possible to identify the exact place of problem at all (at least wi
```

```
bash$ config_dump -d oksconfig:daq/partitions/be_test.data.xml -c Partition
ERROR [OksXmlInputStream::read_tag_start()]:
(line 67, char 2)
Unexpected end of file
ERROR [OksXmlInputStream::read_tag_start()]:
(line 67, char 2)
Unexpected end of file
 Now the resulted exception always contains exact reason of error and keeps full chain of files inclusion and oks entities dependencies
bash$ export OKS_KERNEL_SILENCE=yes
bash$ oks_dump -f /tmpdaq/partitions/be_test.data.xml
Caught oks exception:
oks[10] ***: failed to load data file "/tmp/daq/partitions/be_test.data.xml" because:
oks[9] ***: failed to load include "daq/segments/segments.data.xml" because:
oks[8] ***: failed to load data file "/tmp/daq/segments/segments.data.xml" because:
oks[7] ***: failed to load include "DAQRelease/sw/repository.data.xml" because:
oks[6] ***: failed to load data file "/tmp/DAQRelease/sw/repository.data.xml" because:
oks[5] ***: failed to load include "DAQRelease/sw/external.data.xml" because:
oks[4] ***: failed to load data file "/tmp/DAQRelease/sw/external.data.xml" because:
oks[3] ***: failed to load include "DAQRelease/sw/tags.data.xml" because:
oks[2] ***: failed to load data file "/tmp/DAQRelease/sw/tags.data.xml" because:
oks[1] ***: failed to read 'object "i686-slc3-gcc344-dbg@Tag"'
oks[0] ***: Unexpected end of file while read tag start at (line 66, char 51)
 The same exception will be passed to the ERS exception reported by oksconfig plug-in:
bash$ config_dump -d oksconfig:/tmp/daq/partitions/be_test.data.xml -c Partition
ERROR 2007-Jan-17 11:20:12 [ConfigurationImpl* _oksconfig_creator_(...)
at oksconfig/src/OksConfiguration.cpp:29] oksconfig initialization error
was caused by: ERROR 2007-Jan-17 11:20:12 [virtual void OksConfiguration::open_db(...) at
oksconfig/src/OksConfiguration.cpp:57] cannot load file '/tmp/daq/partitions/be_test.data.xml':
oks[10] ***: failed to load data file "/tmp/daq/partitions/be_test.data.xml" because:
...
oks[2] ***: failed to load data file "/tmp/DAQRelease/sw/tags.data.xml" because:
oks[1] ***: failed to read 'object "i686-slc3-gcc344-dbg@Tag"'
oks[0] ***: Unexpected end of file while read tag start at (line 66, char 51)
 ### OKS Archiving

 Add _Release_ column to _OksSchema_ table to simplify choice of right schema by oks2coral and to allow user easier choice of archived
 * queries on archived configurations by time intervals, release, user, host and partition patterns;
 * sorting result by multiple columns;
```

```
 * selection which columns to be shown.

 The test Web GUI replaced previous one: [http://cern.ch/isolov/cgi-bin/oks-archive.pl](http://cern.ch/isolov/cgi-bin/oks-archive.pl)
 Provide bootstrap files for different RDBMS:
 * create_db.mysql.sql
 * create_db.oracle.sql (renamed old create_db.sql)
 * create_db.sqlite.sql

 Fix several run-time problem for MySQL CORAL plug-in.

 #### API changes

 * Add optional _release_ parameter to several functions. It is used to get HEAD schema and data version per TDAQ release. By default
 * Add function _get_max_schema_version()_ to know maximum schema version number to be used to choose free version number. Note, the e
 * Add function _get_time_host_user()_ to extract time, host and user values from attribute list. It is used by _roks_ library and by

 #### List Archives Utility

 Add several _new options_ to specify archives selection criteria:
```

usage: oks_ls_data

- c | --connect-string connect_string
- w | --working-schema schema_name

[-l | --list-releases]

[-s | --schema-version schema_version]

[-b | --base-data-only]

[-z | --show-size]

[-u | --show-usage]

[-d | --show-description]

[-t | --sorted-by parameters]

[-r | --release release_name]

[-e | --user user_name_pattern]

[-o | --host hostname_pattern]

[-p | --partition partition_name_pattern]

[-S | --archived-since timestamp]

[-T | --archived-till timestamp]

[-v | --verbose-level verbosity_level]

[-h | --help]

Options/Arguments:

- c connect_string database connection string
- w schema_name name of working schema
- l list releases
- s schema_version print out data of this particular version (0 = HEAD version)
- b print out base data versions only
- z print size of version (i.e. number of relational rows to store it)
- u show who, when, where and how used given version

- d show description
- t parameters sort output by several columns; the parameters may contain the following

items (where first symbol is for ascending and second for descending order):

v | V - sort by versions;

t | T - sort by time;

u | U - sort by user names;

h | H - sort by hostnames;

p | P - sort by partition names (i.e. by descriptions);

- r release_name show configuration for given release name
- e user_name_pattern show configuration for user names satisfying pattern (see syntax description below)
- o hostname_pattern show configuration for hostnames satisfying pattern (see syntax description below)
- p partition_pattern show configuration for partition names satisfying pattern (see syntax description below)
- S since_timestamp show configuration archived since given moment (see timestamp format description below)
- T till_timestamp show configuration archived before given moment (see timestamp format description below)
- v verbosity_level set verbose output level (0 - silent, 1 - normal, 2 - extended, 3 - debug, ...)
- h print this message

Description:

The utility prints out details of oks data versions archived in relational database.

The Version is shown as sv.dv[.bv], where:

- sv = schema version;
- dv = data version;
- bv = base version (optional, only appears for incremental data versions).

The Size (numbers of relational rows) is reported in form x:y:z, where the:

- x = number of new[/deleted/updated] objects;
- y = number of new[/deleted/updated] attribute values;
- z = number of new[/deleted/updated] relationship values.

The timestamps to be provided in ISO 8601 format: "YYYY-MM-DD HH:MM:SS".

The allowed wildcard characters used to select by user, host and partition names are:

% (i.e. percent symbol) - any string of zero or more characters;

_ (i.e. underscore symbol) - any single character.

```
#### Other Implementation Changes and Bug Fixes

* replace _OksAlloc_ class used for the memory usage optimisation by the Boost class _boost/pool/pool_alloc.hpp_; header file _oks/al
* when save a data file, keep the file flags
* fix run-time bug appeared on 64-bits architecture (wrong calculation of const string literal size); by chance it worked correctly o

##### OKS Schema Editor

* attach _Range_ and _Initial Value_ properties to the right side of the attribute window to allow see long strings;
* do not show _Non Null Value_ property for _boolean_ attribute.

##### OKS Data Editor
```

```
* mark file updated, when swap objects inside relationship value;
* avoid bug when copy object to existing id;
* fix bug when load query with attribute comparator.


## tdaq-01-06-00

There are no any changes in OKS, that require a user to modify or to convert a consistent database file. However the OKS becomes more

### OKS library

#### Files Consistency

* to improve diagnostic report included files, where '_no files inclusion path between referenced objects_' problem takes place
* allow duplicated objects for archiving purposes (use OKS_KERNEL_ALLOW_DUPLICATED_OBJECTS variable)
* report object id and attribute name when read data with wrong range
* check inclusion of required schema files before saving
* allow empty objects (i.e. without attribute and relationship values)

#### Schema Consistency

* report warning when attributes and relationships change between single-value/multi-value in case of redefinition in derived class o
* change exclusiveness scope of composite relations from object to relationship
  e.g. a module can be exclusively inserted to crate and to detector, but it cannot be inserted to several crates

#### XML Parser

* report correct line number and position for certain types of problems in oks xml
* fix minor memory leak in OKS xml parser
* fix several bugs with xml comments and end of comment
* skip any attributes defined in the oks-schema or oks-data tag (they can come from automatic xml generation tools)
* allow xml style-sheets and xmlns tags
* allow different encodings of xml

#### Relational Methods using RAL

* move from POOL RAL to CORAL and follow changes in CORAL API up to latest used version (CORAL 1.3.0)
* add table to keep used configurations
* normalize schema as it was recommended by CERN Oracle DBA
* store oks date and time as string
* increase length of class name (now it is limited by 64 bytes)
* use xml authentication (see file authentication.xml pointed by the CORAL_AUTH_PATH variable)

#### General Methods

* add method to get referenced objects
* path query: check goal at non-leave object in the path to allow paths with optional branches
* fix bug when TDAQ_DB_PATH is not defined
* do not print warning in silent mode (before few ones left by mistake)

### General OKS Utilities

#### New oks-generate-schema-docs.sh

The utility generates description of the schema files using xsl conversion of standard oks schema xml files. Such conversion is perfo
```

Usage: oks-generate-schema-docs.sh [--help] [--verbose] [--search-dir in-dir] [--search-pattern schema-file-pattern] --target-dir out-dir

Arguments/Options:

- v | --verbose verbose output
- h | --help print this message
- d | --search-dir in-dir directory to search schema files; current value = [/afs/cern.ch/atlas/project/tdaq/cmt/nightly/installed/share/data]
- p | --search-pattern p pattern for schema file names; current value = [*.xml]
- t | --target-dir out-dir directory where to put out files
- n | --page-name name provide name for generated index.html file; current value = [TDAQ Release Schema Files]

```
The example of generated schema description for DAQ/HLT-I nightly release is available on: [http://pcatd12.cern.ch/releases/nightly/i
```

```
The utility can be used by users of the DAQ/HLT-I release to generate descriptions of own schema files using --search-dir to point to

#### New oks-test-duplicated-objects.sh

The utility tests duplicated objects (i.e. objects with equal class names and IDs) stored in oks data files referenced by the TDAQ_DB

The utility has been created to find files, which are _bad_ from archiving point of view. In ideal case it should find no duplications

To filter out files which need to be ignored either put file name pattern(s) into _-s_ command line option, or install into in any sub

#### oks_merge

* merge data files (use _-o_ option) and schema files (use _-s_ option)

#### oks_diff_data

* add options to compare objects of one class or single object

#### oks_diff_schema

* allow data files as input (i.e. compare schemes used by data files)
* change values returned by binary to be able to report number of found differences:
    * 0 - there are no differences between two schema files
    * 253 - bad command line
    * 254 - cannot load database file(s)
    * 255 - loaded file has no any class
    * 1..252 - number of differences (is limited by the max possible value)


#### oks_dump

* return different status in case of different problems:
    * 0 - no problems found
    * 1 - ad command line parameter
    * 2 - bad oks file(s)
    * 3 - bad query passed via -q or -p options
    * 4 - cannot find class passed via -c option
    * 5 - loaded objects have dangling references
* new option _-i_ adds possibility to read files to be printed from input-file instead of command line to help with very long list of
* distinguish lists of schema and data files lists on user choice:
    * use option _-f_ to print list of all oks xml files (is used as before)
    * use new option _-s_ to print list oks schema files
   * use new option _-d_ to print list oks data files
* add option _-r_ to print out list of objects referenced by found objects (can only be used with query)

### Utilities for OKS Archiving

All utilities described below require two parameters:

* the database connection string
* the name of the relational database working schema

The values of above parameters are site specific. For development purposes at CERN they are:

| the connection string: | oracle://devdb10/tdaq_dev_backup |
| the working schema: | onlcool |

For other sites or different purposes different database servers and/or accounts should be used. To create new database (for new owne
```

bash$ sqlplus ${user}/${password}@${host} @$TDAQ_INST_PATH/../oks/src/rlib/create_db.sql

```
where _${user}_, _${password}_ and _${host}_ have site-specific values.

#### New oks-create-new-base-version.sh

The utility has to be used to create a new base version in the archive. It is necessary when the there are changes in the configurati

By default, the utility checks differences between _head_ schema version from archive and schemes found under TDAQ_DB_PATH. If there a
```

Usage: oks-create-new-base-version.sh -c connect_string -w schema_name [--help] [--verbose level] [--skip-files reg-exp*]

Arguments/Options:

- c | --connect-string connect_str database connection string
- w | --working-schema schema_name name of relational database working schema

- v | --verbose level switch on verbose output
- h | --help print this message
- s | --skip-files r1 ... list of regular expressions to ignore files

```
#### New oks_tag_data

The utility is created to set a unique string tag on any existing data in OKS archive. A data can be accessed by such human meaningfu
```

usage: oks_tag_data -c | --connect-string connect_string

- w | --working-schema schema_name
- t | --tag data_tag

[-e | --head-data-version]

[-s | --schema-version schema_version]

[-n | --data-version data_version]

[-v | --verbose-level verbosity_level]

[-h | --help]

Options/Arguments:

- c connect_string database connection string
- w schema_name name of working schema
- t data_tag unique tag
- e tag head data version (for head schema or defined by -s)
- s schema_version use data for given schema version (extra -n or -e is required)
- n data_version use given data version (extra -s is required)
- v verbosity_level set verbose output level (0 - silent, 1 - normal, 2 - extended, 3 - debug, ...)
- h print this message

```
#### New oks_ls_data

The utility is created to print out information about data in OKS archive.
```

usage: oks_ls_data

- c | --connect-string connect_string
- w | --working-schema schema_name

[-s | --schema-version schema_version]

[-b | --base-data-only]

[-z | --show-size]

[-u | --show-usage]

[-v | --verbose-level verbosity_level]

[-h | --help]

Options/Arguments:

- c connect_string database connection string
- w schema_name name of working schema

- s schema_version print out data of this particular version (0 = HEAD version)
- b print out base data versions only
- z print size of version (i.e. number of relational rows to store it)
- u show who, when, where and how used given version
- v verbosity_level set verbose output level (0 - silent, 1 - normal, 2 - extended, 3 - debug, ...)
- h print this message

```
The version is shown as sv.dv[.bv], where:
* **sv** - schema version;
* **dv** - data version;
* **bv** - base version (optional, only appears for incremental data versions).

For example:
* 1.12 - base version with schema-version = 1 and data-version = 12
* 1.34.12 - incremental version with data-version = 34 built on top of base version 1.12

The size is reported when -z option is used explicitly. To get the size it is necessary to execute additional queries and this may ta
* **obj-num** - number of objects rows;
* **attr-num** - number of attribute value rows;
* **rel-num** - number of relationship rows.

For example:
* 951:8271:1498 - the base version contains 951 objects; the objects have 8271 attribute values and 1498 relationships
* 1:1:0 - the incremental version has one object updated comparing with base version (an attribute of the object was modified)

The usage of archived versions is reported when -u option is used explicitly. It requires some additional queries and may take some t

#### oks_put_schema,  oks_put_data,  oks_get_schema and oks_get_data

* use xml authentication instead of explicit user name and password passed via command line
* try to re-use the same options between binaries
* to know exact options per binary run it with _--help_ option

### GUI Editors

#### Schema Editor

* meaningless "Many" cardinality is not supported for relationships
*  heck classes and files consistency during save operation

#### Data Editor

* fix bug, when create new object providing ID of already existing object
* do not exit, if there is an error with file saving (i.e. allow user to change file name or it's permissions)
* check objects and files consistency during save operation
* refresh correctly list of all files when reload a file that changed includes
* warn user about bad files during loading them (e.g. with missing includes); a user should to fix reported problem before any other


## tdaq-01-04-00

There are several changes in the relational OKS back-end:
* use bulk insert for values of oks attributes and relationships
* replace OksDataInt, OksDataNum, OksDataString and OksDataDate tables by single OksDataVal table with appropriate columns to store i
* environment variable OKS_RAL_ORDER_QUERY_RESULTS can be used as switch on/off "order by" statement of queries reading values of att
* to improve performance do not read description of class methods and their implementations, when get oks data only

Change interpretation of **s8** and **u8** oks data types from symbol type to 8 bits integer type:
* u8 and s8 oks data types are interpreted as integers by any output method; before printing out non-alphanumeric symbols as char res
* oks_data_editor allows to edit u8 and s8 types as octal, decimal and hexadecimal numbers
* oks_schema_editor allows to set format for s8 and u8 types

The OksData stream output operator uses '_format_' field to prints out s8, u8, s16, u16, s32 and u32 types. It is used by the oks_dum

Fix bug appeared with few window managers, when under certain conditions oks gui applications do not react on mouse button clicks.

## tdaq-01-02-00

### OKS Relational Backend

An exercise to use a relational database to store oks schema and data information instead of xml files has been done. New _roks_ libr

The following sequence of steps to create relational tables, put/get schema and put/get data should to work:
```

sqlplus $[user/$passwd@$server](mailto:user/password@server) $TDAQ_INST_PATH/../oks/src/rlib/create_db.sql

oks_put_schema -c "oracle://$server" -u $user -p $passwd -f oks-file.xml -t "v1" "first" -s 1

oks_get_schema -c "oracle://$server" -u $user -p $passwd -s "/tmp/v1.schema.xml" -e

oks_put_data -c "oracle://$server" -u $user -p $passwd -f oks-file.xml -v -l -a -t "v1.1" "first"

oks_get_data -c "oracle://$server" -u $user -p $passwd -t "v1.1" -f /tmp/v1.1.data.xml

For more information contact the oks package developer.

Path Query

Add support for path queries. Such query returns path between two objects by navigating via relationships in accordance with user-defi

API

Add _oks::QueryPath_ class to describe special type of query calculating path between two given objects. The constructor uses query as

query-path ::= '(**path-to** "destination-object" _query-path-expression_)'

query-path-expression ::= '(_query-path-type_ "rel-name" [, "rel-name"*] [_query-path-expression_])'

query-path-type ::= '**direct** | **nested'**

If the string cannot be parsed, the exception _oks::bad_query_syntax_ is thrown.

When an oks query path object is created, it can be used to search a path from given source object using the following method:

OksObject::List * OksObject::find_path(const oks::QueryPath& query) const;

If a path is found, non-empty list is returned.

Example of query string

The example of query is shown below:

(path-to "my-id@my-class" (direct "A" "B" (nested "N" (direct "X" "Y" "Z"))))

The destination object is "my-id@my-class". The search can be started from any object of any class. In our example the start object ha

Generic query extensions

The oks query expression was extended to use object ID as part of query expression. The used syntax is '(**object-id** "an-object-id"

The object ID expression is integrated to the oks data editor query constructor (choose "_Object ID_" radio button in an attribute exp

Example of query string

The example of query to search all objects of some class referencing via relationship "my-relationship" an object with id equal to "te

(all ("my-relationship" some (object-id "test" =)))

For example, find all applications including subclasses, which runs on host with id lxplus001.cern.ch:

(all ("RunsOn" some (object-id "lxplus001.cern.ch" =)))

Dangling references

Add '_bool OksKernel::get_bind_objects_status() const_' method, that returns status of last _OksKernel::bind_objects()_ method call. I

OKS improves reporting of the dangling references. In addition to the dangling reference itself the oks reports an object where unreso

WARNING [OksObject::bind()]:

Cannot find object "lxplus053.cern.ch@Computer"

WARNING [OksObject::bind_objects()]:

There are unresolved references from object "lxplus-3x3-23 ctrl@RunControlApplication"

OKS dump

The _oks_dump_ binary returns non-null status, if the loaded files have non-resolved references between objects.

```
It also supports path queries: use '_--path "object" "path-query"_' command line parameters, e.g. to find path between an application
```

bash$ oks_dump --path "onlsw_test_3x3_lxlpus@Partition" '(path-to "lxplus-3x3-21-ctrl@RunControlApplication" (direct "Segments" "OnlineInfrastructure" (nested "Segments" (direct "Applications" "IsControlledBy" "Resources"))))' daq/partitions/lxplus_tests.data.xml

Found 3 objects in the path "(path-to "lxplus-3x3-21-ctrl@RunControlApplication" (direct "Segments" "OnlineInfrastructure" (nested "Segments" (direct "Applications" "IsControlledBy" "Resources"))))" from object "onlsw_test_3x3_lxlpus@Partition":

Object "onlsw_test_3x3_lxlpus@Partition" ...

Object "lxplus-3x3-2@Segment" ...

Object "lxplus-3x3-21@Segment" ...

```

## tdaq-01-01-00

### *Database Consistency*

• No more identical objects allowed. In the past a warning message was printed out and an anonymous object was created, when identical object was read. Now the error message is printed out, the reading of the database file containing duplicated object is stopped and the bad status of the file is returned. The error message contains the object identity and both names of the files containing such objects.

• Non-existent attributes and relationships of objects are reported as warnings. Before the data stored in extended format were converted without any message in case of schema evolution.

• To avoid possible confusion of users with variables converters provided by the **dal** package, the syntax of environment variables description used by oks in filenames is changed. Now the valid syntax is _$(FOO)_. In previous releases it was _${FOO}_. Note, it is not recommended to use environment variables in includes, since it makes database dependent on user's setup. The recommended way is to define includes either relative to a database repository, or to the parent file.

### *Queries Creation and Destruction*

• To allow proper destruction of a query make all internal query-related objects allocated on heap. In the past a query constructed from string was not properly released and memory leak took place. Now all sub-query objects are created on heap and are properly released, when the query object is destroyed. All code using queries (the oks kernel code, tutorial, examples) has been changed.

### *OKS Dump Application*

• Add several command line options:

• --files-only - prints out list names of database files

• --class - dump given class (all objects of class or matching some query)

• --query - print objects matching query (can only be used with class)

### *Bugs Fixes*

• Avoid possible segmentation fault, when read an object without loaded class.

• When load a schema, do not set automatically default value for enumeration attribute, if it was empty. It caused such default value explicitly set, when the schema is saved from the editor and such behavior was not expected by users.
```

14. DAL — “An Introduction to OKS” (docs/README.md, verbatim)

The canonical user-level introduction to OKS, from `DUNE-DAQ/dal/docs/README.md`.

An Introduction to OKS

Overview

OKS (Object Kernel Support) is a suite of packages [originally

written](https://gitlab.cern.ch/atlas-tdaq-software/oks) for the ATLAS

data acquisition effort. Its features include:

- The ability to define object types in XML (known as OKS "classes"), off of which C++ and Python classes can automatically be generated
- Support for class Attributes, Relationships, and Methods. Attributes and Relationships are automatically generated; Methods allow developers to add behavior to classes
- The ability to create instances of classes (known as OKS "objects"), modify them, read them into an XML file serving as a database and retrieve them from the database

This document provides a taste of what OKS has to offer.

Getting Started

To get started working with the DUNE-repurposed OKS packages, you'll want to [set up a work area](https://dune-daq-sw.readthedocs.io/en/latest/packages/daq-buildtools/) and then clone and build this repo ("dal") in order to run the tutorial below. These packages include [dbe (the DataBase Editor GUI)](https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/), dal (Data Access Library, this repo), [oksutils](https://github.com/DUNE-DAQ/oksutils), [oksdalgen](https://github.com/DUNE-DAQ/oksdalgen) (contains code generation executable), [oks](https://github.com/DUNE-DAQ/oks) (core OKS functionality, not to be confused with the entire OKS suite), [conffwk](https://github.com/DUNE-DAQ/conffwk), [oksconflibs](https://github.com/DUNE-DAQ/oksconflibs) and [okssystem](https://github.com/DUNE-DAQ/okssystem). Some of these packages you may never need to worry about, others (such as the dbe GUI) may benefit from further development.

With the work area set up, build dal. After building it, it's time to run some tests to make sure things are in working order. These include:

- `test_configuration.py`: A test script in the conffwk package. Tests that you can create objects, save them to a database, read them back, and remove them from a database.
- `test_dal.py`: Also from the conffwk package. Test that you can change the values of objects, and get expected errors if you assign out-of-range values.
- `algorithm_tests.py`: A script from the dal package. Test that Python bindings to class Methods implemented in C++ work as expected.

If anything goes wrong during the tests, it will be self-evident.

A Look at OKS Databases: XML-Represented Classes and Objects

While ATLAS has various database implementations (Oracle-based, etc.), for the DUNE DAQ we only need their basic database format, which is an XML file on disk. There are generally two types of database file: the kind that defines

classes, which by convention have the `*.schema.xml` extension, and the kind that define instances of those classes (i.e. objects) and have `*.data.xml` extensions. The class files are known from ATLAS as "schema files" and the object files are known from ATLAS as "data files". A good way to get a feel for these files is to start with the tutorial schema, `./install/dal/share/schema/dal/tutorial.schema.xml` from the base of your work area.

Overview of `tutorial.schema.xml`

Let's start with a description of what `tutorial.schema.xml` contains before we even look at its contents. It describes via three classes needed for a (very) simple DAQ: `ReadoutApplication` for detector readout, `RCApplication` for the Run Control in charge of `ReadoutApplication` instances, and a third class, `Application`, of which they're both subclasses. Open the file, and *unless you're purely curious, scroll past the lengthy header which you'll never need to understand* until you see the following:

```
<class name="Application" description="A software executable" is-abstract="yes">
 <attribute name="Name" description="Name of the executable, including full path" type="string" init-value="Unknown" is-not-null="ye
</class>
```

(_n.b. if you want to use `emacs` in this environment you may need to run `spack unload cairo`; this hasn't apparently had any negative effects on OKS functionality_). The `is-abstract` qualifier means that you can't have an object which is concretely of type `Application`, you can only have objects of subclasses of `Application`. However, any class which is a subclass of `Application` will automatically contain a `Name` attribute, which here is intended to be the fully-qualified path of the executable in a running DAQ system.

Next, we see the class for readout:

```
<class name="ReadoutApplication" description="An executable which reads out subdetectors">
  <superclass name="Application"/>
  <attribute name="SubDetector" description="An enum to describe what type of subdetector it can read out" type="enum" range="PMT,Wir
 </class>
```

...where you can see that there's an OKS enumerated type, where here there are only two options, basically a photon detector or a TPC.

Then, there's the run control application:

```
<class name="RCApplication" description="An executable which allows users to control datataking">
  <superclass name="Application"/>
  <attribute name="Timeout" description="Seconds to wait before giving up on a transition" type="u16" range="1..3600" init-value="20"
  <relationship name="ApplicationsControlled" description="Applications RC is in charge of" class-type="Application" low-cc="one" hig
 </class>
```

And here, we have two items of interest:

- A `Timeout` Attribute representing the max number of seconds before giving up on a transition. Represented by an unsigned 2-byte integer, the max timeout is one hour, and defaults to 20 seconds.
- An `ApplicationsControlled` Relationship, which refers to anywhere from one object subclassed from `Application` to "many", which is OKS-speak for "basically unlimited".

OKS also provides tools which parse the XML and provide summaries of the contents of the database (XML file). `config_dump`, part of the conffwk package, is quite useful in this regard. Pass it `-h` to get a description of its abilities; if you just run `config_dump -d oksconflibs:./install/dal/share/schema/dal/tutorial.schema.xml` you'll get a summary of the classes used to defined the objects in the file. Running `config_dump -d oksconflibs:./install/dal/share/schema/dal/tutorial.schema.xml -C` will give you much more detail. For a schema as simple as the one we're showing here, this tool isn't super-useful, but it can be powerful when schemas get bigger and more complex.

Overview of `tutorial.data.xml`

In this section, we're going to _make_ a data file using the classes from `tutorial.schema.xml`. It's extremely simple, just run this script:

```
tutorial.py
```

...and it will produce `tutorial.data.xml`. We'll look at it in a moment, but two things to note first:

1. As you can see if you open up `tutorial.py`, a Python module is actually _generated_ off of `tutorial.schema.xml`. If we add Attributes, Relations, etc. to the classes, the Python code will automatically pick them up without any additional Python needing to be written.

1. `config_dump -d oksconflibs:tutorial.data.xml --list-objects --print-referenced-by` provides a nice summary of `tutorial.data.xml`'s contents

We can also see what `tutorial.py` created by opening up `tutorial.data.xml`. Again, please scroll past the extensive header. What we see is two types of readout application, one ID'd as `PhotonReadout` and the other ID'd as `TPCReadout`; these names, of course, are chosen to reflect the choice of the `SubDetector` enum. Then we also see an instance of `RCApplication` where the `ApplicationsControlled` relationship establishes that run control is in charge of the two readout applications:

```
<obj class="RCApplication" id="DummyRC">
 <attr name="Name" type="string" val="/full/pathname/of/RC/executable"/>
 <attr name="Timeout" type="u16" val="20"/>
 <rel name="ApplicationsControlled">
  <ref class="ReadoutApplication" id="PhotonReadout"/>
  <ref class="ReadoutApplication" id="TPCReadout"/>
 </rel>
</obj>
```

The run control timeout is set to its default of 20 seconds. Say we want to change this, and save the result. For such a small data file it would be easy to manually edit, but if you think of a full-blown DAQ system you'll want to automate a lot of things. Fortunately we can alter the value via Python. Go into an interactive Python environment and do the following:

```
import conffwk
db = conffwk.Configuration('oksconflibs:tutorial.data.xml')
rc = db.get_dal("RCApplication", "DummyRC")  # i.e., first argument is name of the class, the second is the name of the object
print(rc.Timeout)
```

where in the last command, you see the timeout of 20 seconds.

For fun, let's try to set the timeout to an illegal value (i.e., a timeout greater than an hour):

```
rc.Timeout = 7200 # 2 hrs before run control gives up!
```

...you'll get a `ValueError`.

Let's set it to a less-ridiculous 60 seconds, and save the result:

```
rc.Timeout = 60
db.update_dal(rc)
db.commit()
```

If we exit out of Python and look at `tutorial.data.xml`, we see the timeout is now 60 rather than 20. And if we read the data file back in, this update will be reflected.

You're encouraged to experiment yourself with the objects, either interactively or via checking out the dal repo and editing `sourcecode/dal/scripts/tutorial.py`; make sure to run `dbt-build` in the case of the latter.

A Realistic Example

The `tutorial.schema.xml` file and `tutorial.data.xml` files are fairly easy to understand, and meant to be for educational purposes. To see the actual classes which are used on ATLAS, we can look at the following from the base of your work area: `./install/dal/share/schema/dal/core.schema.xml`. This file is quite large, and describes classes which actually model ATLAS's DAQ systems like `ComputerProgram` and `Rack` and `Crate`. If you look in [dal's `CMakeLists.txt` file](https://github.com/DUNE-DAQ/dal/blob/develop/CMakeLists.txt) you see the following:

```
daq_oks_codegen(core.schema.xml)

daq_add_library(algorithms.cpp disabled-components.cpp test_circular_dependency.cpp LINK_LIBRARIES conffwk::conffwk okssystem::okssys
```

`core.schema.xml` gets fed into `daq_oks_codegen` which proceeds to generate code off of the classes defined in `core.schema.xml` that will subsequently be built into the package's main library. Details on `daq_oks_codegen` can be found [here](https://dune-daq-sw.readthedocs.io/en/latest/packages/daq-cmake/#daq_oks_codegen).

You'll notice also that the classes in `core.schema.xml` contain not only Attributes and Relationships as in the tutorial example above, but also Methods. If you look at the `Partition` class (l. 415) and scroll down a bit, you'll see a `get_all_applications` Method declared, along with its accompanying C++ declaration (as well as Java declaration, but we ignore this). The implementation of `get_all_applications` needs to be done manually, however, and is accomplished on l. 1301 of `src/algorithms.cpp`. If you scroll to the top of that file you'll see a `#include "dal/Partition.hpp"` line. In the actual dal repo, there's no such include file. However, assuming you followed the build instructions at the top of this document, you'll find it in the `build/` area of your work area, as the header was in fact generated.

To see the `get_all_applications` function in action, you can do the

following.

```
dal_dump_app_config -d oksconflibs:./install/dal/bin/dal_testing.data.xml -p ToyPartition -s ToyOnlineSegment
```

...where `dal_testing.data.xml` is written specifically for testing dal's functionality. The output will look like the following:

```
Got 2 applications:
================================================================================================================================================
| num | Application Object                              | Host                      | Segment                         | Segment unique
================================================================================================================================================
|   1 | ToyRunControlApplication@RunControlApplication | toyhost.fnal.gov@Computer | ToyOnlineSegment@OnlineSegment | ToyOnlineSegmen
|   2 | SomeApp@CustomLifetimeApplication               | toyhost.fnal.gov@Computer | ToyOnlineSegment@OnlineSegment | ToyOnlineSegmen
================================================================================================================================================
```

Note that in the output, `Application Object`, `Host` and `Segment` are printed in the format `<object name>@<class name>` where the object is an instance of a class. Note also that `RunControlApplication` is an actual ATLAS class and not to be confused with the much simpler `RCApplication` from the tutorial above.

Likewise, you can see a Python script which serves the same function,

but via calling Python bindings to C++ functions. We of course want

the output to be identical:

```
dal_dump_app_config.py -d oksconflibs:./install/dal/bin/dal_testing.data.xml -p ToyPartition -s ToyOnlineSegment
```

You can also play around with `dal_dump_apps` or `dal_dump_apps.py`, pass the

`-h` argument to either program to see your options.

Next Step

Now that we've learned a bit about OKS, let's take a look at the [GUI interfaces to OKS](https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/)

15. DAL — Release Notes (Git-Integration Highlights)

JCF, Jan-23-2023: the following are the original ATLAS TDAQ release notes; they are not guaranteed to be applicable to the DUNE DAQ refactor of this repository

dal

tdaq-09-01-00

OKS git

When TDAQ_DB_REPOSITORY and TDAQ_DB_USER_REPOSTORY are defined, the DAL algorithm calculating environment sets TDAQ_DB_PATH=$TDAQ_DB_USER_REPOSTORY and usets TDAQ_DB_REPOSITORY and TDAQ_DB_USER_REPOSTORY process environment to allow standalone user tests.

SW repository generation utility was removed

Remove dal_create_sw_repository. The create_repo.py has to be used instead. See [CMake TWiki](https://twiki.cern.ch/twiki/bin/viewauth/Atlas/DaqHltCMake#Software_Repository_Generation) for more information.

tdaq-08-03-00

New template-based segments and applications config

Jira: [ADTCC-177](https://its.cern.ch/jira/browse/ADTCC-177)

The generated DAL classes BaseApplication and Segment replaced old AppConfig and SegConfig ones.

Updated partition algorithms:

```
std::vector<const BaseApplication *> Partition::get_all_applications(std::set<std::string> * app_types = nullptr,
  std::set<std::string> * use_segments = nullptr, std::set<const Computer *> * use_hosts = nullptr) const;

const Segment * Partition::get_segment(const std::string& name) const;
```

The segment algorithms:

```
std::vector<const BaseApplication *> Segment::get_all_applications(std::set<std::string> * app_types = nullptr,
  std::set<std::string> * use_segments = nullptr, std::set<const Computer *> * use_hosts = nullptr) const;

const BaseApplication * Segment::get_controller() const;

const std::vector<const BaseApplication *>& Segment::get_infrastructure() const;

const std::vector<const BaseApplication *>& Segment::get_applications() const;

const std::vector<const Segment*>& Segment::get_nested_segments() const;

const std::vector<const Computer*>& Segment::get_hosts() const;

const Segment * Segment::get_base_segment() const;

bool Segment::is_disabled() const;

bool Segment::is_templated() const;

void Segment::get_timeouts(int & actionTimeout, int & shortActionTimeout) const;
```

The application algorithms:

```
const Computer * BaseApplication::get_host() const;

const daq::core::Segment * BaseApplication::get_segment() const;

std::vector<const daq::core::Computer *> BaseApplication::get_backup_hosts() const;
```

```
const daq::core::BaseApplication * BaseApplication::get_base_app() const;

std::vector<const daq::core::BaseApplication *>
BaseApplication::get_initialization_depends_from(const std::vector<const daq::core::BaseApplication *>& all_apps) const;

std::vector<const daq::core::BaseApplication *>
BaseApplication::get_shutdown_depends_from(const std::vector<const daq::core::BaseApplication *>& all_apps) const;
```

The algorithms on Segment and BaseApplication objects may only be called, if such objects in turn were instantiated by get_all_applications() or partition's get_segment() DAL algorithms.

Algorithms changes

Jira: [ADTCC-209](https://its.cern.ch/jira/browse/ADTCC-209)

Several algorithms were never used and deleted or reorganized

Modified algorithms

- BaseApplication::get_output_error_directory() is renamed to Partition::get_log_directory() because the log directory does not depend on the application
- BaseApplication::get_info() remove partition, segment and host parameters
- Segment::get_timeouts() remove partition and database parameters
- SubstituteVariables remove database parameter in the constructor

Removed algorithms

Several algorithms were obsolete and deleted, or not used by user code and removed from public API:

- ComputerProgram::get_parameters()
- BaseApplication::get_application()
- BaseApplication::get_some_info()
- ResourceBase::get_applications()
- Variable::get_value(tag)

16. DAL — ConfigVersion Pattern (Headers + CLI Tools, verbatim)

include/dal/ConfigVersion.hpp

```
#ifndef CONFIGVERSION_H
#define CONFIGVERSION_H

#include <is/info.h>

#include <string>
#include <ostream>


// <<BeginUserCode>>

// <<EndUserCode>>
/**
 * The class is used to store GIT version of the OKS database used for given partition.
 *
 * @author  produced by the IS generator
 */

class ConfigVersion : public ISInfo {
public:

    /**
     * OKS GIT SHA key used for given partition
     */
    std::string                   Version;


    static const ISType & type() {
■static const ISType type_ = ConfigVersion( ).ISInfo::type();
■return type_;
    }

    std::ostream & print( std::ostream & out ) const {
■ISInfo::print( out );
■out << std::endl;
■out << "Version: " << Version << "\t// OKS GIT SHA key used for given partition";
■return out;
    }

    ConfigVersion( )
      : ISInfo( "ConfigVersion" )
    {
■initialize();
    }

    ~ConfigVersion(){

// <<BeginUserCode>>

// <<EndUserCode>>
    }

protected:
    ConfigVersion( const std::string & type )
      : ISInfo( type )
    {
■initialize();
    }

    void publishGuts( ISostream & out ){
■out << Version;
    }

    void refreshGuts( ISistream & in ){
■in >> Version;
    }

private:
    void initialize()
    {
```

```
// <<BeginUserCode>>

// <<EndUserCode>>
    }


// <<BeginUserCode>>

// <<EndUserCode>>
};

// <<BeginUserCode>>

// <<EndUserCode>>
inline std::ostream & operator<<( std::ostream & out, const ConfigVersion & info ) {
    info.print( out );
    return out;
}

#endif // CONFIGVERSION_H
```

apps/dal_get_config_version.cxx

```
#include <boost/program_options.hpp>

#include "dal/util.hpp"
#include "exit_status.hpp"

int
main(int argc, char **argv)
{

  std::string partition;

  boost::program_options::options_description desc(
      "Get configuration version. The partition infrastructure has to be running. The program reads version from information service."
      "Available options are:");

  try
    {
      desc.add_options()
        ("partition,p", boost::program_options::value<std::string>(&partition)->required(), "name of partition")
        ("help,h", "Print help message");

      boost::program_options::variables_map vm;
      boost::program_options::store(boost::program_options::parse_command_line(argc, argv, desc), vm);

      if (vm.count("help"))
        {
          std::cout << desc << std::endl;
          return __SuccessExitStatus__;
        }

      boost::program_options::notify(vm);
    }
  catch (std::exception& ex)
    {
      std::cerr << "Command line parsing errors occurred:\n" << ex.what() << std::endl;
      return __CommandLineErrorExitStatus__;
    }

  try
    {
      std::string version = ::dunedaq::dal::get_config_version(partition);
      std::cout << version << std::endl;
    }
  catch (const dunedaq::conffwk::NotFound & ex)
    {
      std::cerr << ex << std::endl;
      auto params = ex.parameters();
      return (params["type"] == "is value" ? __InfoNotFoundExitStatus__ : __RepositoryNotFoundExitStatus__);
    }
  catch (const dunedaq::conffwk::Exception & ex)
    {
      std::cerr << "ERROR: " << ex << std::endl;
      return __FailureExitStatus__;
    }
```

```
  return __SuccessExitStatus__;
}
```

apps/dal_set_config_version.cxx

```
#include <boost/program_options.hpp>

#include "dal/util.hpp"
#include "exit_status.hpp"

int
main(int argc, char **argv)
{

  std::string partition;
  std::string version;
  bool reload;

  boost::program_options::options_description desc(
      "Set configuration version. The partition infrastructure has to be running. The program publishes new version in information se
      "Available options are:");

  try
    {
      desc.add_options()
        ("version,v", boost::program_options::value<std::string>(&version)->required(), "set configuration version or erase if empty"
        ("partition,p", boost::program_options::value<std::string>(&partition)->required(), "name of partition")
        ("reload,r", "reload database service")
        ("help,h", "Print help message");

      boost::program_options::variables_map vm;
      boost::program_options::store(boost::program_options::parse_command_line(argc, argv, desc), vm);

      if (vm.count("help"))
        {
          std::cout << desc << std::endl;
          return __SuccessExitStatus__;
        }

      reload = vm.count("reload");

      boost::program_options::notify(vm);
    }
  catch (std::exception& ex)
    {
      std::cerr << "Command line parsing errors occurred:\n" << ex.what() << std::endl;
      return __CommandLineErrorExitStatus__;
    }

  try
    {
      ::dunedaq::dal::set_config_version(partition, version, reload);
    }
  catch (const dunedaq::conffwk::NotFound & ex)
    {
      std::cerr << ex << std::endl;
      return __RepositoryNotFoundExitStatus__;
    }
  catch (const dunedaq::conffwk::Exception & ex)
    {
      std::cerr << "ERROR: " << ex << std::endl;
      return __FailureExitStatus__;
    }

  return __SuccessExitStatus__;
}
```

17. DAL — Tutorial Schema, Data File, and tutorial.py (verbatim)

schema/dal/tutorial.schema.xml

```
<?xml version="1.0" encoding="ASCII"?>

<!-- oks-schema version 2.2 -->


<!DOCTYPE oks-schema [
  <!ELEMENT oks-schema (info, (include)?, (comments)?, (class)+)>
  <!ELEMENT info EMPTY>
  <!ATTLIST info
      name CDATA #IMPLIED
      type CDATA #IMPLIED
      num-of-items CDATA #REQUIRED
      oks-format CDATA #FIXED "schema"
      oks-version CDATA #REQUIRED
      created-by CDATA #IMPLIED
      created-on CDATA #IMPLIED
      creation-time CDATA #IMPLIED
      last-modified-by CDATA #IMPLIED
      last-modified-on CDATA #IMPLIED
      last-modification-time CDATA #IMPLIED
  >
  <!ELEMENT include (file)+>
  <!ELEMENT file EMPTY>
  <!ATTLIST file
      path CDATA #REQUIRED
  >
  <!ELEMENT comments (comment)+>
  <!ELEMENT comment EMPTY>
  <!ATTLIST comment
      creation-time CDATA #REQUIRED
      created-by CDATA #REQUIRED
      created-on CDATA #REQUIRED
      author CDATA #REQUIRED
      text CDATA #REQUIRED
  >
  <!ELEMENT class (superclass | attribute | relationship | method)*>
  <!ATTLIST class
      name CDATA #REQUIRED
      description CDATA ""
      is-abstract (yes|no) "no"
  >
  <!ELEMENT superclass EMPTY>
  <!ATTLIST superclass name CDATA #REQUIRED>
  <!ELEMENT attribute EMPTY>
  <!ATTLIST attribute
      name CDATA #REQUIRED
      description CDATA ""
      type (bool|s8|u8|s16|u16|s32|u32|s64|u64|float|double|date|time|string|uid|enum|class) #REQUIRED
      range CDATA ""
      format (dec|hex|oct) "dec"
      is-multi-value (yes|no) "no"
      init-value CDATA ""
      is-not-null (yes|no) "no"
      ordered (yes|no) "no"
  >
  <!ELEMENT relationship EMPTY>
  <!ATTLIST relationship
      name CDATA #REQUIRED
      description CDATA ""
      class-type CDATA #REQUIRED
      low-cc (zero|one) #REQUIRED
      high-cc (one|many) #REQUIRED
      is-composite (yes|no) #REQUIRED
      is-exclusive (yes|no) #REQUIRED
      is-dependent (yes|no) #REQUIRED
      ordered (yes|no) "no"
  >
  <!ELEMENT method (method-implementation*)>
  <!ATTLIST method
      name CDATA #REQUIRED
```

```
      description CDATA ""
  >
  <!ELEMENT method-implementation EMPTY>
  <!ATTLIST method-implementation
      language CDATA #REQUIRED
      prototype CDATA #REQUIRED
      body CDATA ""
  >
]>

<oks-schema>

<info name="" type="" num-of-items="3" oks-format="schema" oks-version="862f2957270" created-by="jcfree" created-on="mu2edaq13.fnal.g

 <class name="Application" description="A software executable" is-abstract="yes">
  <attribute name="Name" description="Name of the executable, including full path" type="string" init-value="Unknown" is-not-null="ye
 </class>

 <class name="ReadoutApplication" description="An executable which reads out subdetectors">
  <superclass name="Application"/>
  <attribute name="SubDetector" description="An enum to describe what type of subdetector it can read out" type="enum" range="PMT,Wir
 </class>

 <class name="RCApplication" description="An executable which allows users to control datataking">
  <superclass name="Application"/>
  <attribute name="Timeout" description="Seconds to wait before giving up on a transition" type="u16" range="1..3600" init-value="20"
  <relationship name="ApplicationsControlled" description="Applications RC is in charge of" class-type="Application" low-cc="one" hig
 </class>

</oks-schema>
```

scripts/dal_testing.data.xml

```
<?xml version="1.0" encoding="ASCII"?>

<!-- oks-data version 2.2 -->


<!DOCTYPE oks-data [
  <!ELEMENT oks-data (info, (include)?, (comments)?, (obj)+)>
  <!ELEMENT info EMPTY>
  <!ATTLIST info
      name CDATA #IMPLIED
      type CDATA #IMPLIED
      num-of-items CDATA #REQUIRED
      oks-format CDATA #FIXED "data"
      oks-version CDATA #REQUIRED
      created-by CDATA #IMPLIED
      created-on CDATA #IMPLIED
      creation-time CDATA #IMPLIED
      last-modified-by CDATA #IMPLIED
      last-modified-on CDATA #IMPLIED
      last-modification-time CDATA #IMPLIED
  >
  <!ELEMENT include (file)*>
  <!ELEMENT file EMPTY>
  <!ATTLIST file
      path CDATA #REQUIRED
  >
  <!ELEMENT comments (comment)*>
  <!ELEMENT comment EMPTY>
  <!ATTLIST comment
      creation-time CDATA #REQUIRED
      created-by CDATA #REQUIRED
      created-on CDATA #REQUIRED
      author CDATA #REQUIRED
      text CDATA #REQUIRED
  >
  <!ELEMENT obj (attr | rel)*>
  <!ATTLIST obj
      class CDATA #REQUIRED
      id CDATA #REQUIRED
  >
  <!ELEMENT attr (data)*>
  <!ATTLIST attr
      name CDATA #REQUIRED
      type (bool|s8|u8|s16|u16|s32|u32|s64|u64|float|double|date|time|string|uid|enum|class|-) "-"
      val CDATA ""
```

```
  >
  <!ELEMENT data EMPTY>
  <!ATTLIST data
      val CDATA #REQUIRED
  >
  <!ELEMENT rel (ref)*>
  <!ATTLIST rel
      name CDATA #REQUIRED
      class CDATA ""
      id CDATA ""
  >
  <!ELEMENT ref EMPTY>
  <!ATTLIST ref
      class CDATA #REQUIRED
      id CDATA #REQUIRED
  >
]>

<oks-data>

<!-- JCF, Jan-18-2023: the entire point of this file is that it defines objects which can be used for the dal repo's algorithm_tests.

<info name="" type="" num-of-items="2" oks-format="data" oks-version="N/A" created-by="jcfree" created-on="mu2edaq13.fnal.gov" creatio

<include>
 <file path="../share/schema/dal/core.schema.xml"/>
</include>

<obj class="Partition" id="ToyPartition">

  <rel name="OnlineInfrastructure" class="OnlineSegment" id="ToyOnlineSegment"/>
  <rel name="DefaultHost" class="Computer" id="toyhost.fnal.gov"/>

  <rel name="Disabled">
    <ref class="ResourceBase" id="AProblematicResource">
  </rel>
</obj>

<obj class="ResourceBase" id="AProblematicResource">
</obj>


<obj class="RunControlApplication" id="ToyRunControlApplication">
 <attr name="InterfaceName" type="string" val="rc/commander"/>
 <attr name="ActionTimeout" type="s32" val="10"/>
 <attr name="ProbeInterval" type="s32" val="5"/>
 <attr name="FullStatisticsInterval" type="s32" val="60"/>
 <attr name="ControlsTTCPartitions" type="bool" val="0"/>
 <attr name="Logging" type="bool" val="1"/>
 <attr name="InitTimeout" type="u32" val="60"/>
 <attr name="ExitTimeout" type="u32" val="5"/>
 <attr name="RestartableDuringRun" type="bool" val="0"/>
 <attr name="IfExitsUnexpectedly" type="enum" val="Error"/>
 <attr name="IfFailsToStart" type="enum" val="Error"/>
 <rel name="Program" class="Binary" id="rc_controller"/>
 <rel name="ExplicitTag" class="Tag" id="ToyTag"/>
</obj>

<obj class="SW_Repository" id="ToyRepository">
  <rel name="Tags">
   <ref class="Tag" id="ToyTag"/>
  </rel>
</obj>

<obj class="Binary" id="rc_controller">
 <attr name="BinaryName" type="string" val="rc_controller"/>
 <attr name="Description" type="string" val="A Controller implementing all the default actions"/>
 <attr name="HelpURL" type="string" val=""/>
 <attr name="DefaultParameters" type="string" val="foo bar"/>
 <rel name="BelongsTo" class="SW_Repository" id="ToyRepository"/>
</obj>

<obj class="Binary" id="coolapp">
 <attr name="BinaryName" type="string" val="coolapp"/>
 <attr name="Description" type="string" val="Just a straight-up amazing application"/>
 <attr name="HelpURL" type="string" val=""/>
 <attr name="DefaultParameters" type="string" val="arg1 arg2 arg3"/>
 <rel name="BelongsTo" class="SW_Repository" id="ToyRepository"/>
```

```
</obj>


<obj class="CustomLifetimeApplication" id="SomeApp">
 <attr name="Lifetime" type="enum" val="UserDefined_Shutdown"/>
 <attr name="AllowSpontaneousExit" type="bool" val="0"/>
 <rel name="Program" class="Binary" id="coolapp"/>
 <rel name="ExplicitTag" class="Tag" id="ToyTag"/>
</obj>


<obj class="OnlineSegment" id="ToyOnlineSegment">
 <attr name="T0_ProjectTag" type="string" val="data_test"/>
 <rel name="IsControlledBy" class="RunControlApplication" id="ToyRunControlApplication"/>
 <rel name="InitialPartition" class="Partition" id="ToyPartition">
 <rel name="Applications">
   <ref class="CustomLifetimeApplication" id="SomeApp"/>
 </rel>
  <rel name="Segments">
    <ref class="Segment" id="ToyChildSegment"/>
  </rel>
</obj>

<obj class="Segment" id="ToyChildSegment"/>
 <rel name="IsControlledBy" class="RunControlApplication" id="ToyRunControlApplication"/>
</obj>

<obj class="Tag" id="ToyTag">
 <attr name="HW_Tag" type="enum" val="x86_64-centos7"/>
 <attr name="SW_Tag" type="enum" val="gcc49-opt"/>
</obj>

<obj class="Computer" id="toyhost.fnal.gov">
 <attr name="HW_Tag" type="enum" val="x86_64-centos7"/>
 <attr name="SW_Tag" type="enum" val="gcc49-opt"/>
 <attr name="Type" type="string" val="Intel(R) Xeon(TM) CPU 3.40GHz"/>
 <attr name="Location" type="string" val=""/>
 <attr name="Description" type="string" val=""/>
 <attr name="HelpLink" type="string" val=""/>
 <attr name="InstallationRef" type="string" val=""/>
 <attr name="State" type="bool" val="1"/>
 <attr name="Memory" type="u16" val="514"/>
 <attr name="CPU" type="u16" val="3400"/>
 <attr name="NumberOfCores" type="s16" val="2"/>
 <attr name="RLogin" type="string" val="ssh"/>
</obj>

<!-- JCF, Jan-17-2023: this snippet is derived from /cvmfs/atlas.cern.ch/repo/sw/tdaq/tdaq/tdaq-09-05-00/TM/data/part_hlt.data.xml as
<obj class="Variable" id="PYTHONPATH">
 <attr name="Description" type="string" val=""/>
 <attr name="Name" type="string" val="PYTHONPATH"/>
 <attr name="Value" type="string" val="${LCG_INST_PATH}/LCG_95/Python/2.7.15"/>
 <rel name="TagValues">
  <ref class="TagMapping" id="ToyTagMapping"/>
 </rel>
</obj>

<obj class="TagMapping" id="ToyTagMapping">
 <attr name="HW_Tag" type="enum" val="x86_64-centos7"/>
 <attr name="SW_Tag" type="enum" val="gcc49-opt"/>
 <attr name="Value" type="string" val="12345"/>
</obj>


</oks-data>
```

scripts/tutorial.py

```
#!/bin/env python3

import conffwk
import os

schemafile=f'{os.environ["DAL_SHARE"]}/schema/dal/tutorial.schema.xml'
datafile="tutorial.data.xml"

# binds a new dal into the module named "tutorial"
tutorial = conffwk.dal.module('tutorial', schemafile)
```

```
db = conffwk.Configuration("oksconflibs")
db.create_db(datafile, [schemafile])

readout_app1 = tutorial.ReadoutApplication("PhotonReadout",
                                           Name="/full/pathname/of/readout/executable",
                                           SubDetector="PMT")

readout_app2 = tutorial.ReadoutApplication("TPCReadout",
                                           Name="/full/pathname/of/readout/executable",
                                           SubDetector="WireChamber")

runcontrol_app = tutorial.RCApplication("DummyRC",
                                                 Name="/full/pathname/of/RC/executable",
                                                 ApplicationsControlled=[readout_app1, readout_app2])

db.update_dal(readout_app1)
db.update_dal(readout_app2)
db.update_dal(runcontrol_app)
db.commit()

print(f"""
Please take a look at {datafile} for the data file this script
created using OKS classes from {schemafile}
""")
```

18. DBE (DataBase Editor) and Schema Editor — GUI User Guide

DBE (DataBase Editor) package

(URL: https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/)

The DBE package provides a GUI interface to the OKS suite, allowing you to edit both schema files and data files.

While DBE was [originally written as part of the ATLAS TDAQ effort](https://gitlab.cern.ch/atlas-tdaq-software/dbe.git), it has been modified and enhanced to build within the DUNE DAQ framework. To get started working with DBE, you'll need to run `spack load dbe`. This command is needed since dbe is not loaded by default when setting up the release since loading in `dbe` will cause emacs to no longer work in the terminal, an undesirable side effect developers don't want to have to experience whenever they set up a work area for reasons unrelated to dbe.

See the individual documents [dbe.md](https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/dbe/) and [schemaeditor.md](https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/schemaeditor/) for more information on running the editors.

The OKS database editor: `dbe_main`

(URL: https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/dbe/ — full text reproduced below)

Prerequisite

Before running either `dbe` or `schemaeditor` you must load the dbe spack package with `spack load dbe`. This can have unwanted side effects like running the wrong version of Python due to spack messing with your PATH and LD_LIBRARY_PATH. To avoid this, keep your editing sessions in a different window to your normal development or create an alias /shell function like:

```
function dbe_main ()
{
    bash -c "spack load dbe; command dbe_main $@"
}
```

Starting the editor

Just running the command `dbe_main` will bring up the database editor with no database loaded. You can then open an existing database or create a new database from the items on the File menu or the tool-bar. If you want to specify an existing database on the command line, you must include the `-f` option before the name of the file. File names not beginning with '/' are taken to be relative to first the current directory, then each member of the list in `DUNEDAQ_DB_PATH` until a match is found.

Structure of the editor

The main window below the tool-bar is initially split into 3 main parts, the `Class View` (1), the `Table View` (2) and the `Info Tabs`(3). The `Class View` and the `Info Tabs` can be undocked and moved out of the main window. Each of the views can also be enabled or disabled from the `View` menu.

Navigating with the `Class view` (1)

The `Class View` is a dock-able widget originally on the left of the main window. The `Class View` is a tree which displays a list of class names and the number of objects of that class that exist in the database. Where the number of objects of a class is non-zero, the item can be expanded to display the list of objects. Further, if an object has relationships to other objects, the object can be expanded to show the list of relationships which in turn can be expanded

to list the objects they are referencing.

- Activating a class name will display all instances of the class in the current `Table View`.
- Activating an instance in the `Tree view` will open the `Object Editor` to edit that instance.

Abstract classes

Abstract classes are usually shown in grey on the `Class View` and are not selectable. If you want to see objects of all the subclasses of an abstract class, you can check the `Enable abstract classes` check box. With this selected, the abstract classes become selectable and activating them will show all instances of of all subclasses in the `Table View`. **Beware** the column headings are taken from the base class and derived classes may have more attributes/resources or a different ordering.

Tooltips

Tooltips show the descriptions of the classes (assuming they have one).

Filtering the view

To filter the list of classes to see just a subset that you are interested in, there is a text entry field at the bottom of the `Class View` where you can enter a regular expression to be matched against the class names. The matching can be done on class name or object name and case sensitive or not according to the settings of the check box and combo box just above the text input.

For example to find all objects including 'ers' in their name, enter 'ers' in the search box and select `Search by Object` from the combo box above it.

Using the `Table view` (2)

The `Table View` area is a set of tabs which display the attributes and relationships of objects in the database. A new tab can be opened by selecting the '+' in the top left corner.

The content of a tab is selected from the `Class View` widget. Activating a class name will add all instances of that class to the current `Table View` tab. Activating individual instances will add only those selected to the tab. Instances can also be dragged from the `Class View` and dropped onto the `Table View`.

Attributes which are set to their default values are marked with a coloured background.

- Activating a row in the `Table View` will open the `Object Editor` on that instance.
- Double clicking on an attribute will allow editing of the attribute directly in the cell
- Double clicking on a relationship will pop up a dialogue box allowing selection of objects of the correct type.

Filtering the view

Like the `Class View`, there is an edit box for an object name filter to limit the display to only matching objects.

Context menu

The `Table view` context menu gives you several options, allowing you to find all objects that refer to the current object, find an object within the current view by name, copy edit or delete the current object.

The `Info Tabs` (3)

The `Info Tabs` section consists of 3 tabs, the `File View`, the `Undo` control and the `Commits log`.

The `File View` tab

The `File View` lists all the loaded data files along with their read/write access and modified status. The list of files included by the currently selected file can be updated by pulling up the include file editor by selecting `Add/Remove Files` from the context menu. A window showing information about the file, including a list of objects defined in it, can be activated by selecting `File information` from the context menu or by double-clicking on the appropriate row.

File Information window

The File Information window has 3 panels showing a list of all the objects in the file and lists of all the schema and data files that this file includes. The schema and data file panels have buttons allowing you to add further includes and this functionality is also available via the context menu. This is a simpler interface to updating the list of includes than using the `Include file editor`.

If the file contains any objects with relationships to objects in files that are not included or the schema file in which the class is defined is not loaded, a 4th panel with warning messages will be shown.

Both the schema file and data file panels have an 'Add' button allowing more includes to be added. These will bring up a standard file selection dialogue with the left-hand column populated by the paths from `DUNEDAQ_DB_PATH`.

The `Undo` tab

The `Undo` tab lists all the modifications that have been made since the last commit to the database. You can go back to any point in the history by selecting the line above the change you want to revert. Apart from navigating the Undo list with the mouse or keyboard, there are also buttons on the tool-bar to undo/redo changes.

Creating new objects

New objects can be created by from the `Class View` panel by using the context menu or the short cut `Ctrl-N` (also from the context menu in an active `Table View` tab). This brings up the `Object Editor` for the selected class. Before you can set the values of the attributes and relationships, you have to set the UID and select the file to store the object in.

New objects can also be created from the context menu in the `Table View`, either an empty one or a copy of an existing object.

Renaming objects or moving to a different file

To rename or move an object, bring up the object editor for that object and use the buttons in the top right-hand corner.

Finding what uses a specific object

To find all objects that refer to an object, first select the object in the table view. Then use one of the `Referenced By` items on the context menu. This will pop up a new window showing a class tree of all the objects that refer to it.

Changing values in multiple objects in one go (batch changes)

Occasionally, you may need to change the value of the same attribute across many objects. Hidden away on the `Edit` menu are two batch change items to do this. `Batch Change` and `Batch Change Table`

- `Batch Change` allows you to change attributes/relationships of all objects of a class with a UID matching an expression.
- `Batch Change Table` allows you to change attributes/relationships of objects in the current `Table View`.

Both menu items will pop up dialogue boxes with both attributes and relationship sections. You have to select the appropriate check-box to determine which you are changing.

`Batch Change Table` only allows you to set a new value for all items in the table.

`Batch Change` is more flexible in that it allows you to filter on the current values of attributes/relationships and apply new values for only matching objects. In the example screen-shot shown above, we changed the `request_handler` relationship for all `DataHandlerConf` objects whose `template_for` attribute started with 'FD'.

The OKS schema editor: `schemaeditor`

(URL: https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/schemaeditor/ — full text reproduced below)

Prerequisite

Before running either `schemaeditor` or `dbe` you must load the dbe spack package with `spack load dbe`. This can have unwanted side effects like running the wrong version of Python due to spack messing with your PATH and LD_LIBRARY_PATH. To avoid this, keep your editing sessions in a different window to your normal development or create an alias /shell function like:

```
function schemaeditor ()
{
    bash -c "spack load dbe; command schemaeditor $@"
}
```

Starting the editor

Just running the command `schemaeditor` will bring up the schemaeditor with no schema files loaded. You can then either open an existing schema file `File -> Open Schema` (or Ctrl+O) or create a new empty schema file with `File->Create new schema` (or Ctrl+N).

To start with an existing schema file, use the `-f` option e.g. `schemaeditor -f schema/appmodel/application.schema.xml`

Structure of the editor

The main window is initially split into 3 main parts, the `Class View` (1), the `SchemaView Tab area` (2) and the `Info Tabs` (3) with a menu bar and tool-bar at the top and a staus bar at the bottom. The `Class View` and the `Info Tabs` can be undocked and moved out of the main window. Each of the views can also be enabled or disabled from the `View` menu.

After adjusting the layout, the current layout can be saved at any point by selecting the `Save layout` option from the `Settings` menu. You can also use this menu to restore the default layout or the last layout you saved.

The Settings menu also has a `Preferences` option which will open a window where you can set fonts colours and default options for schema view diagram tabs.

Setting the 'Active' schema file

All new classes are created in the active schema file. This will initially be the file you loaded or created above unless the loaded file is read-only or locked by another process. To make another file the active schema file, use the context menu in the files tab (right-click).

_NB:_ You cannot add new classes unless you have made a schema file 'Active'.

Handling include files

Most schema files will want to include at least the core dunedaq schema (dunedaq.schema.xml) from confmodel. To add an include to the currently active schema file, either `File->Show schema file info`(I) or right click on the name of the schema file in the File panel. This will bring up a dialog box showing information about the selected or active schema file with a list of included files and the classes contained in this file. This dialog box has a button for adding include files and if any of the classes refer to classes that are not contained in any included file another button to automatically add all the required schema files. The `Add Include File` button brings up a standard open file dialog with all the elements of

`DUNEDAQ_DB_PATH` in the sidebar (use the tooltips to distinguish among the multiple `share` entries).

A list of which classes are in a given schema file can also be seen by opening a file info window from the file tab conext menu (or for the active file Ctl-I). This class list has a context menu allowing adding, editing and removing classes.

Saving files

To save all modified schema files, `File->Save Schema` (Ctrl+S). To save only a single file, use the context menu in the File tab. This also allows you to save files that have not been updated (sometimes useful to ensure proper formatting of files edited outside of the schemaeditor).

Editing classes

To edit a class, activate (double click or select and hit enter) the class name in the `classes` panel, double click on the class in the `class view` panel or select the edit option from the context menu in either panel. It is possible to open the class editor for a read only class from the `classes` panel but the editor will not allow you to make any changes.

Adding new classes

To add a new class Ctrl+A anywhere will bring up the new class dialogue box. From here you can define the attributes and relationships of your new class and set its superclass inheritance from the list of existing classes.

Selecting "Add New Class" from the context menu of the class list in a file info window will add a new class to that file, automatically switching the 'active' file.

New classes can also be added from the context menu in the schema view tabs. This will place the new class on the schema diagram at the current cursor position.

Moving classes between files

To move a class to a different file, open the class editor for the class and press the Move button near the top. Alternatively, from a file info window for the file that currently contains the class, select the class from the class list and activate the "Move Selected Class" item from the context menu. This will open a dialog box asking you to select the destination. Unfortunately, drag and drop between file info windows is not (yet) available.

Schema diagrams

To create a class diagram of the defined classes, simply drag the classes you are interested in from the 'Class Name' list onto a schema view tab. Relationships and inheritance connections will be automatically drawn. By default, only the direct properties of the classes are shown. To display all properties, including those inherited from super-classes, the context menu in the schema view panel includes an option to toggle viewing inherited properties. The context menu on an individual class within the view allows the addition of all its parent/child/related classes to the view. They will all appear in the top left corner and will need to be moved to appropriate places one by one.

Multiple views of the schema can be created by selecting the "+" button next to the view tabs. A tab can be renamed by selecting the 'Name View' button on the toolbar. Tabs can be closed by selecting the cross on the top corner of the tab or with the short cut Ctl-W.

Moving the diagram

To move the whole digram, move the mouse pointer to the point you want the current origin to move to and select `Move scene` from the context menu.

Highlighting classes

Classes may be highlighted in different colors/fonts. The colors and fonts can be set from the `Preferences` item on the `Settings` menu.

Highlighting classes from the active schema file

The classes contained in the current schema fie can be highlighted by selecting this option from the context menu. This can be useful to see at a glance which classes are in which file. Changing the active file from the `File info` tab will change which classes are highlighted accordingly.

Highlighting selected class

The class under the cursor can be highlighted by selecting the appropriate item from the context menu.

Tool-tips

Hovering the mouse over a class in the schema view will bring up a tool-tip with the description fields of the class and all its direct attributes, relationships and methods.

View files

The current schema diagram can be saved from the 'Save View' or 'Save View as' buttons on the toolbar or printed via the 'Print View' button. These options also exist in the SchemaViews menu along with an option to export to an SVG file.

Views can be loaded from files by using the Ctl-V short cut or selecting 'Load View' from the SchemaViews menu. Views load into new tabs unless the current tab is empty.

Notes

Notes can be added to the view by selecting 'Add note to view' from the context menu. Notes are currently written in very simple boxes with plain text. Line breaks should be added by hand.

Keyboard short cuts

| Key | Action | Notes |

| ----- | ---------- | ---------- |

| Ctl-A | Add new schema class | Only available when there is an 'Active' schema file |

| Ctl-N | Create New schema file | |

| Ctl-O | Open new schema file | |

| Ctl-I | Open file info dialog for active file | Only available when there is an 'Active' schema file |

| Ctl-S | Save modified schema files | |

| Ctl-V | Load View | |

| Ctl-K | Save View | |

| Ctl-E | Export View as SVG | |

| Ctl-P | Print View | |

| Ctl-W | Close View tab | |

| Ctl-Q | Quit | |

19. daqconf — Visualization, Manipulation and Generation Tools

Configuration creation, visualization and manipulation (daqconf)

(URL: https://dune-daq-sw.readthedocs.io/en/latest/packages/daqconf/ — full text reproduced below)

This repository contains scripts for generating and manipulating OKS database files.

Visualization tools

`daqconf_inspector`

Commandline utility to visually inspect and verify configurations databases and the objects they contain. Documenation available here: packages/daqconf/Inspector/.

`create_config_plot`

Commandline utility to generate a graphical flow diagram of a full configuration session or one of its applications or segments. Documentation available here: packages/daqconf/ConfigPlotting/.

Manipulation Tools

`oks_enable`

Add Resource objects to or remove from the `disabled` relationship of a Session

`consolidate`

Merge the contents of several database files, putting all objects into a single output file. The output file's include list will contain the schema files included by the source databases (or their includes), but will not contain any object databases (the schema themselves).

`consolidate_files`

Merge the contents of several database files, preserving included databases. Output file will contain only objects defined in files given on command line. The output files' include list will contain the schema files included by the source databases (or their includes), but will not contain any object databases (the schema themselves).

`copy_configuration`

Copy the input file(s) to the specified directory, also moving any included files and updating include paths, to create a clone of the configuration databases.

`get_apps`

Retrieve the DAQ applications defined in the given configuration

`oks-format`

Ensure that database files are in the "DBE format", alphabetized and with correct spacing

`oks_enable_tpg`

Enable or disable TPG for a Session's ReadoutApplications

`validate`

Attempt to determine if a given Session configuration is valid and does not contain common errors

`uniqueness_enforcer`

Attempt to enforce all object names to be unique in a configuration/folder of configurations.

textual_dbe

Attempt to replicate OKS' Data Base editor within Python. Full details are here: packages/daqconf/TextualDBE/. Current implementation is very incomplete so use with caution.

Generation Tools

`createOKSdb`

A script that generates an 'empty' OKS database, just containging the include files for the core schema and any other schema/data files you specify on the commad line.

`dromap2oks`

Convert a JSON readout map file from dunedaq v4 to an OKS file.

`generate_readoutOKS`

Create an OKS configuration file defining ReadoutApplications for all readout groups defined in a readout map.

Additional Python Utilities

`assets.py`

Read the DUNE-DAQ asset file database and return a path to a referenced asset file

`generate.py`

A collection of methods to generate segments and sessions.

`generate_hwmap.py`

Create a set of DetectorToDaqConnection objects, GeoIDs, and streams for the given number of links and applications.

`utils.py`

Utilities for parsing OKS databases. Currently contains an include file search routine.

20. Pre-2012 Release-Note Archaeology (Wayback Machine) — Summary

The original CMT-era release-notes site `pcatd12.cern.ch` is dead (direct fetch: transport error; browser fetch: HTTP 500 'proxy 590 UPSTREAM502'). 33 pages (13 `oks` package notes, 12 `config` package notes, 5 Doxygen `ConfigPackages` snapshots, 3 Javadoc pages of the Java `config` API) were rescued from the Internet Archive Wayback Machine (2011–2012 snapshots) — the individual raw snapshot dumps (~350 KB) are omitted here as boilerplate; the compiled findings are:

5. Observations

• The oldest OKS release note preserved here is `tdaq-01-01-00` (2011): it documents the `$(FOO)` environment-variable syntax change for filenames (`${FOO}` -> `$(FOO)`), the query heap-allocation fix, and new `oks_dump` options `--files-only`, `--class`, `--query`.

• `config` release notes cover `tdaq-01-01-00` .. `tdaq-03-00-00`; the last archived OKS note is `tdaq-04-00-00`. Later notes (05-00-00+) were never archived on this site (verified via CDX with prefix filters); coverage continues in the GIT-era `ooks` repo release notes (see 03-cern-gitlab.md).

• Together the archived pages and the GIT-repo notes document: `tdaq-01-01-00` .. `tdaq-04-00-00` (archived, 2011-2012) => git-era `oks-02-07-02` (2018) .. `oks-08-04-00` (2024).

• The `config` release notes show the pre-OKS era: the `Configuration`/`ConfigObject` APIs were the DB abstracted config data interface that OKS replaced/absorbed; the DAL (see 02-dune-dal.md) was introduced on top of it.

21. Access-Status Ledger for This Harvest

Access-status log for the OKS/DAL documentation extraction

> Status as of 2026-08-08. Per-source result of each access attempt, with exact error messages where access was blocked.

Sources accessed successfully

| Source | What was retrieved | Status |

|--------|--------------------|--------|

| https://github.com/DUNE-DAQ/oks (develop) | Full clone; schema/data + README + docs | OK (git clone) |

| https://github.com/DUNE-DAQ/dal (develop) | Full clone; code + docs + schemas; documented in 02 | OK (git clone) |

| https://gitlab.cern.ch/atlas-tdaq-software/oks | Full clone (HEAD 2026-07-09); all files embedded in 03 | OK (git clone) |

| https://gitlab.cern.ch/atlas-tdaq-software/oks_utils | Full clone (HEAD 2025-08-18); examples, tutorial, help pages | OK (git clone) |

| https://gitlab.cern.ch/atlas-tdaq-software/swrod | Full clone (HEAD 2026-08-04) | OK (git clone) |

| https://gitlab.cern.ch/atlas-tdaq-software/oks2coral | Full clone (HEAD 2021-03-02) | OK (git clone) |

| https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/ | DBE main page, dbe_main, schemaeditor pages | OK |

| https://dune-daq-sw.readthedocs.io/en/latest/packages/daqconf/ | daqconf package page | OK |

| https://web.archive.org/cdx/search/cdx (Wayback CDX API) | Full index of pcatd12.cern.ch; used to locate release notes | OK |

| https://web.archive.org/web/<ts>id_/<url> (Wayback snapshots) | 33 archived pages of pcatd12.cern.ch (oks/config release notes, doxygen, javadoc) | OK |

Sources blocked or partially blocked

| Source | Attempted | Result |

|--------|-----------|--------|

| https://pcatd12.cern.ch/releases/ (live site) | direct fetch | Transport error (connection refused) |

| https://pcatd12.cern.ch/cmt/releases/download/tdaq-02-00-00/RELEASE_NOTES.html (and other tdaq-* versions) | Direct fetch | HTTP 500 "Detected a session error / proxy 590 UPSTREAM502: 0 bytes" (via Apify rag-web-browser; 1 item returned, no content). Site is dead. |

| https://dune-daq-sw.readthedocs.io/en/latest/packages/dal/ | Direct fetch | HTTP 404 - all `packages/dal/*` URLs (README, DalReader, DalWriter, ConfigVersion, DAL-schema, DAL-test-schema, RELEASE_NOTES) are gone; the dal package was removed from the docs project and links from the dal GitHub README are dead. |

| https://dune-daq-sw.readthedocs.io/en/latest/packages/dbe/dbe/ (DbeRoo subtree) | Direct fetch | HTTP 404 for the deep subtree pages (dbe/dbe/... lower-level pages); only the top-level dbe pages are available. |

Notes

- All WayBack fetches used the `id_` flag (raw content) and passed a Mozilla UA; CDX queries with `matchType=domain`, `fl=original,timestamp`, `collapse=urlkey`, `filter=statuscode:200` worked best.

• The oldest OKS release-notes pages available in the archive: `nightly/oks/doc/RELEASE_NOTES.tdaq-01-01-00.html` (snapshot 20111107082403) .. `tdaq-04-00-00` (20111107082709); config package notes `tdaq-01-01-00` .. `tdaq-03-00-00` (snapshots 20111106134653-20111106134818). No `tdaq-05-00-00` or later notes ever archived on this site (CDX prefix filter confirmed); that era is covered by the GIT-era release notes (see 03-cern-gitlab.md).

• Raw HTML snapshot files are preserved under `output/extracts/pcatd12/` bound to the timestamps listed in `output/05-release-notes.md`.